Federal Urdu University of Arts, Sciences and Technology

Department of Computer Science

SkillKart

An AI-Powered Freelance Marketplace Platform

Final Year Project Report

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science in Computer Science (BSCS)

Submitted by

Name	Registration No.
Muhammad Hassan Siddiq	19252020
Syed Sheeryar Uddin	19252082
Alisha Binte Ovais	19252052

Project Supervisor

Sir Siddiq

Session: 2022 – 2026 **Submission date:** *[Month, Year]*

Certificate of Approval

This is to certify that the project titled **"SkillKart — An AI-Powered Freelance Marketplace Platform"** has been carried out by the group members listed above under the supervision of Sir Siddiq, in partial fulfilment of the requirements for the degree of **Bachelor of Science in Computer Science** at Federal Urdu University of Arts, Sciences and Technology.

The project has been examined and is approved for submission.

_____	_____
Project Supervisor	**Head of Department**
Sir Siddiq	Sir Siddiq
_____	_____
Internal Examiner	**External Examiner**

Declaration

We hereby declare that this project report titled **"SkillKart — An AI-Powered Freelance Marketplace Platform"** and the software system it documents are the result of our own work, carried out under the supervision of Sir Siddiq. All external libraries, services, and reference material used in this project have been duly acknowledged and cited. No portion of this work has been submitted in support of an application for any other degree or qualification at this or any other institution.

We further declare that the system described in this report is a functioning software implementation, and that the statistics, architecture, and behaviour reported herein reflect the actual state of the delivered codebase.

_____	_____	_____
Muhammad Hassan Siddiq	Syed Sheeryar Uddin	Alisha Binte Ovais
19252020	19252082	19252052

Dedication

To our parents and families, whose patience and support carried us through the long months of design, development, and testing; and to the freelancers — students, graduates, and self-taught professionals — whose determination to build a career on their own skills inspired this project, in the hope that a fairer and more trustworthy marketplace can give them a better start.

Table of Contents

1. Abstract 1
2. Acknowledgements 2
3. Introduction 3
    3.1 Overview 3
    3.2 Purpose 3
    3.3 Motivation 4
    3.4 Development Approach 4
    3.5 Report Structure 4
4. Problem Statement 5
    4.1 Freelancer Discovery 5
    4.2 Lack of AI Assistance 5
    4.3 Payment Workflow and Trust 5
    4.4 User and Skill Verification 5
    4.5 Contract and Dispute Management 5
    4.6 Summary 6
5. Objectives 7
6. Scope and Requirements 8
    6.1 In Scope 8
    6.2 User Roles 8
    6.3 Functional Requirements 9
    6.4 Non-Functional Requirements 11
    6.5 Assumptions and Constraints 12
    6.6 Out of Scope (Current Phase) 12
7. Literature Review 13
    7.1 The Online Freelance Economy 13
    7.2 Fiverr 13
    7.3 Upwork 13
    7.4 Freelancer.com 14
    7.5 Comparative Analysis 14
    7.6 How SkillKart Differs 14
8. Proposed Solution 15
    8.1 A Unified Dual-Model Marketplace 15
    8.2 An Integrated AI Layer 15
    8.3 Secure Escrow Payments 15
    8.4 Trust, Verification, and Oversight 16
    8.5 Native Mobile Access 16
    8.6 Design Principles 16
9. System Architecture 18

9.1 Architectural Overview 18
9.2 The React Frontend 19
9.3 The Express Backend 19
9.4 The MongoDB Database 21
9.5 The Authentication Subsystem 21
9.6 The Admin Panel Subsystem 21
9.7 Integration Subsystems 21
9.8 The Escrow Services 22
9.9 Scheduled Jobs 22
9.10 Deployment Architecture 23
9.11 Architectural Trade-offs 25
10. Database Design 26
10.1 Overview 26
10.2 Collections by Domain 26
10.3 Key Design Decisions 28
10.4 Key Relationships 29
10.5 Status Vocabularies and State Machines 33
11. Technologies Used 35
11.1 Frontend 35
11.2 Backend 35
11.3 Database 36
11.4 Authentication and Security 36
11.5 Email Delivery 36
11.6 Payments 36
11.7 Artificial Intelligence 37
11.8 File Storage and Media 37
11.9 Push Notifications 37
11.10 Mobile 37
11.11 Discoverability 37
11.12 Tooling and Operations 37
11.13 Why This Stack 38
12. Features 39
12.1 Authentication and Account Security 39
12.2 Freelancer Onboarding and Profile 40
12.3 Client Onboarding and Profile 41
12.4 Skill Verification 41
12.5 Skill Gating and the Apply Gate 44
12.6 Skill Taxonomy and Discovery 44
12.7 Jobs 45
12.8 Gigs 45
12.9 Gig Orders and Custom Offers 45
12.10 Proposals 46

12.11 Contracts and Lifecycle 47
12.12 Fixed-Price Milestone Escrow 48
12.13 Hourly Wallet-Lock Escrow 51
12.14 Deliveries and Revisions 52
12.15 Wallet, Payments and Withdrawals 52
12.16 Reviews and Ratings 52
12.17 Disputes 52
12.18 Messaging 53
12.19 Notifications 53
12.20 Transactional Email 54
12.21 Public Website and Discoverability 54
12.22 Dashboards and Navigation 54
12.23 Reporting Content 55
13. AI Features 56
13.1 AI Capabilities 56
13.2 Matchmaking: A Deterministic Engine 57
13.3 The Provider Router 57
13.4 Resilience: AI Failure Never Crashes the Application 59
13.5 Abuse Controls 60
13.6 What the Skill Test Asks of the AI 60
13.7 What Is Not an LLM 60
13.8 Future AI Expansion 60
14. Admin Panel 61
14.1 Design Rationale 61
14.2 Authentication and Sessions 63
14.3 Modules 63
14.4 Accountability and the Audit Log 64
14.5 Platform Settings 64
14.6 Administrative Surfaces Outside the Panel 65
14.7 Exports 65
15. Mobile Applications 66
15.1 Overview 66
15.2 One Codebase, Three Build Targets 66
15.3 Platform Integration 67
15.4 Native-Specific Engineering 68
15.5 Photos and Files 70
15.6 Build and Distribution 71
15.7 Verification and Limitations 72
16. Security 73
16.1 Authentication 73
16.2 Authorisation (RBAC) 73
16.3 Input Validation and Injection Protection 74

16.4 Transport and Headers 74
16.5 Abuse Prevention 75
16.6 Data Protection 75
16.7 Financial Integrity 76
16.8 Accountability and Monitoring 77
16.9 Resilience as a Security Property 77
16.10 Limitations and Hardening Roadmap 77
17. Workflow 78
17.1 Registration and Onboarding 78
17.2 Login and Session 78
17.3 Skill Verification 80
17.4 Hiring on a Fixed-Price Job 80
17.5 Working on an Hourly Contract 81
17.6 Buying a Gig or Accepting a Custom Offer 81
17.7 Delivery, Revision, and Release 82
17.8 Disputes 82
17.9 Withdrawing Earnings 82
17.10 Reviews 83
17.11 Administrative Moderation 83
17.12 Where the Money Goes: Worked Examples 83
18. Testing 85
18.1 Approach 85
18.2 Manual and Exploratory Testing 85
18.3 Real-Browser End-to-End Verification 85
18.4 Scripted Backend Checks 86
18.5 Test Data: the Demo Seeder 87
18.6 Resilience Testing 87
18.7 Interface and Responsiveness Testing 88
18.8 Performance Considerations 88
18.9 Test Gaps 88
19. Results 89
19.1 Achievement of Objectives 89
19.2 System Statistics 89
19.3 Requirements Coverage 90
19.4 Outcome 90
19.5 System Screenshots 90
19.6 Known Limitations 98
20. Challenges Faced 100
20.1 Multi-Provider AI Routing and Resilient Graceful Degradation 100
20.2 Concurrency and Financial Correctness Without Multi-Document Transactions 100
20.3 Session Integrity and Refresh-Token Family Invalidation 100
20.4 Edge AI Proctoring and Computer Vision on Android WebViews 101

20.5 Cross-Origin Cookie Security and Proxy Trust in Distributed Deployment 101
20.6 Transactional Email Reliability Under Cloud SMTP Egress Restrictions 101
21. Future Enhancements 103
21.1 Quality and Operations 103
21.2 Product Features 103
21.3 AI 103
21.4 Trust and Security 104
22. Conclusion 105
23. References 106
24. Appendix 108
Appendix A — API Summary 108
Appendix B — Folder Structure 109
Appendix C — Environment Variables (without secrets) 109
Appendix D — Major Screen & Interface Verification Matrix 110
Appendix E — Database Collections 111
Appendix F — Running the Project 112

List of Figures

Figure 6.1 — Use-case overview: roles and the capabilities they use 9
Figure 9.1 — System architecture 18
Figure 9.2 — Request lifecycle through the middleware pipeline and the layered backend 20
Figure 9.3 — Deployment architecture 24
Figure 10.1a — Entity relationships: identity and profiles 30
Figure 10.1b — Entity relationships: marketplace 31
Figure 10.1c — Entity relationships: contracts, escrow and payments 31
Figure 10.1d — Entity relationships: communication, reviews and withdrawals 32
Figure 10.1e — Entity relationships: skills, verification and reports 33
Figure 12.1 — Login, two-factor authentication and token lifecycle 40
Figure 12.2 — Skill-test flow: level choice, proctored test, grading and verification 43
Figure 12.3 — The apply gate: eligibility is checked before the proposal form opens 44
Figure 12.4 — Life cycle of a gig order, from requirements to completion 46
Figure 12.5 — Life cycle of a proposal and the effect of accepting it 47
Figure 12.6a — Contract state machine 48
Figure 12.6b — Milestone state machine (fixed-price contracts) 48
Figure 12.7 — Fixed-price milestone funding, delivery and release 50
Figure 12.8 — Hourly escrow: lock, draw-down, top-up and refund 51
Figure 12.9 — Dispute handling for a fixed-price milestone 53
Figure 12.10 — Interaction map of the client and freelancer areas 55
Figure 13.1 — The AI provider router: fallback, validation and the 503 path 59
Figure 14.1 — Admin login: credentials, emailed one-time code, session cookie 62
Figure 15.1 — One codebase, three build targets 67
Figure 15.2 — Mobile social sign-in through the system browser and a deep link 69
Figure 15.3 — Push notification path from an event to the phone 70
Figure 15.4 — Native Android application: sign-in, dashboard, job details and navigation drawer 71
Figure 17.1 — Registration, onboarding and first sign-in 79
Figure 17.2 — End-to-end workflow of a fixed-price engagement 80
Figure 17.3 — Hourly engagement: lock, bill, top up, end 81
Figure 17.4 — Gig order and custom-offer workflow 82
Figure 17.5 — Withdrawal by Stripe Connect payout 83
Figure 19.1 — Public home page 91
Figure 19.2 — Browse Gigs marketplace 91
Figure 19.3 — Gig detail with tiered packages and requirements 92
Figure 19.4 — Browse Projects and open client postings 92
Figure 19.5 — Job detail and proposal submission 93
Figure 19.6 — Client post-a-job wizard 93
Figure 19.7 — Client reviewing received proposals 94

Figure 19.8 — Client My Projects with milestone escrow status 94
Figure 19.9 — Freelancer dashboard and verification center 95
Figure 19.10 — Freelancer active contracts, timer, and work delivery 95
Figure 19.11 — Client wallet: balance, escrow, and transaction history 96
Figure 19.12 — Freelancer wallet and earnings overview 96
Figure 19.13 — AI-powered project cost estimator 97
Figure 19.14 — Interactive API documentation (Swagger UI) 97
Figure 19.15 — Native Android application: client mobile workspace 98
Figure 19.16 — Native Android application: freelancer mobile workspace 98

List of Tables

Table 4.1 — Problems identified and the SkillKart response 6
Table 6.1 — User roles 8
Table 6.2 — Functional requirements: accounts and verification 9
Table 6.3 — Functional requirements: marketplace 10
Table 6.4 — Functional requirements: engagement and payments 10
Table 6.5 — Functional requirements: communication 11
Table 6.6 — Functional requirements: administration and platform 11
Table 6.7 — Non-functional requirements and how they are met 11
Table 7.1 — Comparison of SkillKart with existing freelance marketplaces 14
Table 9.1 — Size of the code base 18
Table 9.2 — Services that move money 22
Table 9.3 — Scheduled jobs 22
Table 10.1 — Marketplace, profile and communication schemas 26
Table 10.2 — Financial escrow, verification and governance schemas 27
Table 10.9 — Principal relationships between collections 29
Table 10.10 — Status vocabularies and the transitions that occur 33
Table 12.1 — Authentication parameters 40
Table 12.2 — Skill-test levels 41
Table 12.3 — Fixed-price and hourly escrow compared 51
Table 12.4 — Platform settings that govern money 52
Table 12.5 — Dispute resolution outcomes 52
Table 12.6 — Notification events 54
Table 13.1 — LLM-backed AI capabilities 56
Table 13.2 — Matchmaking score components 57
Table 13.3 — AI providers 58
Table 14.1 — Admin panel modules and their capabilities 63
Table 14.2 — System settings 64
Table 15.1 — Capacitor plugins and their use 67
Table 16.1 — Rate limits 75
Table 17.1 — Worked examples of the three escrow models 84
Table 18.1 — Defects found through real-browser testing 85
Table 18.2 — Fixed demo accounts 87
Table 19.1 — Achievement of the project objectives 89
Table 19.2 — System statistics 89
Table A.1 — API route groups 108
Table C.1 — Environment variable groups 109
Table D.1 — Interface and screen verification matrix 111

List of Abbreviations

Abbreviation	Meaning
AES	Advanced Encryption Standard
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package (installable Android application file)
BSCS	Bachelor of Science in Computer Science
CMS	Content Management System
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
CV	Curriculum Vitae (résumé)
FCM	Firebase Cloud Messaging
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IP	Internet Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MERN	MongoDB, Express, React, Node.js
OAuth	Open Authorization (delegated sign-in)
OTP	One-Time Password
OWASP	Open Worldwide Application Security Project
PDF	Portable Document Format
PKR	Pakistani Rupee
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
SEO	Search Engine Optimisation
SHA	Secure Hash Algorithm
SME	Small and Medium-sized Enterprise
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application
TLS	Transport Layer Security
TTL	Time To Live (automatic expiry of a record)
UI	User Interface
URL	Uniform Resource Locator

1. Abstract

The freelance economy has become a significant part of the global digital workforce, particularly for students and recent graduates seeking flexible, skill-based income. Existing freelance platforms, however, present recurring challenges: freelancer discovery is inefficient, intelligent assistance is minimal, payment security is a persistent concern, and the verification of skills is weak. These barriers are especially acute for newcomers, who lack the experience to write competitive proposals, price their work correctly, and prove their skills to a stranger.

SkillKart is an AI-powered, dual-sided freelance marketplace built to address these problems in a single, coherent platform. It combines the two dominant freelancing models — pre-packaged **Gigs** (Fiverr-style) and custom **Jobs with Proposals** (Upwork-style) — and adds **custom offers** made inside a chat conversation. It augments them with an integrated artificial-intelligence layer, a wallet-native escrow payment system, layered verification built around a proctored skill test, and a comprehensive administrative panel, and it is delivered both as a responsive web application and as two native Android applications built from the same codebase through Capacitor.

The system is implemented on the **MERN stack** (MongoDB, Express.js, React, Node.js). The frontend is a React 19 single-page application built with Vite; the backend is a layered Express REST API backed by MongoDB through Mongoose, comprising 40 data models, 26 route modules, and 317 endpoints in about 118,000 lines of code. Fourteen AI capabilities — a conversational assistant, cost estimation, proposal drafting, brief suggestion, skill validation, skill-test generation and grading, CV analysis, and others — are delivered through a provider router that fails over automatically across Gemini, DeepSeek, Qwen, Groq, Claude, and a local Ollama model, so that an AI outage never crashes the platform. Freelancer matchmaking is handled separately by a deterministic, rules-based scoring engine. Payments use a per-milestone escrow for fixed-price work, a balance-lock escrow with automatic billing for hourly work, and a Stripe-backed deposit flow for gig orders, with Stripe also handling wallet top-ups and freelancer payouts. Skill verification is offered at three levels (Easy, Intermediate, and Expert) and is proctored by an on-device face-presence check and a browser guard; verified skills gate which jobs a freelancer may apply for and which gigs they may publish. Security rests on rotating JSON Web Tokens, role-based access control, one-time-password verification, input validation, rate limiting, bot detection, and an audit log.

The completed system delivers the full engagement lifecycle — from registration and verification, through job posting, proposals, and gig orders, to escrow-secured delivery, reviews, disputes, and withdrawals — under the supervision of a dedicated, isolated 17-module admin panel. All project objectives were achieved and demonstrated on realistic, seeded data. This report documents the motivation, design, implementation, testing, results, and limitations of the SkillKart project.

Keywords: Freelance Marketplace, MERN Stack, Artificial Intelligence, Escrow Payments, Skill Verification, Role-Based Access Control, Capacitor, MongoDB, React.

2. Acknowledgements

We would like to express our sincere gratitude to our project supervisor, Sir Siddiq, whose guidance, feedback, and encouragement were invaluable throughout the development of SkillKart. Their insights helped us refine both our technical decisions and our understanding of what it takes to build a production-quality software system.

We are thankful to the Department of Computer Science, Federal Urdu University of Arts, Sciences and Technology for providing the academic environment, resources, and structured evaluation that shaped this project.

We also thank the developers and maintainers of the open-source technologies on which SkillKart is built — including React, Node.js, Express, MongoDB, Capacitor, MediaPipe, and the many supporting libraries — as well as the providers of the AI, payment, email, and notification services integrated into the platform.

Finally, we thank our families and peers for their continuous support and patience during the many months of design, development, and testing.

Muhammad Hassan Siddiq, Syed Sheeryar Uddin, Alisha Binte Ovais

3. Introduction

3.1 Overview

SkillKart is a web-based and mobile-ready, AI-powered freelance marketplace that connects **clients** who need work completed with **freelancers** who offer their skills and services. Unlike a simple job board, SkillKart manages the *entire* engagement lifecycle: discovery, negotiation, contracting, secure payment, delivery, feedback, and dispute resolution. Artificial intelligence assists users at each stage, and a dedicated administrative layer oversees the platform.

The platform deliberately unifies the two most successful freelancing paradigms:

- **Gigs** — freelancers publish pre-packaged service offerings with tiered pricing (Basic / Standard / Premium), which clients can purchase directly. This is the model popularised by Fiverr.
- **Jobs and Proposals** — clients post custom job requirements and freelancers submit competitive proposals. This is the model popularised by Upwork.

By supporting both, SkillKart accommodates a wide range of work, from quick, standardised tasks to large, custom engagements that are split into milestones or billed by the hour. A third path joins the two: a freelancer can send a **custom offer** inside a chat conversation, which the client can accept to start paid work immediately.

The system is built on the **MERN stack** (MongoDB, Express.js, React, Node.js). A single React codebase is delivered three ways: as a responsive web application, as a native Android application for clients and freelancers, and as a separate native Android application dedicated to platform administrators.

3.2 Purpose

The purpose of SkillKart is to lower the barriers that make freelancing difficult, particularly for students and new entrants, while raising the level of trust and security for every participant. Specifically, the platform aims to:

1. **Simplify discovery** — helping clients find suitable talent and freelancers find relevant work through structured categories, search, and rules-based matchmaking.
2. **Provide intelligent assistance** — using AI to help freelancers write proposals, price their work, validate their skills, and build their profiles from a CV.
3. **Guarantee payment security** — holding funds in escrow so that freelancers are assured of payment and clients are assured of delivery.
4. **Establish trust** — through layered verification (email one-time passwords, proctored AI-generated skill tests, and identity checks) and a transparent two-way review system.
5. **Enable oversight** — giving administrators the tools to moderate content, resolve disputes, monitor the system, and analyse platform activity, with every sensitive action recorded in an audit trail.

3.3 Motivation

The freelance economy has become a significant part of the global digital workforce. For students and recent graduates in particular, it offers flexible, skill-based income without the need for a traditional employer. Yet the same students are the people least equipped to succeed on the large international platforms. They lack a review history, they are unsure how to price their work, they struggle to write a proposal that stands out among hundreds, and they have no way to prove their skills to a stranger. On the client side, small businesses and individuals worry about paying in advance for work that may never arrive.

Three observations motivated this project:

- **The trust problem is structural, not cosmetic.** A marketplace in which money can be lost, or skills misrepresented, will not retain either side. Escrow, verification, and dispute handling therefore had to be designed into the core of the system rather than added afterwards.
- **AI is most useful where users are least experienced.** Drafting a proposal, estimating a budget, or parsing a CV are tasks where an assistant gives a newcomer a real advantage — but only if the assistant is dependable. Free-tier AI services are rate-limited and intermittently unavailable, so the platform had to treat AI as an enhancement that can fail without taking the application down with it.
- **A final-year project should be a real system.** The team set out to build something that behaves like a production service: real payment flows, real security controls, real administrative tooling, and real mobile applications — and to test it as such.

3.4 Development Approach

SkillKart was developed iteratively between January and October 2026, in a Git repository that records more than 300 commits. Work proceeded in feature increments — authentication and onboarding first, then the marketplace and contract lifecycle, then payments and escrow, then the AI layer, the admin panel, and finally the mobile applications — followed by repeated hardening passes that audited the system for money-correctness, validation gaps, and usability defects.

A distinguishing practice was **verification in the running application**: each significant change was exercised end to end in a real browser (and, for the mobile build, on an Android emulator) rather than being judged only by reading code. Several of the most instructive defects described in Chapter 20 were found this way, because they were invisible to static inspection.

3.5 Report Structure

This report is organised as follows. Chapters 4–6 define the problem, the objectives, and the scope and requirements. Chapter 7 reviews existing platforms, and Chapter 8 presents the proposed solution. Chapters 9–11 describe the system architecture, database design, and technologies. Chapters 12–16 describe the platform's features, AI capabilities, administrative panel, mobile applications, and security model. Chapter 17 walks through the complete workflows. Chapters 18–20 cover testing, results, and the challenges encountered, and Chapters 21–22 discuss future work and conclusions. References and appendices follow.

4. Problem Statement

Freelance marketplaces have matured over the past decade, yet several structural problems persist — problems that disproportionately affect new freelancers and small clients. SkillKart is designed directly against the following five challenges.

4.1 Freelancer Discovery

On large platforms, quality talent is often buried beneath thousands of profiles. Search and categorisation are frequently keyword-based and shallow, making it hard for clients to find the right person and for capable-but-new freelancers to be seen. Without intelligent matching, discovery is inefficient for both sides: clients spend time sifting, and newcomers never surface.

4.2 Lack of AI Assistance

New freelancers face a steep learning curve. Writing a compelling proposal, pricing a project competitively, and presenting skills professionally are non-trivial tasks that existing platforms largely leave to the user. There is little built-in, intelligent guidance to help a newcomer succeed, and what AI tooling exists tends to be bolted on rather than woven into the workflow.

4.3 Payment Workflow and Trust

Payment is the single biggest source of anxiety in freelancing. Freelancers fear completing work and not being paid; clients fear paying and receiving poor or no work. A robust escrow mechanism — where funds are held securely until both parties are satisfied — is essential, but it must be implemented correctly and transparently. It must also handle the awkward cases: a client who goes silent after receiving work, a long project that cannot sensibly be paid for in a single lump, an hourly engagement with no fixed price, and a dispute that needs a fair resolution path.

4.4 User and Skill Verification

Trust depends on verification. Fake profiles, exaggerated skills, and unverified identities erode confidence in a marketplace. Many platforms offer only basic verification, and rigorous skill testing is often paid or absent. A self-declared skill list tells a client very little; a test that is taken without supervision tells them little more, because it can be taken by someone else or with the answers open in another window. A trustworthy platform needs layered verification of email, skills, and identity, and it needs the skill verification itself to be hard to cheat.

4.5 Contract and Dispute Management

A freelance engagement is a contract with a defined lifecycle — start, delivery, revision, completion — and occasionally disputes. When this lifecycle is not modelled explicitly, engagements become ambiguous and conflicts are hard to resolve fairly. Comprehensive contract tracking and a structured, administrator-mediated dispute process are necessary but frequently underdeveloped.

4.6 Summary

In short, existing solutions leave gaps in **discovery, assistance, payment trust, verification, and lifecycle management**. SkillKart addresses all five within one integrated, AI-augmented, admin-supervised platform. Table 4.1 maps each problem to the part of the system that answers it.

Table 4.1 — Problems identified and the SkillKart response

Problem	SkillKart response	Chapter
Poor freelancer discovery	Structured taxonomy, public search and filters, rules-based matchmaking engine	12, 13
Little intelligent assistance	Buddy assistant, AI proposal drafting, cost estimator, brief auto-fill, CV analysis	13
Payment anxiety on both sides	Wallet-native escrow: per-milestone for fixed-price work, balance-lock for hourly work, auto-release	12, 17
Weak verification	Email OTP, proctored AI skill tests at three levels, skill-gated applying and publishing	12, 13
Ambiguous contracts and disputes	Explicit contract and milestone state machines, per-milestone disputes, admin resolution with audit log	10, 14

5. Objectives

The aim of the project is to design and implement a complete, secure, and intelligent freelance marketplace that supports both gig-based and proposal-based engagements and that is accessible from the web and from native mobile devices. The specific objectives are as follows.

1. **Build a complete dual-sided marketplace** supporting both Gigs (packaged services) and Jobs with Proposals (custom engagements), together with custom offers made in chat, with full freelancer and client experiences.
2. **Integrate artificial intelligence** to assist users — including a conversational assistant, AI cost estimation, AI proposal drafting, brief auto-fill, AI skill validation and test generation, and AI CV analysis — together with a rules-based freelancer matchmaking engine.
3. **Implement a secure escrow payment system** — a wallet-native escrow (per-milestone for fixed-price work, balance-lock for hourly work) plus a Stripe-backed flow for gig orders, with Stripe also handling wallet top-ups and freelancer payouts — including wallets, deposits, escrow release, platform-fee calculation, and withdrawals.
4. **Provide multi-factor verification** to establish trust — email OTP verification, proctored AI-generated skill tests at three levels (Easy, Intermediate, Expert), and skill-based gating of who may apply for or publish what.
5. **Develop a comprehensive administrative panel** with isolated authentication for moderating users, jobs, contracts, payments, disputes, and reviews, and for monitoring and analytics.
6. **Enforce strong security** throughout — JWT authentication with refresh-token rotation, role-based access control, password hashing, OTP, rate limiting, input validation and sanitisation, bot mitigation, and audit logging.
7. **Model the complete contract lifecycle** explicitly — as state machines — so that engagements are unambiguous, auditable, and safe.
8. **Ensure resilience and reliability**, particularly for third-party services (AI, payments, email, push), so that failures degrade gracefully rather than crashing the application.
9. **Deliver native mobile access** — package the same web codebase as native Android applications via Capacitor (a client/freelancer app and a separate, standalone admin app), with platform-level integration (camera, push notifications, offline detection, native back-button navigation, file sharing) rather than a bare web wrapper.

6. Scope and Requirements

6.1 In Scope

The following capabilities are within the scope of the project and are implemented.

- **User management:** registration, login with optional two-factor authentication, social sign-in, role-based onboarding, and profile management for freelancers and clients.
- **Verification:** email one-time-password verification, AI-generated and proctored skill tests at three levels, and skill-based gating of who may apply for work and publish gigs.
- **Marketplace:** job posting, gig publishing with tiered pricing, gig ordering, custom offers in chat, proposals, and search and browsing by category and skill.
- **Engagement lifecycle:** contract creation, milestone and hourly escrow, delivery and revision, time tracking, completion, and administrator intervention.
- **Payments:** a wallet for each user, Stripe top-ups and payouts, escrow, release, platform-fee handling, auto-release, and withdrawals.
- **Communication and feedback:** in-app chat with contact-information masking, notifications (in-app and push), transactional email, and a two-way review system.
- **Dispute resolution:** raising, tracking, and administrator resolution of disputes.
- **AI tools:** assistant, cost estimator, proposal drafting, brief suggestion, skill validation and tests, CV analysis, and rules-based matchmaking.
- **Administration:** a dedicated, isolated admin panel with moderation, finance, monitoring, audit logs, and analytics.
- **Mobile:** two native Android applications built from the same codebase.
- **Security and operations:** authentication, authorisation, input hardening, abuse prevention, and a demo data seeder.

6.2 User Roles

The platform defines six roles, of which four are used through a user interface.

Table 6.1 — User roles

Role	Who	Principal capabilities
Visitor / public viewer	An unauthenticated visitor, or a social-login user who has not yet chosen a role	Browse jobs, gigs, and freelancers; use the AI cost estimator and matchmaking; chat with Buddy (limited); register
Freelancer	A person offering skills	Build a profile; verify skills; publish gigs; apply to jobs; send custom offers; deliver work; log time; chat; withdraw earnings; review clients
Client	A person or business hiring	Post jobs; order gigs; review proposals and hire; fund milestones; approve work; top up the wallet; chat; review freelancers

Role	Who	Principal capabilities
Administrator	A platform operator (admin panel)	Moderate users, jobs, contracts, and reviews; resolve disputes; manage wallets and withdrawals; broadcast notifications; monitor and audit
Content editor	A CMS author (back-end role)	Create, update, and publish CMS pages through the API; no user interface exists yet
SuperAdmin	The bootstrap operator (marketplace credential)	Bypasses marketplace role checks; no user interface

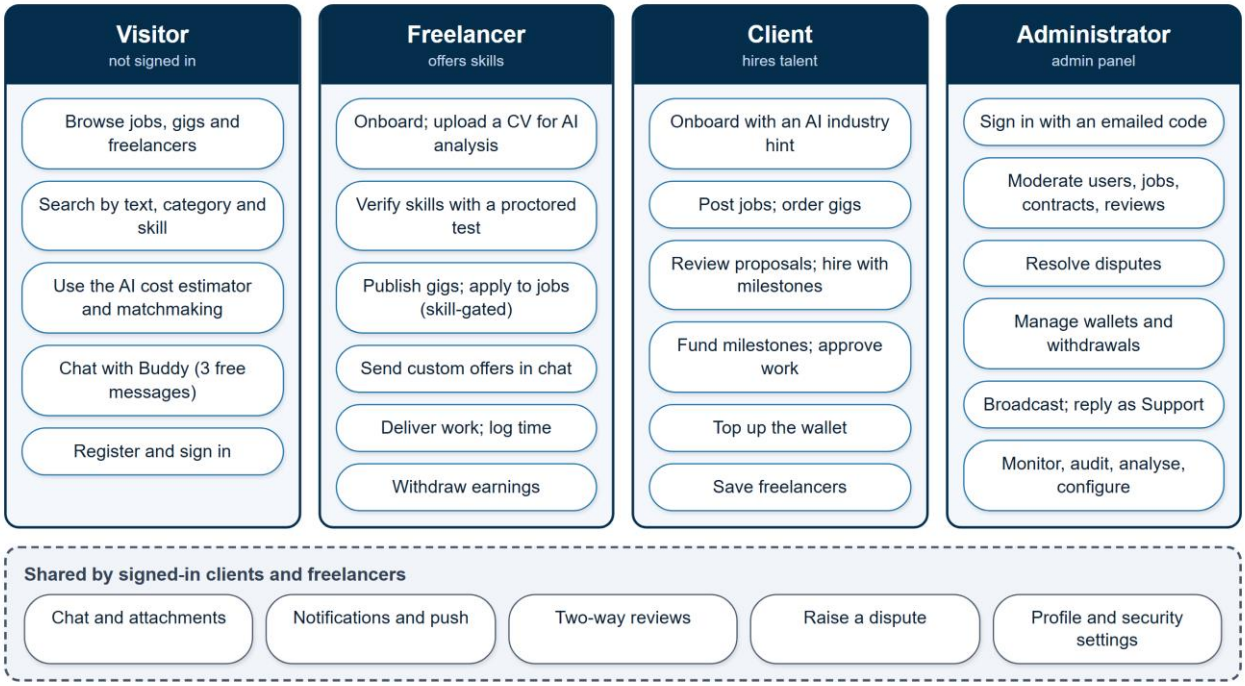


Figure 6.1 — Use-case overview: roles and the capabilities they use

Visitors, freelancers, clients, and administrators each have a distinct set of capabilities (Section 6.2).

6.3 Functional Requirements

The functional requirements are grouped by area. The last column names the section in which each is implemented.

Table 6.2 — Functional requirements: accounts and verification

ID	Requirement	Section
FR-A1	A visitor shall be able to register as a client or a freelancer with an email address and password	12.1
FR-A2	The system shall verify an email address with a one-time password	12.1
FR-A3	A user shall be able to enable two-factor login, sign in with a social provider, and reset a forgotten password	12.1
FR-A4	The system shall lock an account after repeated wrong passwords and shall revoke all sessions on password change	12.1, 16.1
FR-A5	A freelancer shall complete a guided onboarding, optionally with AI-assisted CV analysis	12.2

ID	Requirement	Section
FR-A6	A client shall complete a guided onboarding, with an AI industry recommendation	12.3
FR-A7	A freelancer shall be able to verify a skill by taking a proctored AI-generated test at the Easy, Intermediate, or Expert level	12.4
FR-A8	The system shall restrict applying to jobs and publishing gigs to freelancers with a matching verified skill	12.5

Table 6.3 — Functional requirements: marketplace

ID	Requirement	Section
FR-M1	A client shall be able to post a job, save it as a draft, and publish it without a wallet balance	12.7
FR-M2	A freelancer shall be able to publish a gig with up to three pricing tiers	12.8
FR-M3	A client shall be able to order a gig by supplying written requirements and paying a deposit	12.9
FR-M4	A freelancer shall be able to send a priced custom offer in a conversation, which a client may accept	12.9
FR-M5	A freelancer shall be able to submit a proposal, including an AI-drafted cover letter	12.10
FR-M6	A client shall be able to accept a proposal, splitting a fixed price into milestones, or reject it with or without allowing resubmission	12.10
FR-M7	Users shall be able to browse and search jobs, gigs, and freelancers by text, category, and skill	12.6, 12.21
FR-M8	The system shall rank freelancers for a project brief	13.2

Table 6.4 — Functional requirements: engagement and payments

ID	Requirement	Section
FR-P1	The system shall lock the first milestone's amount from the client's wallet when a proposal is accepted	12.12
FR-P2	The system shall prevent work submission on an unfunded milestone	12.12
FR-P3	The system shall release a milestone on client approval, and automatically after a review window	12.12
FR-P4	The system shall lock a week's estimate for an hourly contract and bill logged time against it automatically	12.13
FR-P5	The system shall refund unused hourly escrow when a contract ends	12.13
FR-P6	A client shall be able to add money to their wallet through Stripe	12.15
FR-P7	A freelancer shall be able to withdraw earnings, automatically through Stripe Connect or by an administrator-approved request	12.15
FR-P8	The system shall deduct a configurable platform fee on release	12.12, 14.5
FR-P9	Either party shall be able to raise a dispute, which an administrator resolves	12.17
FR-P10	Both parties shall be able to review each other after completion	12.16

Table 6.5 — Functional requirements: communication

ID	Requirement	Section
FR-C1	Users shall be able to chat one-to-one, with attachments	12.18
FR-C2	The system shall mask off-platform contact details and block repeat offenders	12.18
FR-C3	The system shall notify users of significant events in-app and, on Android, by push	12.19, 15.4
FR-C4	The system shall send security-related and verification email	12.20
FR-C5	A signed-in user shall be able to report a profile, job, gig, or review to the moderation team	12.23

Table 6.6 — Functional requirements: administration and platform

ID	Requirement	Section
FR-D1	An administrator shall sign in through an isolated, two-step login	14.2
FR-D2	An administrator shall be able to moderate users, jobs, contracts, reviews, and chat	14.3
FR-D3	An administrator shall be able to inspect and adjust financial records, with every action audit-logged	14.3, 14.4
FR-D4	An administrator shall be able to monitor errors, API traffic, and email delivery	14.3
FR-D5	An administrator shall be able to review AI-suggested taxonomy entries	12.6
FR-D6	The platform shall be usable on Android through two native applications	Ch. 15

6.4 Non-Functional Requirements

Table 6.7 — Non-functional requirements and how they are met

ID	Quality	Requirement	How it is met
NFR-1	Security	Protect accounts, money, and data from common web attacks	Chapter 16: hashed credentials, rotating tokens, validation, sanitisation, rate limits, bot detection, audit trail
NFR-2	Financial integrity	Money shall never be created, lost, or applied twice	Atomic conditional updates, claim-before-pay, idempotent confirmations, check-before-write ordering (Section 16.7)
NFR-3	Reliability	A failing third-party service shall not take the platform down	Fail-open optional integrations, AI fallback chain, controlled 503, operational-error guard (Sections 13.4, 16.9)
NFR-4	Isolation	Administrative access shall be separate from marketplace access	Separate secret, cookie, middleware, and session store (Section 14.1)
NFR-5	Usability	New freelancers shall be able to onboard and succeed without outside help	Guided wizards, AI assistance, optional verification, in-place error messages
NFR-6	Portability	One codebase shall serve web and Android	Capacitor, with a single platform check (Chapter 15)
NFR-7	Maintainability	Business rules shall be defined in one place	Layered architecture and single choke points for money, notifications, and email (Section 8.6)

ID	Quality	Requirement	How it is met
NFR-8	Configurability	Operators shall be able to tune behaviour without code changes	Environment-driven AI order, email provider, proctoring strictness, and administrator-editable fee, release window, and withdrawal minimum
NFR-9	Deployability	The system shall run with minimal configuration and degrade gracefully	Only a database URI, two JWT secrets, and the API address are mandatory (Section 9.10)
NFR-10	Discoverability	Public content shall be crawlable	Live sitemap and `robots.txt` (Section 12.21)

6.5 Assumptions and Constraints

- **Free-tier AI services** are rate-limited and unreliable; the design assumes that any provider can fail at any time.
- **A single currency**, Pakistani Rupees, is used throughout, and Stripe processes card payments in it.
- **A single language** (English) is supported.
- The **deployment host blocks outbound SMTP**, which constrains how email can be sent.
- The **project was built by a small team within one academic year**, which favoured simple, well-understood solutions (polling, Context state, rules-based matching) over more elaborate ones.

6.6 Out of Scope (Current Phase)

The following are deliberately deferred and documented in Chapter 21.

- **Real-time WebSocket chat and video calls** — messaging currently uses efficient polling.
- **Embedding-based semantic search** — matchmaking is deterministic and rules-based.
- **Internationalisation** — multi-language support and multiple currencies.
- **An iOS application** — the codebase is portable, but a build was out of scope.
- **A multi-administrator role model** and an **automated test suite**.

7. Literature Review

To position SkillKart, we studied the three most prominent freelance marketplaces, reviewed published research on trust in online marketplaces, and identified where our platform differs.

7.1 The Online Freelance Economy

Online labour platforms have grown into a measurable part of the global labour market. Kässi and Lehdonvirta [21] developed the Online Labour Index to track the number of projects and tasks posted across the major English-language platforms over time, making the size and composition of the online gig economy — which spans software development, creative work, writing, and clerical tasks — observable for the first time. For a student or recent graduate, this is an attractive entry point to paid work. It is also a hard one to enter: work is found through a handful of large platforms, and on each of them success depends heavily on a reputation that a newcomer does not yet have.

A second strand of research explains why reputation and institutional safeguards matter so much. Resnick and Zeckhauser [36], studying eBay's feedback system, show that buyers and sellers who do not know each other rely on reputation signals to decide whether to transact. Pavlou and Gefen [22] extend this to the question of what makes strangers trust an online marketplace, finding that institutional mechanisms — feedback systems, escrow services, and guarantees — increase a buyer's trust in the community of sellers as a whole. Both findings bear directly on freelance marketplaces, where the buyer pays for something that does not yet exist. They motivated the two design commitments that run through SkillKart: money is held in escrow until work is accepted, and credibility is built from evidence (tested skills, completed work, two-way reviews) rather than self-description.

7.2 Fiverr

Fiverr [11] popularised the **Gig** model, in which freelancers publish fixed, packaged service offerings that buyers purchase directly. Its strengths are simplicity and speed of transaction: a buyer can choose a package, pay, and receive work without negotiating. Its limitations include relatively high service fees, a heavy dependence on a seller's own marketing and search ranking for visibility, and minimal intelligent assistance for sellers who are new to the platform. A new seller with no reviews competes against established sellers on the same search page.

7.3 Upwork

Upwork [12] is built around the **Job and Proposal** model, in which clients post projects and freelancers bid. It supports hourly and fixed-price contracts, time tracking, and escrow-style payment protection. Its strengths are contract flexibility and a mature payment system. Its limitations include a complex bidding process that disadvantages newcomers, a paid, credit-based mechanism for submitting proposals, and limited built-in guidance for writing a competitive proposal or pricing a project.

7.4 Freelancer.com

Freelancer.com [13] supports both models and a wide range of project types through a bidding and contest system. It is broad, but it is often criticised for a crowded, highly competitive bidding environment in which price dominates, and for inconsistent quality control and verification.

7.5 Comparative Analysis

Table 7.1 — Comparison of SkillKart with existing freelance marketplaces

Aspect	Fiverr	Upwork	Freelancer.com	SkillKart
Marketplace model	Gigs	Jobs and proposals	Both	**Gigs, jobs and proposals, and custom offers in chat**
Built-in AI assistance	Limited	Limited	Minimal	**Integrated: assistant, proposal drafting, cost estimation, brief auto-fill, CV analysis**
Skill verification	Basic	Tests (varies)	Basic	**Proctored AI-generated tests at three levels; verified skills gate applying and publishing**
Escrow payments	Yes	Yes	Yes	**Yes: wallet-native, per-milestone for fixed price, balance-lock for hourly, auto-release**
Dispute resolution	Yes	Yes	Yes	**Per-milestone disputes, admin-mediated, fully audit-logged**
Mobile applications	Yes	Yes	Yes	**Native Android client/freelancer app plus a separate admin app, from one codebase**
Target focus	General	General	General	**Student- and graduate-friendly, low-friction onboarding**

The comparison reflects the public positioning of each platform at the time of writing and is intended to show where SkillKart's design choices differ, not to claim that SkillKart replaces mature commercial services.

7.6 How SkillKart Differs

SkillKart's differentiation is not a single feature but an *integration philosophy*. AI assistance is woven through onboarding, discovery, bidding, and pricing rather than bolted on. Verification is layered and partly AI-driven, and the test that underpins it is proctored, so a verified skill carries real meaning. Payments, contracts, and disputes are unified in one state-machine model and supervised by a purpose-built admin panel with full audit logging. And every optional external dependency — AI providers, payment processor, email transport, push service, bot-detection — is isolated behind a service layer that degrades gracefully, so that the failure of a free-tier service never stops a freelancer from working or a client from paying. The result is a platform explicitly designed to lower the barrier for new freelancers while maintaining trust for clients.

8. Proposed Solution

SkillKart is proposed as a single, integrated platform that resolves the problems identified in Chapter 4 through five pillars, each described below, and that is built according to a small set of design principles that recur throughout the system.

8.1 A Unified Dual-Model Marketplace

By supporting both **Gigs** and **Jobs with Proposals** — and by letting a freelancer send a **custom offer** inside a conversation — SkillKart serves the full spectrum of freelance work. A freelancer can both publish standardised services and bid on custom projects; a client can both buy a ready-made service and post a bespoke requirement. All three routes end in the same place: a paid, tracked engagement whose state, money, and messages are managed uniformly (Chapters 10 and 12).

8.2 An Integrated AI Layer

AI is used to reduce friction rather than to decorate the interface:

- The **Buddy assistant** answers platform and freelancing questions conversationally.
- **AI proposal drafting** helps freelancers respond to jobs competitively.
- The **AI cost estimator** helps price projects using market context.
- **Brief auto-fill** infers a job's category, skills, and budget from a free-text description.
- **AI skill validation and test generation** verify capabilities objectively.
- **AI CV analysis** parses a résumé, scores it, and auto-fills the profile.
- **Matchmaking** ranks freelancers for a given project brief. Unlike the AI features above, this is a deterministic, rules-based scoring engine — it does not call an LLM provider, and so it cannot fail when AI is unavailable.

Crucially, the AI subsystem is engineered for **resilience** (Chapter 13): a prioritised chain of providers with automatic fallback, ending in a controlled `503` and a graceful interface message if every provider is down.

8.3 Secure Escrow Payments

The platform operates in Pakistani Rupees (PKR) and keeps client and freelancer money in internal **wallets**. Escrow works differently for each kind of engagement, so that each is protected in the way that suits it.

- **Fixed-price work** is paid by milestone. The moment a proposal is accepted, the first milestone's amount is debited from the client's wallet and locked — never left as an unfunded promise. A large project can be split into any number of sequential milestones (provided the amounts add up to the agreed price); each later milestone is funded only when the client is ready to start that phase, so capital is not locked for work that has not begun.
- **Hourly work** has no fixed price to lock, so a week's estimated cost (hourly rate multiplied by the freelancer's estimated weekly hours) is locked at hire. Each logged hour is then billed

automatically against that lock, and whatever is left is refunded to the client when the contract ends.

- **Gig orders** use a deposit-then-release flow processed through Stripe.
- **Auto-release.** If a client goes silent after a freelancer submits work, the milestone's escrow is released automatically after a configurable review window (default seven days) instead of leaving the freelancer unpaid indefinitely.
- **No gate at posting.** Posting a job carries no wallet-balance check: a client may publish with any balance, including zero, so the job is visible and can collect proposals. The wallet is checked, and funds locked, only when a proposal is accepted — the earliest moment at which real money must be committed.

When escrow is released, the platform fee (a default of 0.5% of the amount, adjustable by an administrator) is deducted and the remainder is credited to the freelancer's wallet, from which they can withdraw — automatically through Stripe Connect, or through a manually approved request. This protects both parties and gives disputes a clear financial basis.

8.4 Trust, Verification, and Oversight

Trust is built through layered verification, a transparent two-way review system, and a structured dispute process.

- **Identity** is anchored by email one-time-password verification and optional two-factor login.
- **Skills** are verified by AI-generated tests that the freelancer takes at one of three levels — Easy, Intermediate, or Expert — under proctoring: a camera check that cancels the attempt if no face is in view, and a guard against switching tabs or leaving full screen. The level taken is the rank earned, and verified skills gate which jobs a freelancer may apply for and which gigs they may publish.
- **Behaviour** is recorded in two-way reviews that feed profile ratings, and in an append-only contract history.
- **Oversight** is provided by an isolated admin panel with its own authentication, a seventeen-module management surface, and an immutable audit log of every sensitive action.

8.5 Native Mobile Access

The same React codebase is packaged, through Capacitor, as two Android applications: one for clients and freelancers, and a separate one for administrators. They are not wrapped websites: they integrate the camera, push notifications, the hardware back button, offline detection, and native file sharing (Chapter 15).

8.6 Design Principles

Five principles, applied consistently, explain many of the decisions described in later chapters.

1. **Server-side enforcement.** A rule that protects money or trust is enforced on the server, not merely implied by the interface. A freelancer cannot submit work to an unfunded milestone, and a skill gate is checked by the API, not only by a disabled button.
2. **Single choke points.** Each sensitive operation passes through exactly one function: all milestone money movement goes through the milestone service, every notification goes through one `notify()` function, and every outgoing email goes through one email service. This makes behaviour consistent and makes each operation auditable and testable in one place.
3. **Order of operations in place of transactions.** Because the system does not use multi-document database transactions, every money-moving flow validates before it writes, performs its guarded atomic update, and only then creates dependent records.
4. **Fail open for optional integrations.** Every third-party integration that is not essential — reCAPTCHA, push notifications, AI, an email provider — degrades quietly when it is unconfigured or unreachable, so that an outage can never lock all users out of the platform.
5. **Isolation of privilege.** The administrative system shares no tokens, cookies, secrets, or middleware with the marketplace, so a marketplace credential can never grant administrative access.

9. System Architecture

9.1 Architectural Overview

SkillKart follows a **decoupled client–server architecture**. A React single-page application (SPA) communicates with a stateless Express REST API over HTTPS, exchanging JSON, in the architectural style described by Fielding [23]. MongoDB is the single source of truth for all marketplace data. Every external integration — payments, AI, social login, email, file storage, push notifications, and bot detection — is isolated behind a dedicated service module, so that the core application is never tightly coupled to any one vendor and any optional service can be absent without stopping the system.

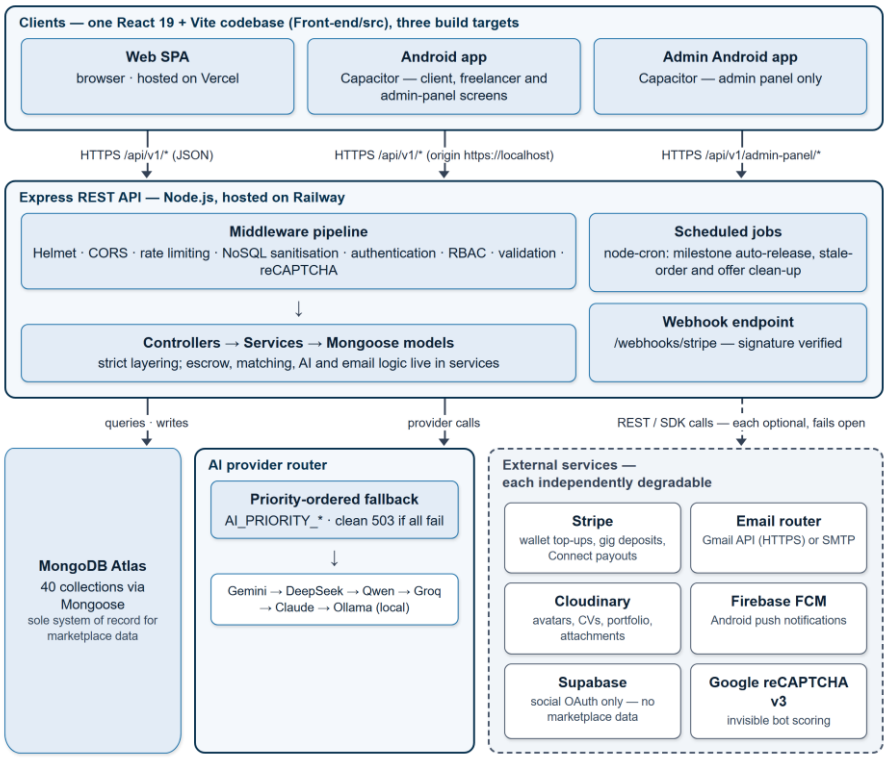


Figure 9.1 — System architecture

One React codebase is delivered as a web application and two Android applications; all of them talk to one Express API, which uses MongoDB as the only required service (Section 9.1).

The project is organised as a single repository with two independently run applications: `Backend/` (the Express API) and `Front-end/` (the React application, including the two Capacitor Android projects), together with `docs/`. Table 9.1 gives its size.

Table 9.1 — Size of the code base

Part	Files	Lines
Backend (`Backend/src`, JavaScript)	197	36,475
Frontend (JavaScript and JSX)	202	50,378
Frontend stylesheets (CSS)	76	31,086
Application source in total	**475**	**117,939**

9.2 The React Frontend

The frontend is a **React 19** SPA [4] built with **Vite 7** [5] and routed by **React Router 7**. Its source is organised by audience: `pages/public` (38 files), `pages/freelancer` (22), `pages/client` (11), `pages/adminPanel` (5), `pages/common` (2), and one legacy admin page, together with 57 reusable components and nine custom hooks.

State management uses the React Context API, with five providers — authentication, administrator authentication, client data, freelancer data, and notifications — combined with local component state. A heavier state library was judged unnecessary for an application whose server data is fetched per page.

One HTTP client. All server communication passes through a single Axios module that is wrapped with interceptors. The *request* interceptor attaches the access token and, for the four reCAPTCHA-protected endpoints, mints and attaches a reCAPTCHA token. It also **de-duplicates identical in-flight GET requests** so that components requesting the same data share one network call. The *response* interceptor, on a 401, refreshes the session through the rotating refresh token and replays the request once, or signs the user out; it recognises a blocked-account response, suppresses duplicate toasts, and shows the single "AI is temporarily out of service" notice for any failed request to an AI endpoint (Section 13.4). Each API resource has its own small module (24 in total) built on this client. The admin panel uses its own client, so no admin request can carry a marketplace token.

Role-based routing. `ProtectedRoute` guards every authenticated route tree: an unauthenticated visitor is redirected to the login page and returned to the page they wanted afterwards, and a user with the wrong role is redirected to an *unauthorised* page. Unknown paths go to a 404 page.

Performance and robustness. Twenty-two lower-traffic pages are loaded lazily, so that their code is fetched only when visited; the choice between the marketplace and admin route trees is itself lazy. Error boundaries wrap both route trees, and each section of the home page has its own boundary, so that one failing section cannot blank the page. Toasts use a small, dependency-free utility, and all console output goes through a logger that is silent in production builds, both conventions that arose from defects described in Chapter 20.

9.3 The Express Backend

The backend is a **Node.js** [2] and **Express 4** [3] application with a strict layered structure:

```
Request → Route → Middleware (limits, authentication, role checks, validation)
        → Controller (thin: validate and delegate)
        → Service (business logic)
        → Model (Mongoose) → MongoDB
```

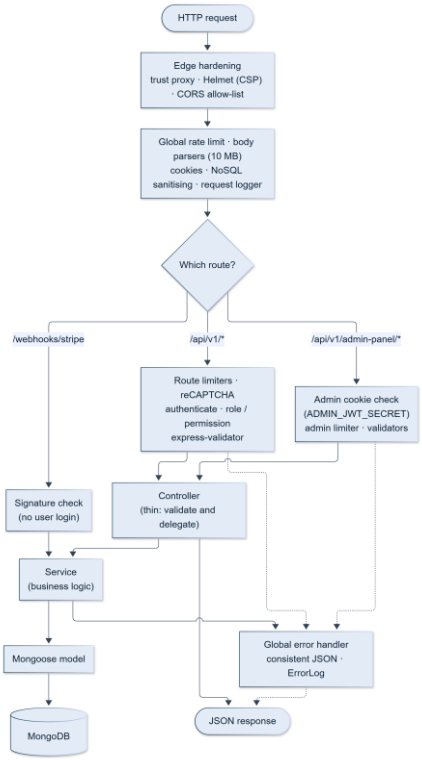


Figure 9.2 — Request lifecycle through the middleware pipeline and the layered backend

Authentication, validation and permission failures are routed to the centralised error handler, which returns a controlled response (Section 9.3).

It comprises **26 route modules** (25 under `/api/v1/*` plus the isolated admin panel) exposing **317 endpoints**, **35 controllers** (26 marketplace and nine admin-panel), **49 service files** (26 core modules, nine for the AI router, and fourteen for the email subsystem), **40 Mongoose models**, ten middleware modules, eleven validator modules, and fifteen utility modules. Controllers stay thin and delegate business logic to services; for example, `stripe.service`, `milestone.service`, `hourlyEscrow.service`, `gigOrder.service`, `matchmaking.service`, `taxonomy.service`, `notification.service`, and `email.service`.

Request pipeline. The order of the middleware matters, and is deliberate: trust proxy → Helmet → CORS → global rate limit → body parsers (which keep the raw body for webhook signature verification) → cookie parsing → NoSQL sanitisation → request logging → compression → routes → the admin-panel routes → the webhooks → a 404 handler → a single global error handler. That handler maps Mongoose validation errors, duplicate-key errors, JSON Web Token errors, cast errors, and CORS rejections to consistent JSON responses, forwards structured error data (for instance, the required and available amounts of an insufficient-balance error) so that the interface can react to it, hides stack traces outside development, and persists every error for the monitoring page.

Start-up. The server verifies its safety guards (Section 16.7), connects to MongoDB (and exits if it cannot), then initialises each optional subsystem in turn — Redis, Supabase, the AI router (with a real one-token call to the first working provider), and the email provider — logging and continuing past any that fail, and finally starts the scheduler and begins listening on all interfaces at the port

named in the environment. Redis is connected at start-up for future caching, but no feature uses it yet.

API documentation. Interactive Swagger (OpenAPI [26]) documentation is served at `/api/v1/docs`. It currently documents 38 operations across eleven route files (authentication, freelancer, client, public, job, proposal, custom offer, user, CMS, and the legacy administration routes); the 93 admin-panel endpoints are not yet annotated.

9.4 The MongoDB Database

MongoDB [1], accessed through Mongoose [6] and hosted on MongoDB Atlas [33], stores all persistent data in 40 collections (Chapter 10). Relationships are modelled with `ObjectId` references and enforced in the application layer. Collections carry indexes for the common query patterns, TTL indexes remove expired one-time codes, refresh tokens, and admin sessions on their own, and list endpoints are paginated on the server.

9.5 The Authentication Subsystem

Authentication is JWT-based, with rotating refresh tokens that are tracked server-side, per-request user verification, optional two-factor login, and optional social sign-in through Supabase (Sections 12.1 and 16.1). The admin panel has a **completely isolated** authentication flow, so that a marketplace token can never grant administrative access (Section 14.1).

9.6 The Admin Panel Subsystem

The admin panel is served under an isolated route namespace with its own provider, protected routes, HTTP client, and scoped stylesheet (`ap-` class prefix). It provides 17 management modules and is architecturally separated from the marketplace, so that no administrative capability can leak into the public application (Chapter 14).

9.7 Integration Subsystems

Every external service is reached through one service module, which hides the vendor and degrades gracefully when the service is absent.

AI provider router. A router presents a uniform interface and tries a prioritised chain of providers — Gemini, DeepSeek, Qwen, Groq, Claude, and a local Ollama model — with the order set by environment variables (Chapter 13).

Email provider router. Controllers and services call only `email.service`, never a provider. Underneath, a **registry** maps a provider name to a class and a **factory** selects one at start-up from the `EMAIL_PROVIDER` setting. Two providers exist: **SMTP** through Nodemailer, and the **Gmail API**, which sends over HTTPS using OAuth credentials for a dedicated platform mailbox [25]. If the Gmail credentials are missing or fail verification at start-up, the factory falls back to SMTP. To add a provider such as SES or Mailgun, one new class and one registry line suffice, with no change to any controller. The Gmail provider exists because the deployment host blocks outbound SMTP ports (Section 20.7).

Payments. `stripe.service` wraps Stripe [7] for Checkout sessions, Connect accounts, transfers, and payouts. With no key configured, or with `STRIPE_DUMMY=true`, every call is simulated so that the full payment flow can be demonstrated without real money. Simulated checkout-session identifiers are signed and tied to the user and the amount, so a client cannot fabricate one to credit a wallet.

File storage. Uploads pass through `upload.service` to Cloudinary [18]; no request handler touches the Cloudinary SDK directly, and old assets are cleaned up on a best-effort basis when they are replaced.

Push notifications. `push.service` wraps Firebase Cloud Messaging [17] and is called from the single `notify()` function (Section 15.4).

Social login and bot detection. Supabase [20] is used only to authenticate users through Google, LinkedIn, and GitHub; it holds no marketplace data. Google reCAPTCHA v3 [19] is verified by `recaptcha.service` and fails open (Section 16.5).

9.8 The Escrow Services

Money movement is concentrated in a few services, each of which is the *only* place its kind of operation happens.

Table 9.2 — Services that move money

Service	Responsibility
`milestone.service`	Fund a milestone from the client's wallet; release a milestone to the freelancer's wallet; approve a milestone and advance or complete the contract. Both manual approval and automatic release call the same approval function
`hourlyEscrow.service`	Lock a week's estimate; draw an amount down for a time log; re-lock on top-up; refund, release, or split what remains
`gigOrder.service`	Create a gig-order job; the single activation function that every deposit-confirmation path calls
`jobCompletion.service`	Roll up completion statistics exactly once per job
`stripe.service`	Checkout, Connect, transfers, payouts, and the simulated mode

9.9 Scheduled Jobs

A small scheduler (`node-cron`) starts with the server and runs five jobs in the server's local time.

Table 9.3 — Scheduled jobs

Schedule	Job	Purpose
Every hour, on the hour	Milestone auto-release	Release submitted milestones whose review deadline has passed
Every hour, at :30	Stale order and offer clean-up	Cancel gig orders left unpaid for seven days; expire custom offers past their expiry
Sunday, 23:59	Hourly retry	Retry pending time logs on active hourly contracts
Every day, 02:15	Skill verification expiry	Revert skill verifications older than six months to unverified, and notify the freelancer

Schedule	Job	Purpose
Every hour, at :45	Scheduled CMS publication	Publish CMS pages whose scheduled time has passed

An open job's deadline expiry is deliberately *not* a scheduled job: it is applied lazily, the next time the job is read.

9.10 Deployment Architecture

The three tiers are deployed independently.

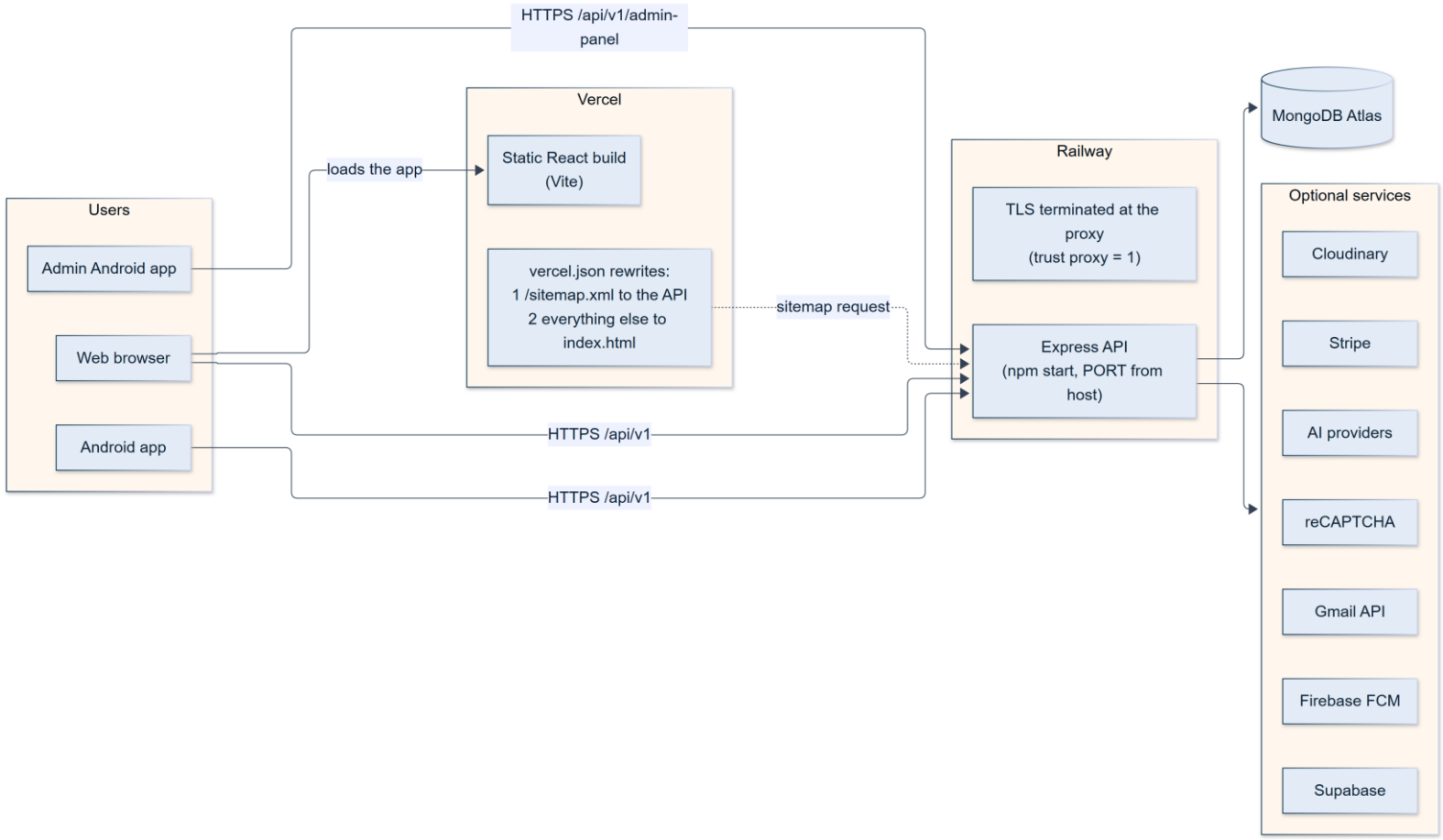


Figure 9.3 — Deployment architecture

The frontend and the API are deployed independently and live on different domains; every third-party service other than MongoDB is optional (Section 9.10).

- The **frontend** is a static Vite build hosted on Vercel [32]. Its `vercel.json` contains two rewrites whose order is load-bearing: `/sitemap.xml` is proxied to the backend, and every other path is rewritten to `index.html` for client-side routing. Reversing them would let the catch-all swallow the sitemap request.
- The **backend** runs on Railway [31] with `npm start`, bound to the platform-supplied port. Railway terminates TLS and forwards plain HTTP, which is why `trust proxy` is enabled, and Railway blocks outbound SMTP ports, which is why the Gmail API provider is mandatory there.
- The **database** is MongoDB Atlas [33].
- **Mobile builds** are produced locally and distributed as signed APK files (Chapter 15).

The minimum configuration to run the backend is a MongoDB connection string and two JWT secrets; the frontend needs the API address. Everything else — Stripe, Supabase, the AI providers, email, reCAPTCHA, Cloudinary, and Firebase — degrades gracefully when unset.

9.11 Architectural Trade-offs

Several decisions trade capability for simplicity, and are stated openly.

- **REST and polling in place of GraphQL and WebSockets.** Chat and the project pages poll every few seconds. This works through any proxy and needs no connection state, at the cost of latency and of extra requests.
- **Context API in place of a state library**, which keeps the dependency count low and suits per-page data.
- **No multi-document transactions.** Operation ordering and atomic conditional updates provide the needed guarantees, at the price of careful discipline in each money flow (Section 16.7).
- **Sequential AI fallback with no circuit breaker**, which is simple and predictable, but slow when several providers are down (Section 13.3).

10. Database Design

10.1 Overview

SkillKart's data layer consists of **40 MongoDB collections** managed through Mongoose schemas. MongoDB is the sole system of record for marketplace data: users, jobs, gigs, proposals, contracts, payments, messages, and everything else described below. (Supabase, where it is configured, handles optional social sign-in only and holds no marketplace data.)

The design is reference-based. Related documents point to each other with `ObjectId` references, which keeps each collection small and lets the same record — a user, a job — be shared by many others without duplication. Embedded sub-documents are used only where the child has no life of its own: a skill inside a freelancer profile, a dispute inside a contract or milestone, a FAQ inside a gig. Enumerated fields (`enum`) constrain every status and type to a fixed vocabulary, so a state cannot be set to a value the code does not understand.

Three further characteristics shape the rest of the system.

- **No multi-document transactions.** The codebase does not use MongoDB transactions. Correctness instead comes from strict operation ordering — validate first, write after — and from atomic single-document updates (`findOneAndUpdate` with a guard condition) wherever money or a state transition is involved. Section 12.12 describes how this is applied to escrow.
- **Referential integrity at the application layer.** MongoDB does not enforce foreign keys, so the service layer is responsible for keeping references valid. The demo-data seeder builds every reference from a record that already exists, so the seeded dataset is consistent by construction (Section 18.5).
- **Self-cleaning collections.** Time-limited records expire by themselves through TTL indexes: one-time passwords (`OTP`), refresh tokens (`RefreshToken`), and admin sessions (`AdminSession`) are removed by MongoDB once their `expiresAt` passes.

10.2 Collections by Domain

The 40 collections are organised into core marketplace entities and platform governance schemas. Tables 10.1 and 10.2 summarize the primary schemas and their essential operational fields.

Table 10.1 — Marketplace, profile and communication schemas

Collection	Purpose	Key Fields
`User`	Core authentication and account identity	`email`, `password`, `role`, `authProvider`, `isEmailVerified`, `isActive`, `isBlocked`, `tokenVersion`, `loginAttempts`, `lockUntil`, `pushTokens`
`RefreshToken`	Refresh token state and theft detection	`jti`, `familyId`, `userId`, `revokedAt`, `replacedByJti`, TTL `expiresAt`
`OTP`	Passcode hashing and verification	`otpHash`, `purpose`, `attempts`, `maxAttempts`, `isBlocked`, TTL `expiresAt`

Collection	Purpose	Key Fields
FreelancerProfile	Freelancer identity, skills and earnings	title, bio, hourlyRate, skills[] (name, level, isVerified, verificationScore, expiresAt), portfolio[], rating, walletBalance
ClientProfile	Client organisation and spend tracking	companyName, companyType, industry, budgetRange, totalProjectsPosted, totalSpent, walletBalance
Job	Project requirements and gig order instances	title, description, budget (min, max, type), skills, category, status, source, gigId, gigTier, requirements
Gig	Packaged freelancer service listings	title, category, skills, pricing (basic, standard, premium), media, faqs[], status, stats
Proposal	Freelancer bid submissions on open jobs	coverLetter, fixedPrice, hourlyRate, estimatedDeliveryTime, estimatedWeeklyHours, status, canResubmit
CustomOffer	Direct chat proposals converted to jobs	title, description, price, deliveryDays, revisions, status, expiresAt, jobId
Conversation	Chat sessions and unread tracking	participants[], lastMessage, unreadCount map
Message	Chat messages and attachments	content, attachments, type (text/custom_offer), customOfferId, readBy
Review	Two-way post-completion ratings	rating, comment, type, isHidden, unique (jobId, reviewerId)

Table 10.2 — Financial escrow, verification and governance schemas

Collection	Purpose	Key Fields
Contract	Central engagement lifecycle engine	type, fixedPrice/hourlyRate, milestoneMode, status, timer, hourlyEscrow (lockedBalance, weeklyLockAmount), endRequest
Milestone	Funded phase of fixed-price contracts	order, title, amount, status, reviewDeadline, embedded dispute
Payment	Escrow ledger and transaction audit	type, milestoneId, totalAmount, feeRate, platformFee, freelancerPayout, currency, status, Stripe IDs
TimeLog	Hourly contract work sessions	duration, startTime, endTime, source (timer/manual), status (pending, approved, billed)
Withdrawal	Freelancer payout disbursement requests	amount, method (bank_transfer, paypal, stripe), status
SkillNode	Knowledge graph vertices	name, slug, type (industry, category, skill, technology), aliases, weight, status
SkillRelationship	Directed typed edges in taxonomy graph	from, to, type (15 edge types), weight, unique (from, to, type)
SkillTestAttempt	Proctored skill examination records	skillsTested, level, questions[], totalScore, passed, violations[], proctoring summary
AdminAuditLog	Immutable record of sensitive actions	adminEmail, action, category, resource, resourceId, previousValue, newValue, severity

Collection	Purpose	Key Fields
`Report`	Abuse, spam and contract dispute flags	`category`, `priority`, `status`, `reportedItem`, `resolvedBy`
`SystemSettings`	Key–value platform runtime parameters	`key`, `value`, `category` (`fees`, `security`, `general`), `valueType`
`ApiLog` / `ErrorLog`	Operational telemetry and crash dumps	`method`, `path`, `statusCode`, `duration`, `userId`, `ip`, `errorMessage`

10.3 Key Design Decisions

One entity for projects and gig orders. Ordering a gig does not create a separate "order" collection. It creates a `Job` with `source: 'gig'`, linked back to the `Gig` and the chosen pricing tier (or to the `CustomOffer` it came from). The order then reuses the same escrow, delivery, completion, and statistics lifecycle as any other job instead of a parallel one. Only `source: 'project'` jobs — a client's own posted requirement, matched through proposals — go on to spawn a `Contract`.

Milestones are a top-level collection. A `Milestone` is not embedded in its `Contract`. Each funded phase has its own state machine, its own `Payment` row, its own review deadline, and its own dispute, and it must be addressable by ID from the payment and delivery endpoints. A fixed-price hire with no explicit split still receives one implicit "Full Project" milestone, so there is no separate no-milestone code path.

Disputes live where the money is. For a fixed-price contract the dispute is embedded in the specific `Milestone` in question; for an hourly contract, which has no milestones, it is embedded in the `Contract`. A fixed-price contract therefore never has a contract-level dispute, and any code that reads a dispute must branch on `contract.type`.

Wallet balances live on the profiles. The freelancer's and the client's wallet balances are fields on `FreelancerProfile` and `ClientProfile`, adjusted only through atomic, guarded updates. Frozen wallets are represented by `isFrozenWallet` and its audit fields.

A single `Payment` status vocabulary. Every payment type shares one status enum. Locking funds is recorded as `deposit_paid` and releasing them moves the same row to `released`; there is no separate "locked" status. A fixed-price contract has several `Payment` rows (one per milestone), so a payment is always resolved through its milestone rather than through the contract alone.

Verification data lives with the skill. A freelancer's verified status is not a single flag. Each entry in `FreelancerProfile.skills[]` carries its own `level`, `isVerified`, `verificationScore`, `lastTestedAt`, and `expiresAt` (verification expires after six months), which is what allows applying and publishing to be gated per skill and per experience level.

Uniqueness is enforced by indexes. Duplicate reviews are prevented by a unique index on `(jobId, reviewerId)`; duplicate skill-graph edges by a unique index on `(from, to, type)`; one testimonial per user by a unique index on `userId`; and one profile per user on both profile collections.

10.4 Key Relationships

Table 10.9 — Principal relationships between collections

Relationship	Cardinality	Notes
`User` — `FreelancerProfile` / `ClientProfile`	1 : 1	A user has the profile matching their role
Client (`User`) — `Job`	1 : N	
`Job` — `Proposal`	1 : N	A freelancer may have one proposal per job
Accepted `Proposal` — `Contract`	1 : 1	Acceptance creates the contract
`Contract` — `Milestone`	1 : N	Fixed-price only
`Milestone` — `Payment`	1 : 1	One escrow payment per milestone
`Contract` — `Payment`	1 : N	Hourly lock and invoice payments
`Contract` — `Delivery`, `TimeLog`, `ContractHistory`	1 : N	
`Conversation` — `Message`	1 : N	A message may carry a `CustomOffer`
`CustomOffer` — `Job`	1 : 1	Created when the client accepts
`Gig` — `Job`	1 : N	Each order is a job linked to the gig
`Job` — `Review`	1 : N	At most one review per reviewer
`SkillNode` — `SkillRelationship` — `SkillNode`	N : N	The taxonomy graph

The **Contract** is the central hub of the engagement lifecycle: an accepted proposal creates a contract, which then drives milestones, payments, deliveries, messaging, reviews, and — when necessary — disputes. Figures 10.1a to 10.1e show the core entities and their relationships.

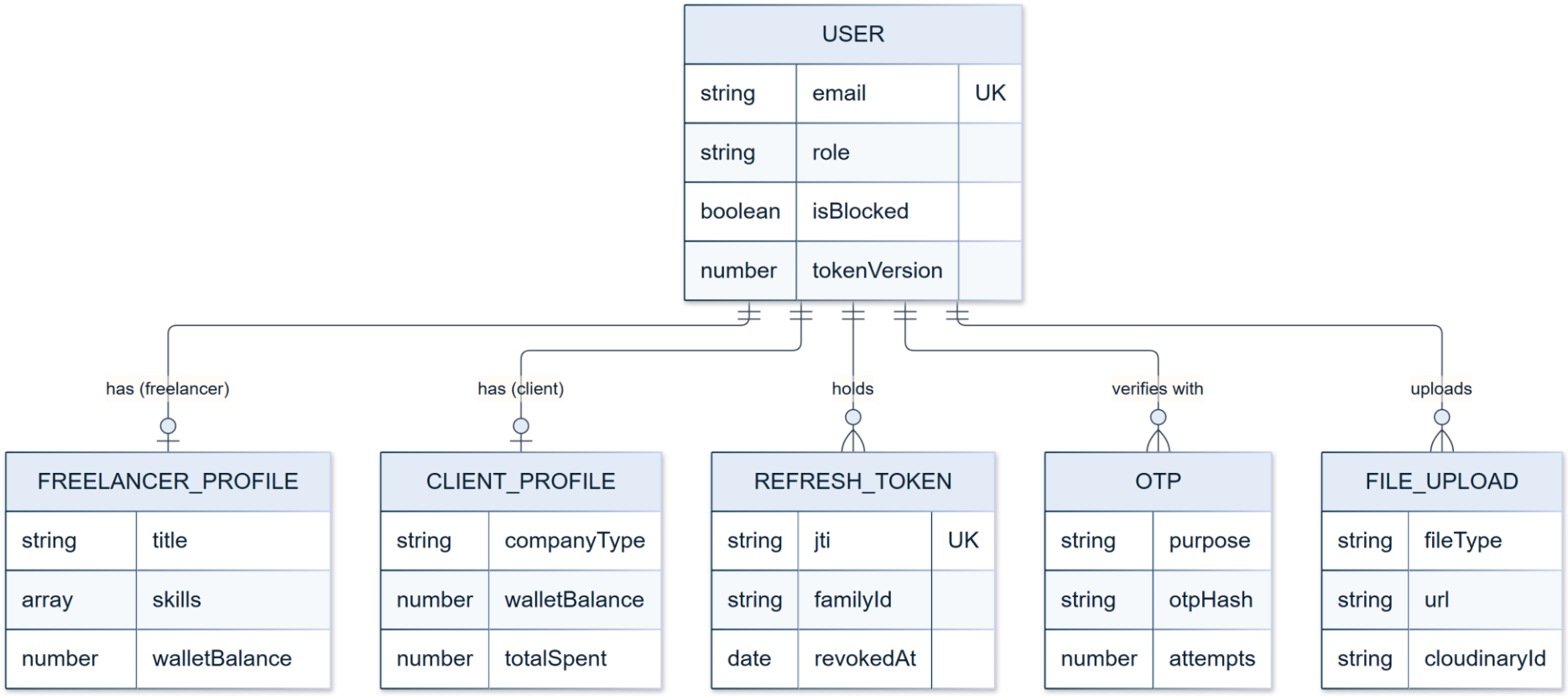


Figure 10.1a — Entity relationships: identity and profiles

The entity relationships are shown as five diagrams, (a) to (e), so that every entity name stays legible when printed.

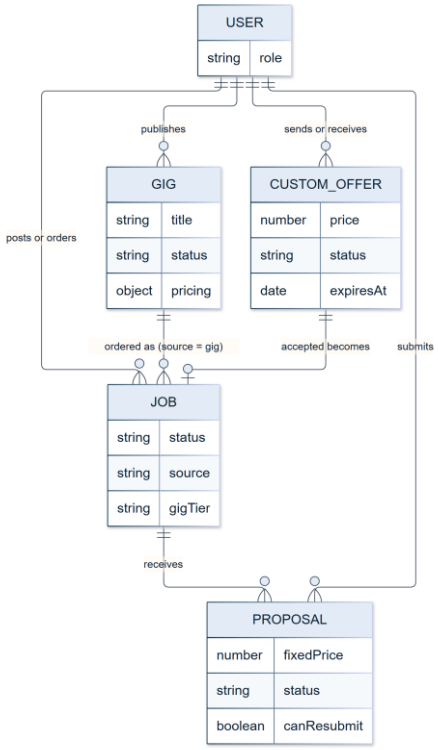


Figure 10.1b — Entity relationships: marketplace

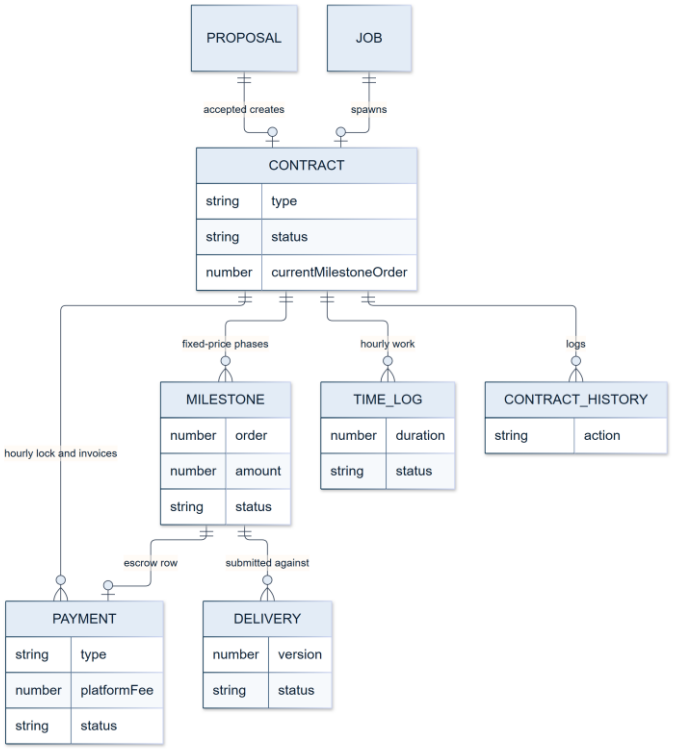


Figure 10.1c — Entity relationships: contracts, escrow and payments

A gig order is a Job with no Contract; its payments link to the Job directly (Section 12.9).

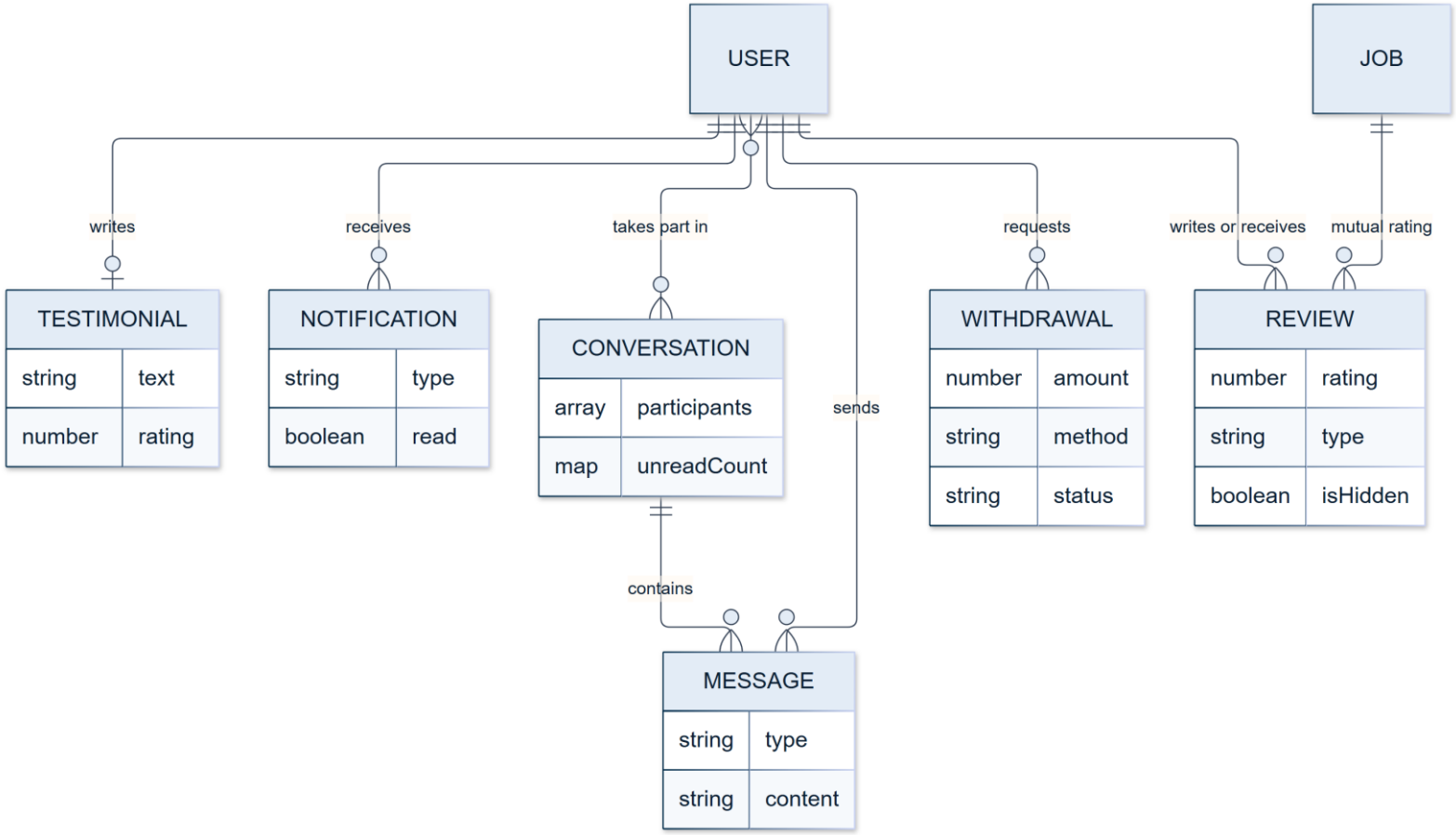


Figure 10.1d — Entity relationships: communication, reviews and withdrawals

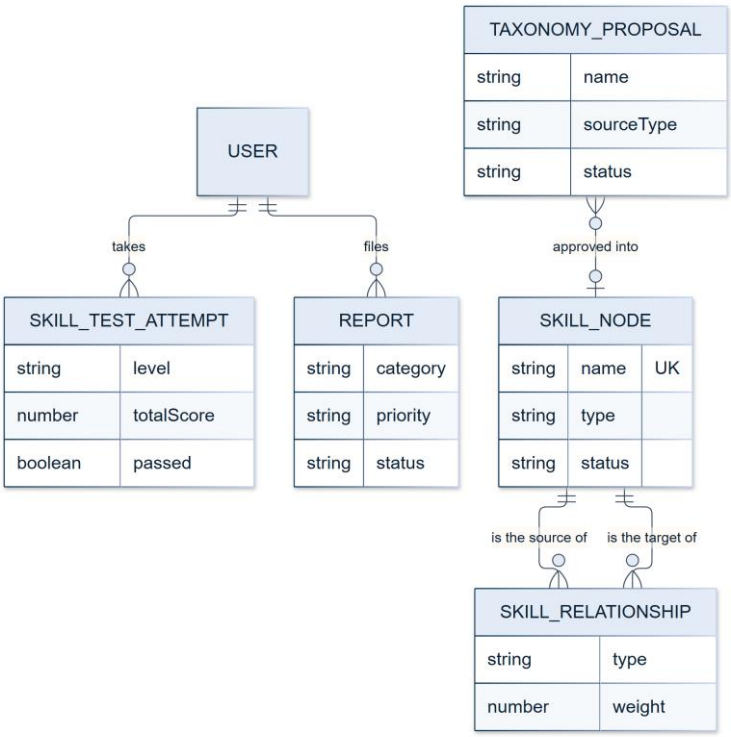


Figure 10.1e — Entity relationships: skills, verification and reports

10.5 Status Vocabularies and State Machines

Every entity with a life cycle has a fixed `enum` of statuses, and the code moves it between them only along defined transitions. Table 10.10 lists each vocabulary and the transitions that occur. A few values are *reserved*: they are part of the schema so that a future feature can use them, but no current code path sets them.

Table 10.10 — Status vocabularies and the transitions that occur

Entity	Statuses	Transitions in practice
`Job`	`draft`, `open`, `pending_payment`, `in-progress`, `completed`, `cancelled`, `expired`, `closed`	`draft` → `open` (publish); `open` → `in-progress` (proposal accepted); `in-progress` → `completed` or `cancelled`; `open` → `expired` (deadline passed, applied when read) or `closed` (client or administrator). A gig order is created as `pending_payment` and becomes `in-progress` when the first deposit lands
`Proposal`	`pending`, `accepted`, `rejected`, `withdrawn`	`pending` → `accepted` or `rejected`; a rejection with `canResubmit` allows `rejected` → `pending` again
`Contract`	`active`, `delivered`, `in_revision`, `completed`, `under_review`, `cancelled`, `paused`, `pending_cancellation`	`active` → `delivered` → `in_revision` ⇄ `delivered` → `active` (next milestone) or `completed`; a dispute moves the contract to `under_review`; `paused` is set only by an administrator; `pending_cancellation` is reserved

Entity	Statuses	Transitions in practice
`Milestone`	`pending_funding`, `funded`, `submitted`, `in_revision`, `approved`, `disputed`, `released`, `refunded`, `cancelled`	`pending_funding` → `funded` → `submitted` → `released`, with `in_revision` ⇄ `submitted` for revisions and `disputed` as a side branch; `refunded` and `cancelled` result from administrator action. `approved` is reserved: release follows submission directly
`Payment`	`deposit_pending`, `deposit_paid`, `released`, `refunded`	`deposit_pending` → `deposit_paid` → `released` or `refunded`. An `hourly_escrow` lock row stays `deposit_paid`; each billed log is its own `hourly_invoice` row, created already `released`
`TimeLog`	`pending`, `approved`, `rejected`, `billed`	`pending` → `billed`, automatically. `approved` and `rejected` are reserved: no client approval step exists
`Withdrawal`	`pending`, `processing`, `completed`, `failed`, `cancelled`	A Stripe withdrawal is created `processing` or `completed`; a manual one is created `pending` and an administrator moves it to `completed` or `cancelled` (with an automatic refund). `failed` is reserved
`CustomOffer`	`pending`, `accepted`, `declined`, `withdrawn`, `expired`	`pending` → any of the other four; expiry is applied by the hourly clean-up job

The contract and milestone machines, which are the most consequential, are drawn in Figures 12.6a and 12.6b (Section 12.11).

11. Technologies Used

This chapter explains each major technology and the reason it was chosen. Versions are those installed in the project at the time of writing.

11.1 Frontend

- **React 19.2** [4] — a component-based UI library chosen for its ecosystem, performance, and suitability for a complex, stateful single-page application.
- **Vite 7** [5] — the build tool and development server, chosen for fast hot-module replacement and optimised, code-split production builds.
- **React Router 7** — declarative client-side routing with nested routes and route guards.
- **React Hook Form 7** — performant form management with validation and minimal re-rendering.
- **Axios 1.13** — the HTTP client, wrapped in a module with interceptors for authentication, token refresh, request de-duplication, and error handling.
- **Tailwind CSS 4** [35] with hand-authored component CSS (76 stylesheets) — utility styling combined with scoped component styles.
- **Framer Motion 12 and GSAP 3** — animation libraries for transitions and interactive effects.
- **lucide-react** — a consistent icon set; **react-select** and **react-easy-crop** — searchable selects and in-browser image cropping.
- **MediaPipe Tasks Vision 1.0.1** [16] — on-device face detection for skill-test proctoring.
- **Supabase JS 2** [20] — the client for optional social sign-in.

11.2 Backend

- **Node.js 22.3 or later** [2] — the JavaScript runtime, enabling one language across the stack.
- **Express 4** [3] — a minimal, flexible web framework for the layered REST API.
- **Mongoose 8** [6] — an object–document mapper adding schemas, validation, hooks, and relationship modelling to MongoDB.
- **Multer** — multipart upload handling; **node-cron 4** — the scheduler for the five background jobs.
- **Swagger (swagger-jsdoc and swagger-ui-express)** [26] — interactive API documentation at `/api/v1/docs`.
- **ExcelJS and PDFKit** — generation of the administrators' Excel and PDF exports; **pdf-parse and mammoth** — text extraction from PDF and Word résumés; **ua-parser-js** — browser and device details for login-alert emails.

11.3 Database

- **MongoDB** [1] on **MongoDB Atlas** [33] — a document database chosen for its flexible schema, scalability, and natural fit for the varied, evolving data of a marketplace: nested gig tiers, embedded disputes, skill arrays with per-skill verification.

11.4 Authentication and Security

- **JSON Web Tokens** [9] (`jsonwebtoken`) — access and refresh tokens, with refresh rotation backed by a collection.
- **bcrypt** [24] (`bcryptjs`, cost factor 12) — one-way password hashing.
- **One-time passwords** — six-digit, hashed, emailed codes for verification and two-factor login.
- **Supabase Auth** [20] — optional Google, LinkedIn, and GitHub sign-in. Supabase authenticates *end users* only; it is deliberately decoupled from the platform's outbound email.
- **Google reCAPTCHA v3** [19] — invisible, score-based bot mitigation on login, registration, and the contact and newsletter forms.
- **Helmet** [37], **express-rate-limit**, **express-validator**, **express-mongo-sanitize**, **sanitize-html** — header hardening, abuse prevention, input validation, NoSQL-injection sanitisation, and HTML stripping.
- **crypto-js** — AES encryption of the SuperAdmin credential file.

11.5 Email Delivery

Transactional email is sent through a **pluggable provider architecture**, selected at runtime by a single `EMAIL_PROVIDER` variable (Section 9.7):

- **Gmail API** (`googleapis`) [25] — the provider used in deployment. It sends over HTTPS using OAuth credentials for a dedicated platform mailbox, with a long-lived refresh token; the application performs no OAuth authorisation flow at runtime.
- **SMTP** (`nodemailer`) — the default for local development, and the automatic start-up fallback when the Gmail credentials are absent or fail verification.

Two constraints drove this design. First, **cloud hosts commonly block outbound SMTP ports** (465 and 587), and the deployment target is one of them, so an HTTPS-based transport is a functional requirement in production, not a preference. Second, **the Gmail send permission cannot be satisfied by the existing Supabase Google sign-in**: Supabase authenticates end users, whereas a transactional message is sent *as the platform*, often to someone who has never used Google sign-in and at moments when no user session exists, such as a password-reset code. The two concerns are therefore kept entirely separate.

11.6 Payments

- **Stripe 22** [7] — Checkout for wallet top-ups and gig deposits, Connect Express accounts and payouts for freelancer withdrawals, and webhooks with signature verification. A **simulated**

mode (`STRIPE_DUMMY`) allows the entire payment flow to be demonstrated without real transactions.

11.7 Artificial Intelligence

- **An in-house provider router** over **Google Gemini** [8] (`@google/genai`), **DeepSeek** [14], **Alibaba Qwen** [29], **Groq** [28] (`groq-sdk`), **Anthropic Claude** [27] (`@anthropic-ai/sdk`), and **Ollama** [30] (local). Each provider is wrapped in a common interface (Chapter 13).

11.8 File Storage and Media

- **Cloudinary** [18] — hosted storage and delivery for avatars, logos, covers, portfolio images, gig banners, job attachments, and chat attachments. Résumés are parsed in memory and are not stored.

11.9 Push Notifications

- **Firebase Cloud Messaging** [17] through `firebase-admin` 14 on the server and `@capacitor/push-notifications` on the device, hooked into the single `notify()` function and designed to fail open.

11.10 Mobile

- **Capacitor 8** [15] with nine plugins (app, browser, camera, filesystem, network, push notifications, share, splash screen, status bar), building two Android applications from the web codebase (Chapter 15).

11.11 Discoverability

- A **live-generated XML sitemap**, a `robots.txt` that excludes authenticated and transactional routes, a **web-app manifest** (brand colours, icons, and shortcuts), and **Open Graph and Twitter Card** metadata for link previews (Section 12.21).

11.12 Tooling and Operations

- **ESLint 9 and Prettier 3** — linting and formatting; **Nodemon** — automatic restart during development; **Concurrently** — runs the backend and frontend together with one command.
- **Playwright** [34] — browser automation, used throughout development for end-to-end verification in a real browser (Chapter 18).
- **Git and GitHub** — version control, with more than 300 commits across the project.
- **Hosting** — Vercel [32] (frontend), Railway [31] (backend), MongoDB Atlas [33] (database).
- **Winston and Chart.js** appear in the dependency manifests, but the application does not currently use them: logging is done through the console, request logging, and database log collections, and the administrator analytics are drawn with plain CSS bars. They are noted here for completeness and are candidates for removal or adoption (Chapter 21).

11.13 Why This Stack

Choosing a stack is a set of engineering trade-offs under real constraints. Equally capable alternatives exist — Django with PostgreSQL, Laravel with MySQL, Spring Boot, ASP.NET Core — and none is inferior on capability grounds. The reasoning below explains why MERN suits *this* system.

One language removes a class of integration cost. Every layer — Express handlers, Mongoose schemas, validators, the React frontend, even the AI prompts that request JSON — describes JSON-shaped data in JavaScript. There is no mapping between a data-transfer object and an entity and no second type system to keep in step at the API boundary, which narrows the surface for "the backend and frontend disagree about a field" defects and lets any team member work on any part of the code.

The workload is I/O-bound, which suits Node. Most of this application's time is spent waiting: for MongoDB, and for HTTP responses from AI providers, Stripe, Cloudinary, and the email transport. Node's non-blocking event loop handles many concurrent slow operations cheaply. The AI router's sequential fallback, for example, issues several awaited calls in turn without blocking any other request in the meantime.

A document database fits the data. A gig has up to three pricing tiers with their own features; a contract embeds a dispute; a freelancer profile carries arrays of skills each with its own verification record. These are natural documents, and reading one is a single query. The cost is that referential integrity must be maintained in the application layer (Section 10.1).

The ecosystem covers every integration. npm provides maintained clients for Stripe, Firebase, the AI providers, Cloudinary, and Capacitor. This is not a claim that only Node could integrate them; it is that no capability gap arose that would have forced another stack.

A real trade-off is designed around, not hidden. A single-threaded event loop can be blocked by CPU-heavy work. In this system the main candidate is password hashing, which uses the asynchronous implementation of bcrypt so that it does not stall other requests.

12. Features

This chapter details the functional architecture across the user journey: identity and authentication, profile onboarding, skill verification and gating, marketplace interactions (jobs and gigs), proposal negotiation, milestone and hourly escrow lifecycles, payments, communication, and administration reporting.

12.1 Authentication and Account Security

Registration and Access Control. Visitors register as either a **Client** or a **Freelancer**. The requested role is strictly validated against a server whitelist to prevent role-escalation exploits. Passwords require a minimum of eight characters, containing uppercase, lowercase, and numeric characters, hashed via bcrypt at cost factor 12 before persistence [24].

One-Time Passwords (OTP). Registration, credential changes, and account lifecycle events issue a 6-digit cryptographic OTP. OTPs are stored exclusively as SHA-256 hashes, enforce a 10-minute time-to-live managed by MongoDB TTL expiry indexes, permit up to 5 verification attempts, and enforce a 1-minute resend cooldown. The unified OTP service manages 11 security contexts, including password resets, email updates, recovery email binding, and account deactivation/deletion.

Two-Factor Authentication and Account Locking. Accounts are locked for 2 hours after 5 consecutive incorrect password submissions. Two-factor authentication (2FA) via email OTP is enabled by default across all client and freelancer accounts, requiring OTP validation before JWT issuance. Users may toggle 2FA and security alerts within settings.

Social Authentication. OAuth 2.0 integration via Supabase Auth supports Google, LinkedIn, and GitHub federated identity [20, 38]. Identity claims are verified server-side. New social users receive a neutral `publicviewer` role until completing role selection. On Android, OAuth executes inside Chrome Custom Tabs and redirects via deep linking (`skillkart://auth-callback`) to comply with Google OAuth policies.

JWT Token Architecture. The platform implements dual-token authentication [9]. The 7-day access token resides in memory and session storage, proactively refreshed every 55 minutes. Refresh tokens (30-day lifetime) rotate on every use. Presenting a previously rotated token outside a 10-second grace window triggers reuse detection, revoking the token family and incrementing the account's `tokenVersion` to invalidate sessions globally. Sessions terminate after 30 minutes of inactivity.

Per-Request Enforcement. Middleware inspects user state and `tokenVersion` on every request. Administrative account suspensions permit read access while rejecting state-mutating requests with HTTP 403. Account deactivation and deletion are prohibited while contracts or open jobs remain active.

Table 12.1 — Authentication parameters

Parameter	Value
Password hashing	bcrypt, cost factor 12
Password policy	at least 8 characters; upper case, lower case and digit
OTP	6 digits, 10-minute expiry, SHA-256 hashed, 5 attempts, 1-minute resend cooldown
Account lock	5 wrong passwords, locked for 2 hours
Access token	JWT, 7 days; refreshed proactively every 55 minutes
Refresh token	JWT, 30 days; rotating with reuse detection (10-second grace)
Inactivity logout	30 minutes (browser)
Rate limits (per IP)	global 1,000 requests per 15 minutes; login/register/reset-request 5 per 15 minutes; OTP endpoints 20 per 15 minutes; public AI tools, contact and newsletter 10 per minute
Bot protection	reCAPTCHA v3 on register, login, contact and newsletter; score threshold 0.5

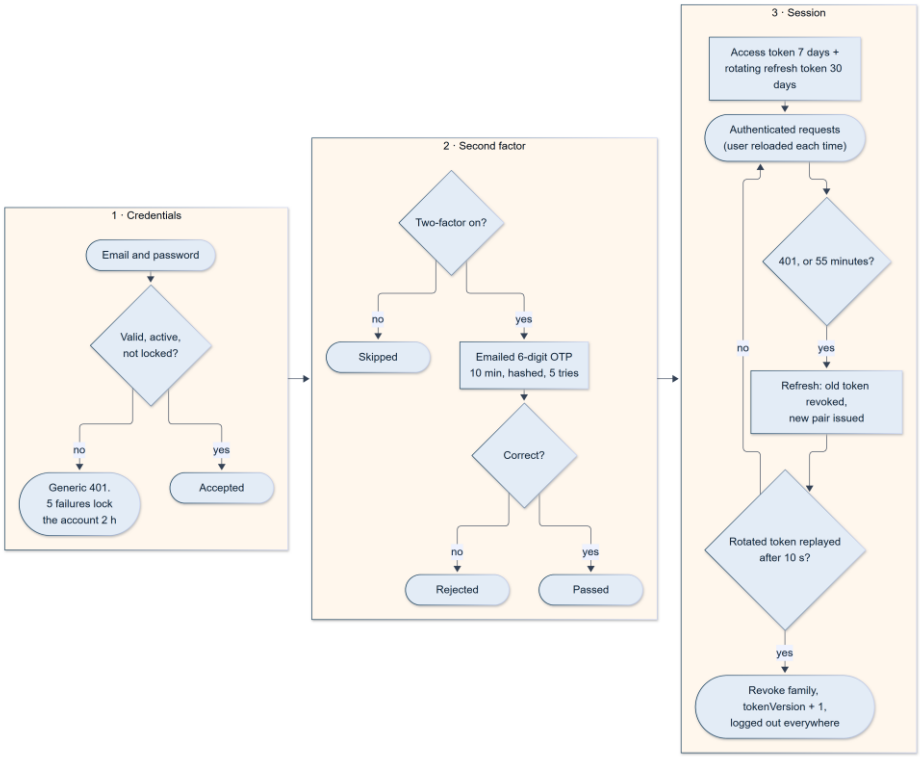


Figure 12.1 — Login, two-factor authentication and token lifecycle

Access tokens are renewed through a rotating refresh token; reuse of an already-rotated token revokes the whole token family and logs the user out everywhere (Section 12.1).

12.2 Freelancer Onboarding and Profile

Freelancers complete an 8-step onboarding sequence:

1. **Résumé Extraction (Optional):** Server parses uploaded PDF/DOC/DOCX files (up to 5 MB), using AI to extract biographies, skills, and education, assigning an automated quality classification (Strong, Average, Weak).

2. **Basic Information:** Professional title, bio, hourly rate (PKR), availability status, time zone, experience tier (Entry, Intermediate, Expert), and profile photo.
3. **Skills:** Tag selection backed by taxonomy graph resolution. AI validates and normalises tags (e.g., "reactjs" $\rightarrow$ "React.js"). Unrecognised tags queue for admin review.
4. **Languages:** Language proficiencies (Basic, Conversational, Fluent, Native).
5. **Portfolio:** Project showcases with media uploads.
6. **Education:** Academic credentials and degrees.
7. **Certifications:** Professional accreditations.
8. **Review and Submission:** State validation and draft finalisation.

The onboarding wizard auto-saves drafts in browser storage. Profiles maintain computed verification status requiring verified email, complete profile elements, and tested skills. Passing the skill test is optional during onboarding but strictly required before bidding or publishing gigs. Public profiles expose verified badges and portfolio metrics while shielding sensitive phone, wallet, and payment records.

12.3 Client Onboarding and Profile

Clients register through a 5-step wizard capturing: (1) Identity and organization (Individual, Startup, SME, Enterprise); (2) Industry background assisted by AI recommending the top 3 categories from 30 domain options; (3) Project requirements (technology stack, hiring frequency, budget); (4) Timeline and communication preferences; and (5) Security terms confirmation. Drafts persist server-side. Public client profiles display hire counts, total project spend, and freelancer reviews, keeping internal wallet balances private.

12.4 Skill Verification

Skill verification addresses unverified claims through AI-generated, proctored examinations tailored to three proficiency tiers.

Table 12.2 — Skill-test levels

Level	Questions	Multiple choice / written	Rank earned if passed
Easy	5	4 / 1	Beginner
Intermediate	10	8 / 2	Intermediate
Expert	15	12 / 3	Expert

Each examination allocates 20 seconds per multiple-choice question (MCQ) and 30 seconds for written responses within a locked fullscreen interface.

Grading and Validity. The AI evaluator assigns scores between 0 and 1, including partial credit for written explanations. Scores $\ge$ 25% grant verified status at the chosen level rank. Verifications remain valid for 6 months, after which daily cron maintenance flags them as expired. Retakes are unrestricted and never revoke existing passing scores.

Client-Side Automated Proctoring. Proctoring incorporates two complementary client-side guards:

- **Camera Guard:** Google MediaPipe Face Landmarker [16] runs locally at 700 ms intervals on the device CPU. Absence of a detected face for 3 continuous seconds terminates the examination immediately.
- **Browser Guard:** Window blur, tab switching, and fullscreen exits increment strike violations. Three strikes trigger test termination. Copy, paste, right-click, and developer shortcuts are suppressed.
- **Strictness Control:** Environment parameter `SKILL_TEST_PROCTOR_STRICTNESS` (0.0 to 1.0, default 0.5) configures violation tolerances and guard enforcement.

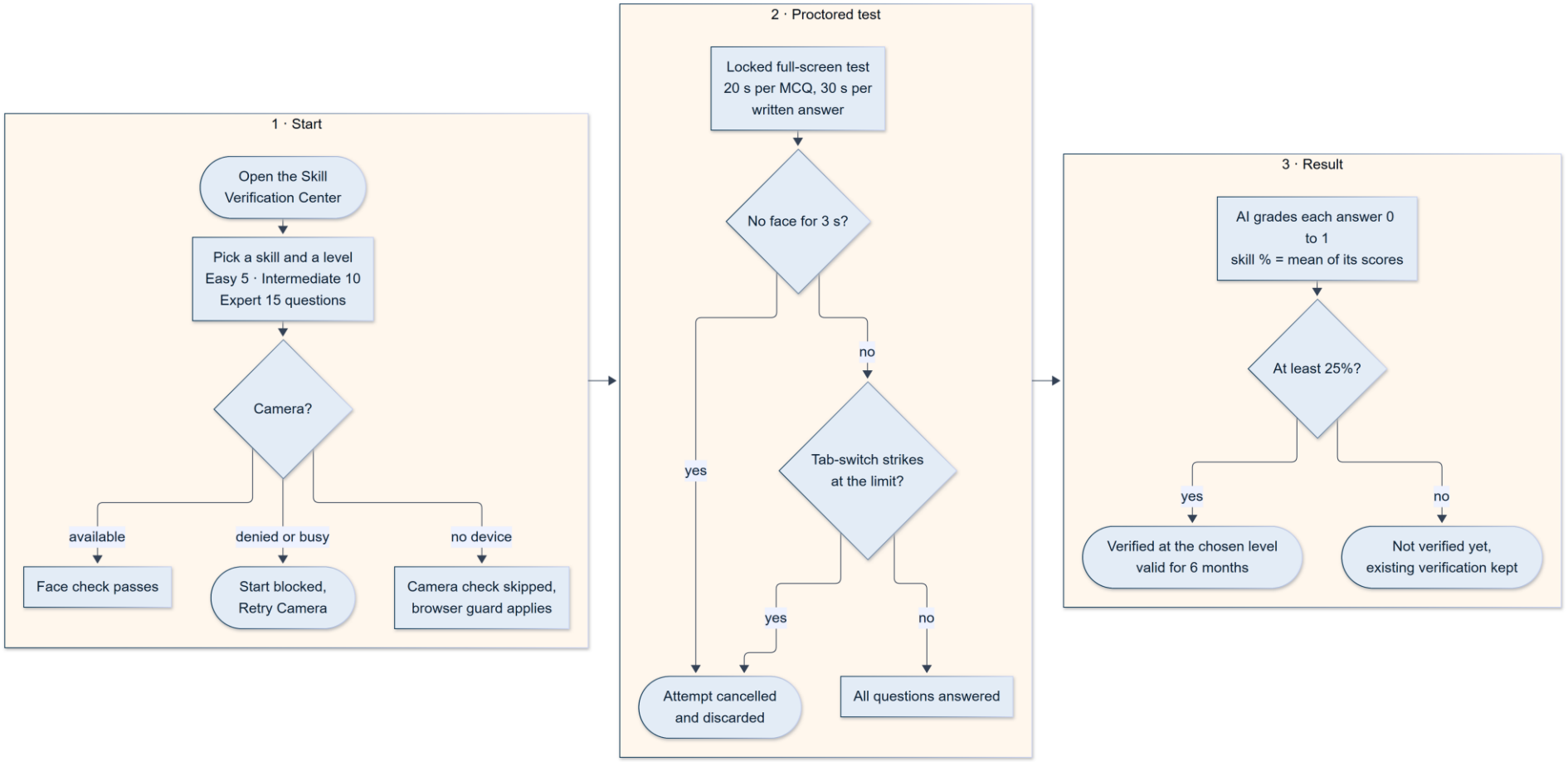


Figure 12.2 — Skill-test flow: level choice, proctored test, grading and verification

The camera and browser guards can cancel an attempt at any point; a cancelled attempt is discarded and never affects an existing verification (Section 12.4).

12.5 Skill Gating and the Apply Gate

Skill gating enforces competence across marketplace operations: freelancers must hold at least one active, non-expired verified skill covering the job category and matching or exceeding the required experience level. Gating rules are evaluated via skill graph coverage. When viewing jobs, an automated pre-check determines proposal eligibility, displaying guidance and test links for missing prerequisites while the backend performs definitive authorization checks upon submission.

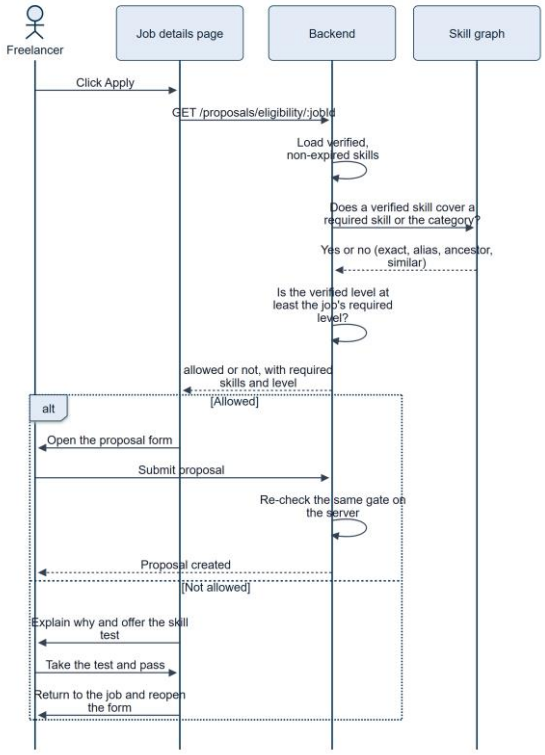


Figure 12.3 — The apply gate: eligibility is checked before the proposal form opens

The server re-checks eligibility when the proposal is submitted, so the pre-check is a convenience and not the security boundary (Section 12.5).

12.6 Skill Taxonomy and Discovery

The platform taxonomy is structured as a directed graph comprising ~180 `SkillNode` vertices and 15 typed `SkillRelationship` edge variants across 68 predefined categories in 10 groups. The graph powers:

- **Canonical Resolution:** Maps user synonyms and aliases to standardized taxonomy nodes.
- **Search Expansion:** Broadens search queries to encompass parent categories, sub-disciplines, and related technologies.
- **Deterministic Match Scoring:** Calculates compatibility weights: exact match (1.0), specialized sub-skill (0.85), broader parent skill (0.70), and related skill (0.55).
- **Supervised Graph Evolution:** Unrecognised terms prompt AI taxonomy classification stored as `TaxonomyProposal` entities, requiring administrative approval via `/admin/taxonomy`.

Freelancer discovery (`/browse-freelancers`) supports multi-criteria filtering by skills, hourly rates, verified status, and reviews, ordering verified profiles with higher test scores first.

12.7 Jobs

Clients initiate requirements through a 5-step posting wizard specifying title, taxonomy category/subcategory, skill tags, fixed or hourly budget, duration, and attachments (up to 5 images and 5 documents). AI helpers offer category classification and brief expansion.

Posting Mechanics and Lifecycle. Clients may publish jobs with zero wallet balance; funding checks occur exclusively upon proposal acceptance. Jobs transition across statuses: `draft`, `open`, `in-progress`, `completed`, `cancelled`, `expired`, and `closed`. Expired deadlines trigger automated closure, rejecting pending proposals. Active engagements cannot be cancelled unilaterally.

12.8 Gigs

Gigs represent packaged freelancer services featuring up to three pricing tiers (Basic, Standard, Premium), delivery windows, revision counts, media galleries, and FAQ entries. Gig publishing is strictly skill-gated against verified capabilities. Gigs maintain individual telemetry (views, orders, average rating) and support pausing.

12.9 Gig Orders and Custom Offers

Direct Gig Orders. Clients initiate orders by selecting a tier and submitting requirements ($\ge$ 30 characters). The system creates a `Job` entity (`source: 'gig'`) in `pending_payment` status, invisible to the freelancer until deposit confirmation. Gig orders follow a 50/50 escrow split: 50% deposit upfront and 50% upon delivery approval. Unpaid orders auto-cancel after 7 days.

In-Chat Custom Offers. Freelancers generate custom contractual offers within direct messaging. Offers define title, description, price (Rs. 5 to Rs. 10,000,000), delivery timeframe (1–365 days), and revisions, expiring after 7 days. Offers enforce contact-masking filters and limit active pending offers to five per conversation. Acceptance creates a `pending_payment` gig order.

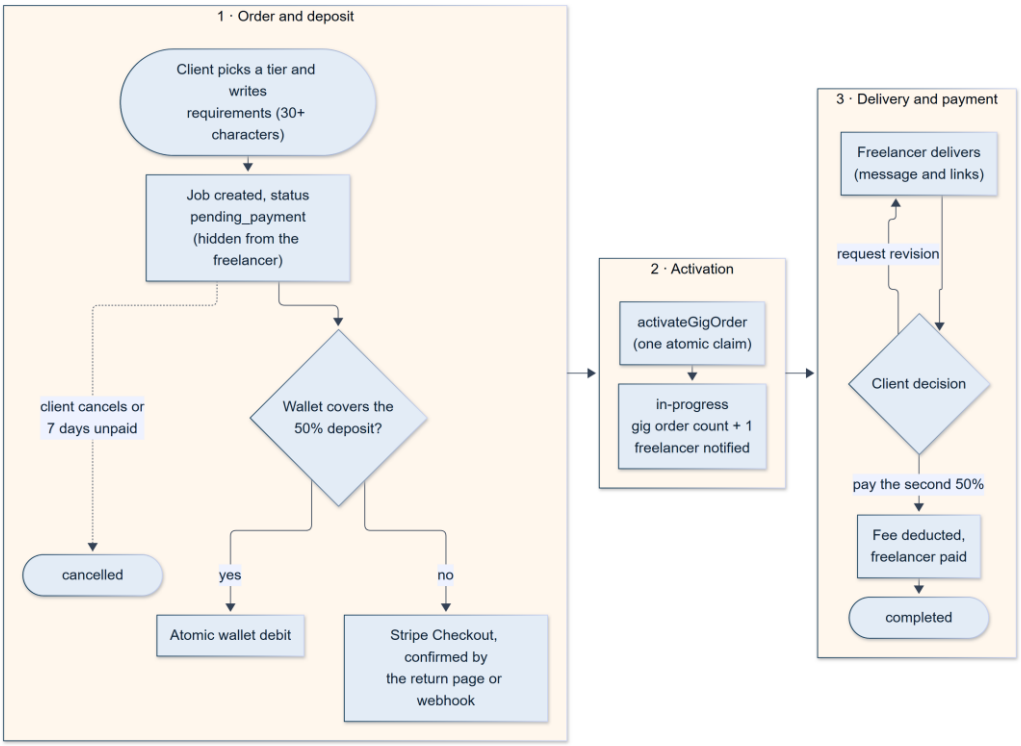


Figure 12.4 — Life cycle of a gig order, from requirements to completion

A gig order starts as pending_payment and becomes in-progress only when a deposit lands, through a single activation function (Section 12.9).

12.10 Proposals

Freelancers bid on open jobs by submitting a proposal containing a cover letter (100–2,000 characters, with optional AI drafting), bid amount, and delivery estimate. Freelancers are restricted to one proposal per job. Clients review proposals, negotiate via messaging, and can reject or allow resubmissions.

Atomic Proposal Acceptance. Acceptance commits client funds through a 4-step sequence:

1. Validates job availability and price parity.
2. For fixed-price contracts, verifies client wallet coverage for the first milestone (or entire project); for hourly work, verifies coverage for one week's estimated cost (hourly rate $\times$ weekly hours).
3. Atomically updates job status to `in-progress`, accepts the winning proposal, generates the `Contract` and `Milestone` records, auto-rejects competing bids, and debits/locks wallet funds.
4. Reverses state changes if wallet lock operations encounter concurrent race conditions.

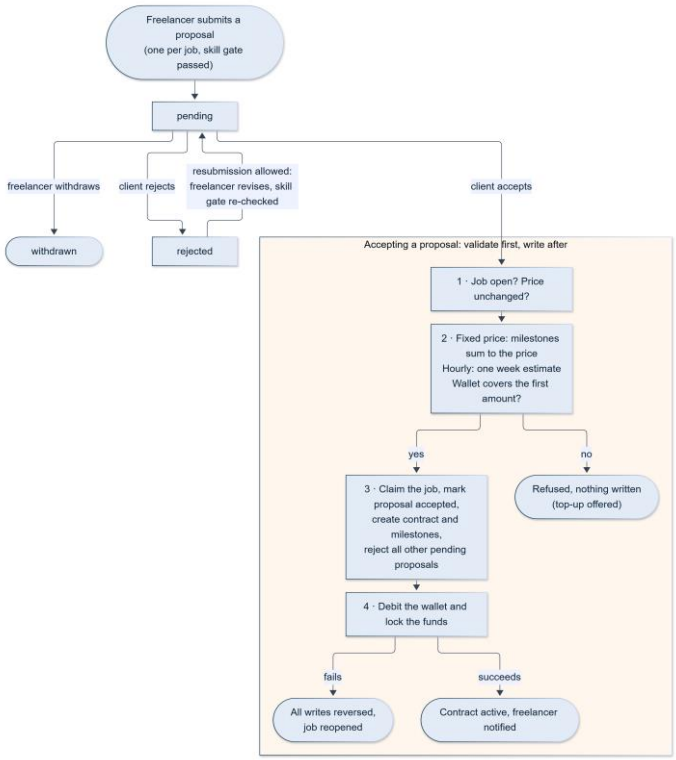


Figure 12.5 — Life cycle of a proposal and the effect of accepting it

Acceptance validates before it writes and reverses its writes if funding fails (Section 12.10).

12.11 Contracts and Lifecycle

Accepted proposals instantiate a `Contract` entity governing execution. Contract status operates as a state machine: `active`, `delivered`, `in_revision`, `under_review`, and `completed`, with administrative states `paused` and `cancelled`. Complete audit logs record state transitions in `ContractHistory`. Hourly contracts allow mutual termination requests; fixed-price contracts conclude upon milestone approvals or dispute resolution.

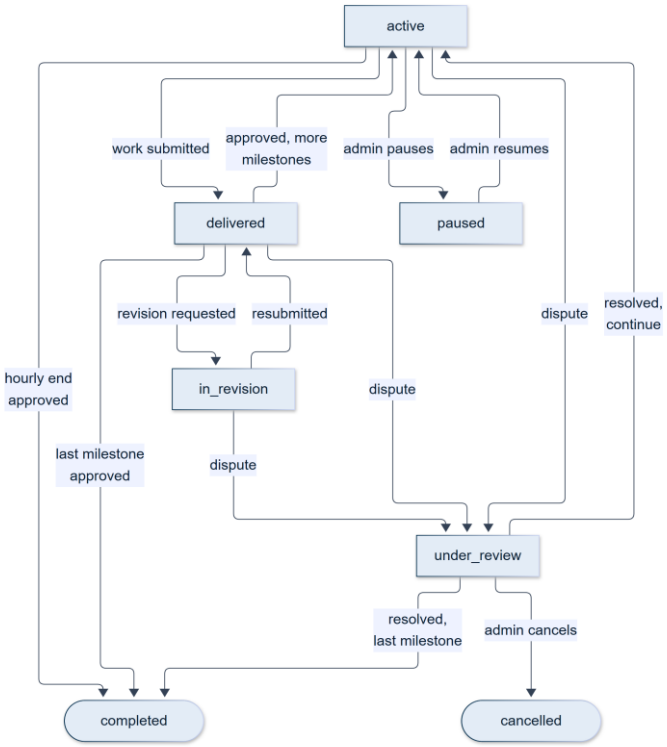


Figure 12.6a — Contract state machine

A contract and its current milestone move together; disputes attach to the milestone for fixed-price work and to the contract for hourly work (Sections 12.11 and 12.17).

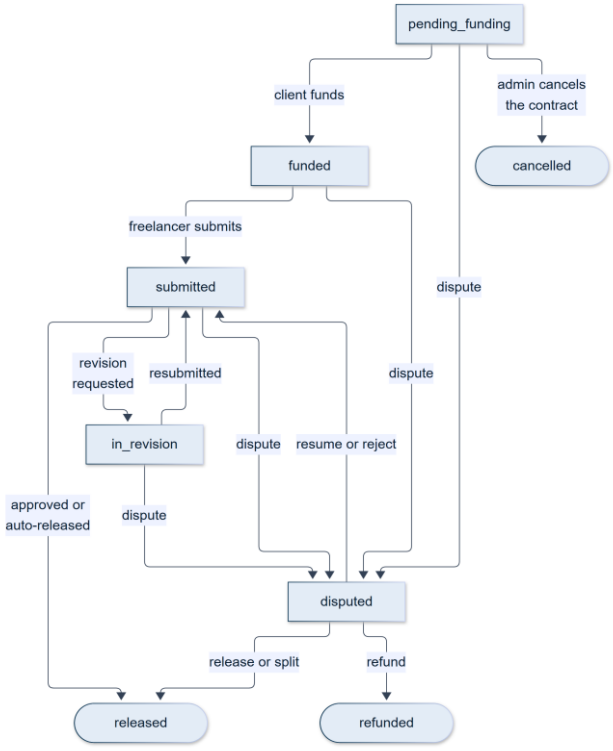


Figure 12.6b — Milestone state machine (fixed-price contracts)

12.12 Fixed-Price Milestone Escrow

Fixed-price engagements utilize wallet-native milestone escrow. Clients fund milestone phases sequentially, avoiding upfront capital lockup for subsequent stages.

Escrow Lifecycle and Safety. Freelancer deliverable submissions require active milestone funding. Submissions initiate a 7-day review window. Client approval releases funds to the freelancer wallet minus the platform fee (0.5% default). If the client remains inactive past the review deadline, a background hourly cron executes automated release. Financial operations execute within `milestone.service` via atomic conditional claims, preventing double-release races.

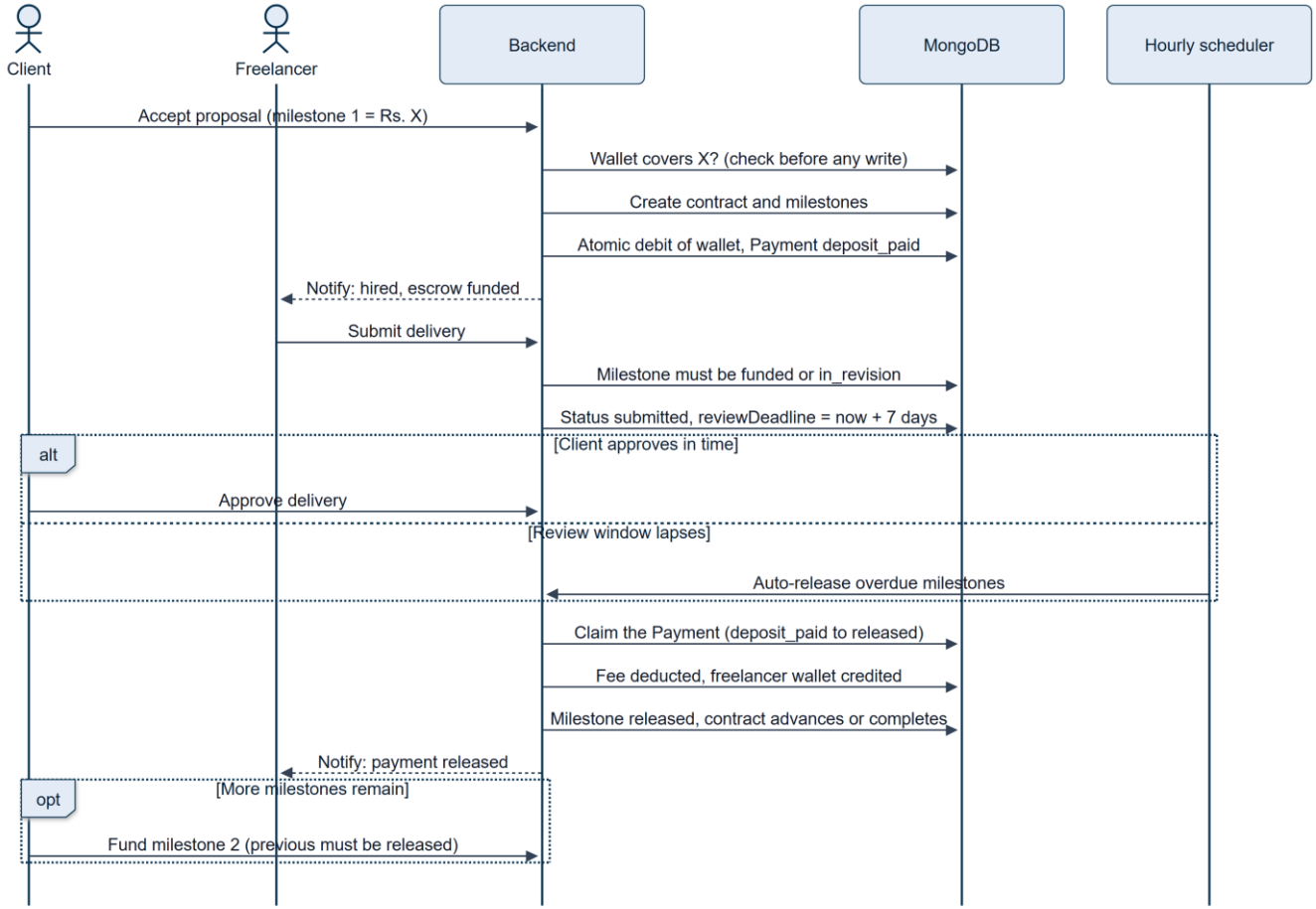


Figure 12.7 — Fixed-price milestone funding, delivery and release

Funding, submission and release are separate steps, each guarded by a state check (Section 12.12).

12.13 Hourly Wallet-Lock Escrow

Hourly contracts implement balance-lock escrow, holding one week's estimated expenditure in escrow at inception.

Time Logging and Automated Billing. Freelancers track hours via a live timer ($\ge$ 60-second threshold) or manual logs. Before logging, unbilled time is validated against the locked balance. Log generation immediately draws down funds from the locked balance to the freelancer wallet, net of platform fees. Shortfalls pause logging until the client replenishes the lock. Unused locked balances are refunded in full upon contract completion.

Table 12.3 — Fixed-price and hourly escrow compared

Aspect	Fixed-price (milestones)	Hourly (wallet lock)
Locked at hire	The first milestone's amount	One week's estimate (rate × weekly hours)
Further funding	Client funds each next milestone	Client tops up the lock
Release trigger	Client approval, or auto-release after the review window	Each logged hour, automatically
Unused funds	None: each milestone is released or refunded	Remaining lock refunded when the contract ends
Dispute scope	The specific milestone	The whole contract
Payment records	One `job_escrow` row per milestone	One `hourly_escrow` lock row and one `hourly_invoice` row per billed log

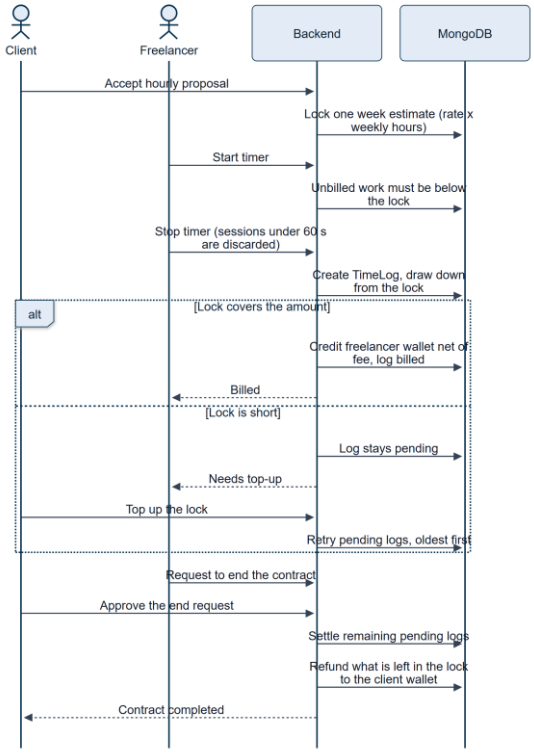


Figure 12.8 — Hourly escrow: lock, draw-down, top-up and refund

Logged time is billed immediately against the lock; whatever remains is refunded when the contract ends (Section 12.13).

12.14 Deliveries and Revisions

Freelancers submit work through numbered version deliveries containing links and documentation. Submissions require contracts to be in `active` or `in_revision` status. Clients may accept deliveries or request revisions with written feedback, clearing auto-release deadlines until resubmission.

12.15 Wallet, Payments and Withdrawals

All financial operations execute in Pakistani Rupees (PKR) through atomic profile balance mutations.

Funding and Disbursal. Clients deposit funds via Stripe Checkout; transactions record Stripe session IDs to ensure idempotency. Freelancers disburse earnings through Stripe Connect Express accounts (Rs. 5 minimum threshold). Payout requests atomically reserve wallet balances prior to invoking Stripe transfer APIs, eliminating concurrency races. Simulated transactions (`STRIPE_DUMMY=true`) utilize signed cryptographic nonces and are blocked in production environments.

Frozen Wallets and Ledgers. Administrators may freeze suspect wallets, preventing outbound debits while allowing inbound escrow releases. All financial flows record `Payment` entities across four types (`job_escrow`, `hourly_escrow`, `hourly_invoice`, `wallet_deposit`).

Table 12.4 — Platform settings that govern money

Setting (`SystemSetting`)	Default	Effect
`platform_fee_rate`	0.005 (0.5%)	Fee deducted from the freelancer's payout on release; captured on each payment when funded
`escrow_release_days`	7	Review window after a milestone is submitted before auto-release
`withdrawal_min_amount`	Rs. 5	Smallest allowed withdrawal

12.16 Reviews and Ratings

Completed engagements permit mutual 1-to-5 star ratings with 1,000-character reviews. Reviews update aggregate reputation scores for users and gigs. Administrators retain moderation authority to hide or expunge violating reviews, dynamically recalculating ratings. Freelancers may also submit home page testimonials.

12.17 Disputes

Contract disputes are submitted with justification and categorized by contract scope: fixed-price disputes attach to specific milestones (leaving other milestones unaffected), whereas hourly disputes attach to the entire contract.

Table 12.5 — Dispute resolution outcomes

Outcome	Fixed-price (per milestone)	Hourly (whole contract)
Release to freelancer	Milestone escrow paid to the freelancer, minus the platform fee	Remaining locked balance paid to the freelancer, contract completed
Refund client	Full milestone amount returned to the client's wallet	Remaining lock refunded, contract cancelled

Outcome	Fixed-price (per milestone)	Hourly (whole contract)
Split	A chosen percentage (1–99%) goes to the freelancer (fee on that share only); the rest is refunded	The same split, applied to the remaining lock
Cancel contract	Only when the milestone was never funded and so holds no escrow; the contract and job are cancelled	Not available
Resume / reject	Dismiss the dispute: the milestone returns to `submitted` with a fresh review window	Dismiss the dispute and resume work

Dispute adjudications execute atomically and log critical severity entries in the admin audit ledger.

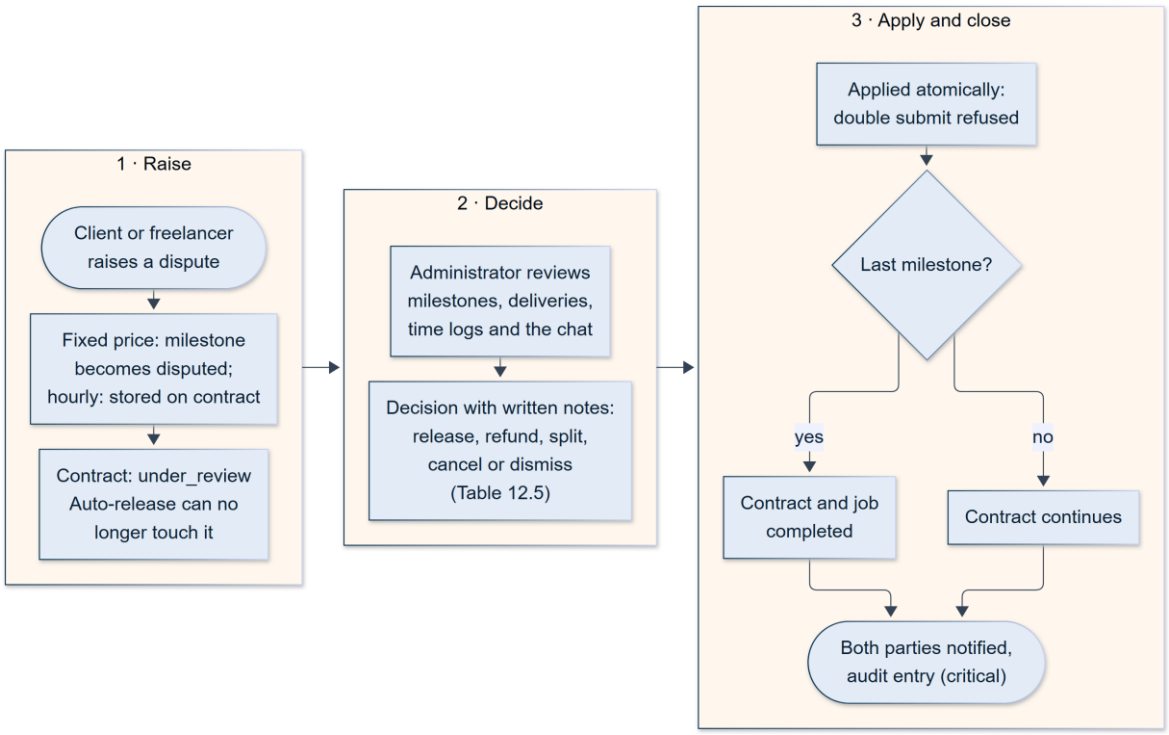


Figure 12.9 — Dispute handling for a fixed-price milestone

The administrator's decision is applied to the disputed milestone only; the contract then continues, completes, or is cancelled (Section 12.17).

12.18 Messaging

SkillKart provides one-to-one messaging supporting up to 10 cloud attachments ($\le$ 10 MB). Chat utilizes HTTP polling (3-second active conversation polling, 10-second list cadence).

Contact Anti-Leakage Guard. Outgoing messages are scanned for email addresses and phone numbers. Detected contact information is automatically masked. The system issues a warning notification on the first infraction and temporarily suspends the account upon the second infraction. Administrators can communicate directly with users under the *SkillKart Support* system persona.

12.19 Notifications

System notifications are generated via a unified, non-throwing `notify()` service, dispatching database records and native Firebase Cloud Messaging push alerts across 25 distinct events.

Table 12.6 — Notification events

Group	Events
Proposals	received, resubmitted, accepted, rejected (with a hint when resubmission is allowed)
Orders and offers	gig ordered (sent only after the deposit is paid), custom offer received, accepted, declined
Delivery and payment	deposit or milestone funded, work delivered, revision requested, payment released, milestone auto-released, escrow refunded, project completed
Withdrawals	withdrawal requested, withdrawal completed
Contracts and disputes	contract status changed, contract disputed, dispute resolved, dispute dismissed, job cancelled
Social and policy	new message, review submitted, contact-information warning, skill verification expired

12.20 Transactional Email

Transactional communications (OTPs, security notices, administrative alerts) route through a provider layer supporting the Gmail API (production HTTPS) and SMTP (development). Transient delivery failures trigger up to 3 retries with exponential backoff and jitter. Dispatched emails log to `EmailLog` for operational diagnostics, with sandbox modes available for testing.

12.21 Public Website and Discoverability

The public platform includes landing pages, talent directories, category listings, and AI utility interfaces (Buddy AI, Cost Estimator). Search engine discovery utilizes a dynamic `/sitemap.xml` generated directly from active database records and cached for 1 hour. Indexing controls are defined via `robots.txt` and Open Graph metadata tags.

12.22 Dashboards and Navigation

Role-based navigation routes users to dedicated workspace dashboards:

- **Client Workspace (`/client/*`):** Metrics dashboard, project and proposal management, wallet checkout, candidate search, and security controls.
- **Freelancer Workspace (`/freelancer/*`):** Earnings telemetry, active contracts, time tracking logs, gig management, Skill Verification Center, and withdrawal requests.

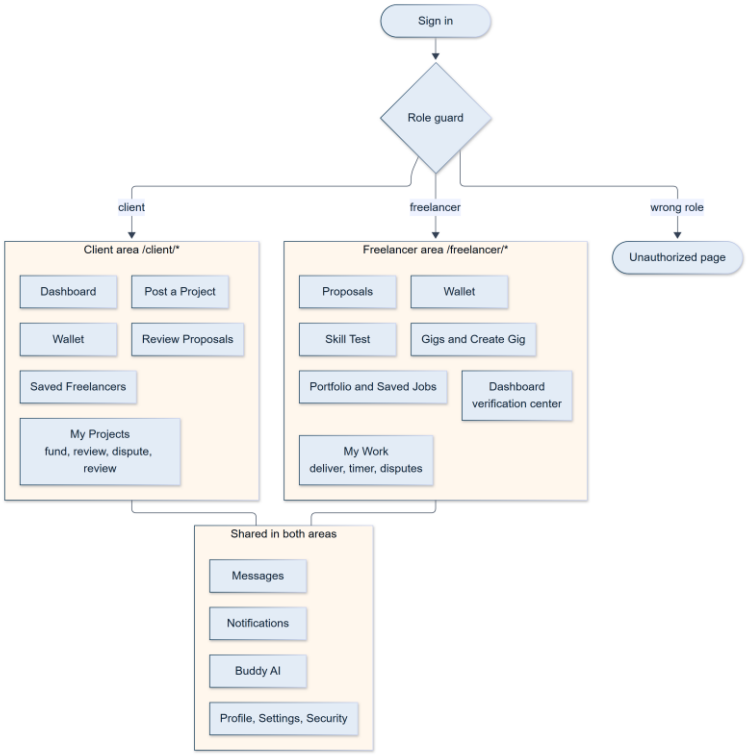


Figure 12.10 — Interaction map of the client and freelancer areas

Each role sees only its own route tree; role guards redirect everything else to an 'unauthorized' page (Section 12.22).

12.23 Reporting Content

Users may report policy violations across profiles, jobs, gigs, and reviews. Anti-abuse rules enforce server-side target resolution, limit submissions to one open report per user per entity, and impose a 10 reports/hour rate limit. Reports queue within the administration panel for moderator review and action.

13. AI Features

Artificial intelligence is a first-class part of SkillKart, woven into onboarding, discovery, bidding, pricing, and verification. This chapter describes what the AI layer does, how the requests are routed across providers, how the system behaves when providers fail, and — equally important — which intelligent-looking features are *not* powered by a language model.

13.1 AI Capabilities

The platform provides **fourteen LLM-backed capabilities**. Table 13.1 lists them with the screen that uses each. Thirteen are reachable through an API endpoint; the fourteenth (taxonomy suggestion) runs only in the background.

Table 13.1 — LLM-backed AI capabilities

#	Capability	What it does	Used in
1	**Buddy assistant**	Conversational help on freelancing, hiring, pricing, and using the platform; short replies (about 120 words) using the last eight turns of conversation	Floating widget on public pages, `/ai/buddy`, and a *Buddy AI* tab in both dashboards
2	**Cost estimator**	From a brief (title, description, category, skills, complexity, timeline) returns a budget range in PKR, a rationale, and a breakdown	`/ai/cost-estimator`
3	**Brief suggestion**	From a free-text description infers a title, a category, up to eight skills, a budget, and an experience level	"Suggest" buttons in job, gig, onboarding, and matchmaking forms
4	**Skill validation**	Normalises a typed skill ("reactjs" becomes "React.js"), rejects non-professional or offline inputs, and suggests alternatives	Skill tag inputs throughout the site
5	**Semantic skill expansion**	Returns five to eight specific skills related to a query	API only; the user-facing search uses the non-LLM skill graph instead
6	**Generate with AI**	Drafts a field from scratch or expands the user's own draft, aware of whether the writer is a client or a freelancer	About twenty text fields: job and gig descriptions, biography, portfolio, FAQs, success stories
7	**Skill-test generation**	Writes a test of 5, 10, or 15 questions at the chosen level	Skill test
8	**Skill-test evaluation**	Scores each answer between 0 and 1	Skill test
9	**CV parsing and analysis**	Extracts title, biography, skills, education, experience, and portfolio from a CV, scores it, and classifies it	Freelancer onboarding
10	**Industry recommendation**	Suggests the three most suitable industries from a fixed list of 30	Client onboarding and profile
11	**Job category suggestion**	Suggests up to three of the 68 categories	Post-a-job wizard
12	**Job subcategory suggestion**	Suggests a subcategory	Post-a-job wizard

#	Capability	What it does	Used in
13	**Proposal drafting**	Writes a first-person cover letter of 120–200 words from the job and the freelancer's profile, without inventing facts	Job details page
14	**Taxonomy suggestion**	Proposes where an unrecognised skill belongs in the skill graph, with reasoning, for administrator review	Background; reviewed in the taxonomy admin page

Each capability follows the same pattern: a provider method builds a task-specific prompt, requests a **structured JSON** response, and parses it defensively (tolerating code-fenced or slightly malformed output), so that AI output is machine-usable and predictable. The router then checks that the result has the expected shape — for example, a proposal draft must contain a cover letter and a cost estimate must have a minimum or a maximum — and treats a malformed result like a failed call.

13.2 Matchmaking: A Deterministic Engine

Matchmaking is deliberately not an LLM feature. Given a project brief — pasted into `/ai-matchmaking` or taken from a saved job — a scoring engine ranks up to 100 candidate freelancers and returns the best eight, each with a score from 0 to 100 and a list of plain-language reasons. Because it uses no external service, it is fast, free, reproducible, and unaffected by AI outages.

Table 13.2 — Matchmaking score components

Component	Maximum points	How it is computed
Skill match	50	Graph-aware overlap between required and held skills (Section 12.6); if the brief lists no skills, 5 points per candidate skill up to 35
Verified skills	10	3 points per verified skill
Rating	15	Rating out of 5, scaled to 15
Experience	10	Completed projects divided by two, up to 10
Budget fit	15	15 if the freelancer's rate is within the budget, 9 if within 30% above, otherwise 3; 8 if unknown
Availability	5	5 for full-time, 3 for part-time

The components add up to 105 and the total is capped at 100. The engine also produces an estimated cost range, from roughly one day to one week of the freelancer's hourly rate. The scoring weights are plain constants in code, which makes them easy to explain, audit, and tune; Chapter 21 discusses replacing the skill-overlap component with embeddings.

13.3 The Provider Router

All LLM calls go through a **provider router**, a single object that presents one interface to the rest of the application and, internally, tries a prioritised list of providers.

Table 13.3 — AI providers

Provider	Access	Default model	Methods implemented
Google Gemini [8]	`@google/genai` SDK	`gemini-3.1-flash-lite`	All 14
DeepSeek [14]	OpenAI-style HTTP API, JSON mode, 60-second timeout	`deepseek-flash`	All 14
Alibaba Qwen [29]	OpenAI-compatible HTTP API, 60-second timeout	`qwen-plus`	5: CV parsing, test generation and evaluation, skill validation, industry recommendation
Groq [28]	`groq-sdk`	`llama-3.3-70b-versatile`	The same 5
Anthropic Claude [27]	`@anthropic-ai/sdk`	`claude-3-haiku-20240307`	The same 5
Ollama [30] (local)	HTTP to a local server, 120-second timeout	`llama3`	7 light, interactive ones (Buddy, cost estimate, brief suggestion, skill validation, industry and category recommendation, proposal drafting)

Models for Gemini, DeepSeek, Qwen, and Ollama can be changed through environment variables.

Fallback algorithm. For each request the router takes its providers in priority order and calls the same method on each in turn. A provider that lacks the method, has no API key configured, or throws is skipped with a warning, and the first provider to return a well-formed result wins. The order is set by numbered environment variables, `AI_PRIORITY_<PROVIDER>`: `0` is tried first, higher numbers later, and a negative value such as `-1` disables the provider. The default order is Gemini → DeepSeek → Qwen → Groq → Claude → Ollama. This design answers the reality that free-tier AI services are rate-limited and unreliable: when one provider exhausts its quota or is unavailable, the next takes over, and the local Ollama model — which has no quota — is the last resort for the capabilities it supports. The effective order is logged at startup, and a real one-token call verifies that at least one provider works.

The router is intentionally simple: calls are sequential and there is no circuit breaker, so a provider that is down is retried on every request and the worst-case latency is the sum of the providers' timeouts. A circuit breaker is listed among the future enhancements.

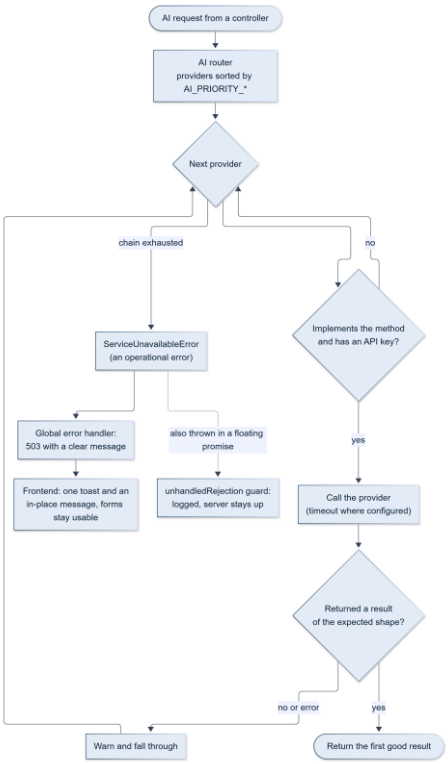


Figure 13.1 — The AI provider router: fallback, validation and the 503 path

Each provider is tried in priority order; if every provider fails, the API returns a controlled 503 and the interface shows a single notice (Section 13.4).

13.4 Resilience: AI Failure Never Crashes the Application

The AI subsystem is engineered so that **AI failures never take the application down**.

- If every provider fails, the router throws a dedicated `ServiceUnavailableError` — marked as an *operational* error — which the global error handler turns into a clean `503` response with a recognisable message: "AI is temporarily out of service."
- The process-level `unhandledRejection` handler distinguishes **operational** errors (expected, recoverable failures such as this one), which it logs and survives, from genuinely unexpected rejections, which it logs and then shuts down on. A provider outage therefore cannot terminate the server.
- The frontend client treats any failed request to an AI endpoint the same way and shows a single, consistent toast; each AI screen also renders its own in-place message, so that forms remain usable by hand.
- Four capabilities never return a 503 at all, because a sensible default exists: industry recommendation falls back to a static list of three industries, category suggestion falls back to a keyword matcher, subcategory suggestion returns an empty suggestion, and the background taxonomy suggestion simply logs and stops.
- Optional configuration follows the same philosophy: a provider without an API key is skipped, and the server starts even if no AI provider works ("proceeding without AI features").

13.5 Abuse Controls

The public AI tools do not require a login, so they are rate-limited: all of them share a limit of ten requests per minute per IP address. **Buddy** has an additional guest quota — a visitor who is not signed in may send three messages per IP address in a rolling 24-hour window, after which the interface asks them to sign in; signed-in users are not capped beyond the per-minute limit. Inputs are length-limited (for example, a Buddy message at 1,000 characters and a skill at 60) before they reach a provider.

13.6 What the Skill Test Asks of the AI

Skill verification (Section 12.4) uses two LLM capabilities. **Generation** asks the provider for the right number of questions for the chosen level, about 80% multiple choice, and retries once if the multiple-choice share falls below 65%. **Evaluation** sends only the question, the skill, and the candidate's answer to the grader; the grader scores each answer from 0 to 1, with partial credit. Because the grading prompt contains no answer key, a multiple-choice answer is judged on the grader's own knowledge of the subject. Only the Gemini, DeepSeek, Qwen, Groq, and Claude providers can serve these capabilities; Ollama cannot, and so a test requested while all five are down returns the controlled 503.

13.7 What Is Not an LLM

Three features that a visitor may reasonably think of as "AI" are deliberately not LLM calls, and keeping them separate is a design decision (Section 8.6).

- **Matchmaking** is the rules-based engine described in Section 13.2.
- **Skill search and gating** use the skill graph (Section 12.6): graph traversal and a match-score formula, not a model.
- **Face-presence proctoring** uses MediaPipe's Face Landmarker [16], a small computer-vision model that runs on the user's own device, so no camera frame ever leaves it.

13.8 Future AI Expansion

The provider abstraction makes it straightforward to add providers or models: a new provider implements the base interface and is registered in the router. Planned directions include a circuit breaker for failing providers, embedding-based semantic matching to complement the rules-based engine, grading multiple-choice answers against the stored answer key, AI-assisted dispute summaries, and automated profile-improvement coaching (Chapter 21).

14. Admin Panel

14.1 Design Rationale

The admin panel is a **separate, isolated application** under `/admin-panel/*`. Isolation prevents privilege escalation: marketplace tokens cannot grant admin access, and credentials never cross the boundary:

- **Independent Secret (`ADMIN_JWT_SECRET`):** Distinct issuer/audience ensures cross-token rejection.
- **Isolated Cookie (`admin_token`):** Stored as `httpOnly`, decoupled from marketplace bearer tokens.
- **Isolated Middleware & Limiters:** Dedicated `adminAuth`, rate limits, and express-validator chains.
- **Separate Frontend Context:** Isolated HTTP client that never touches marketplace session state.
- **Dedicated Route Tree:** Powers both the web panel and the standalone *SkillKart Admin* Android app.

The panel organizes administrative operations into **17 management modules** (Section 14.3), 7 detail screens, and a global Ctrl+K search.

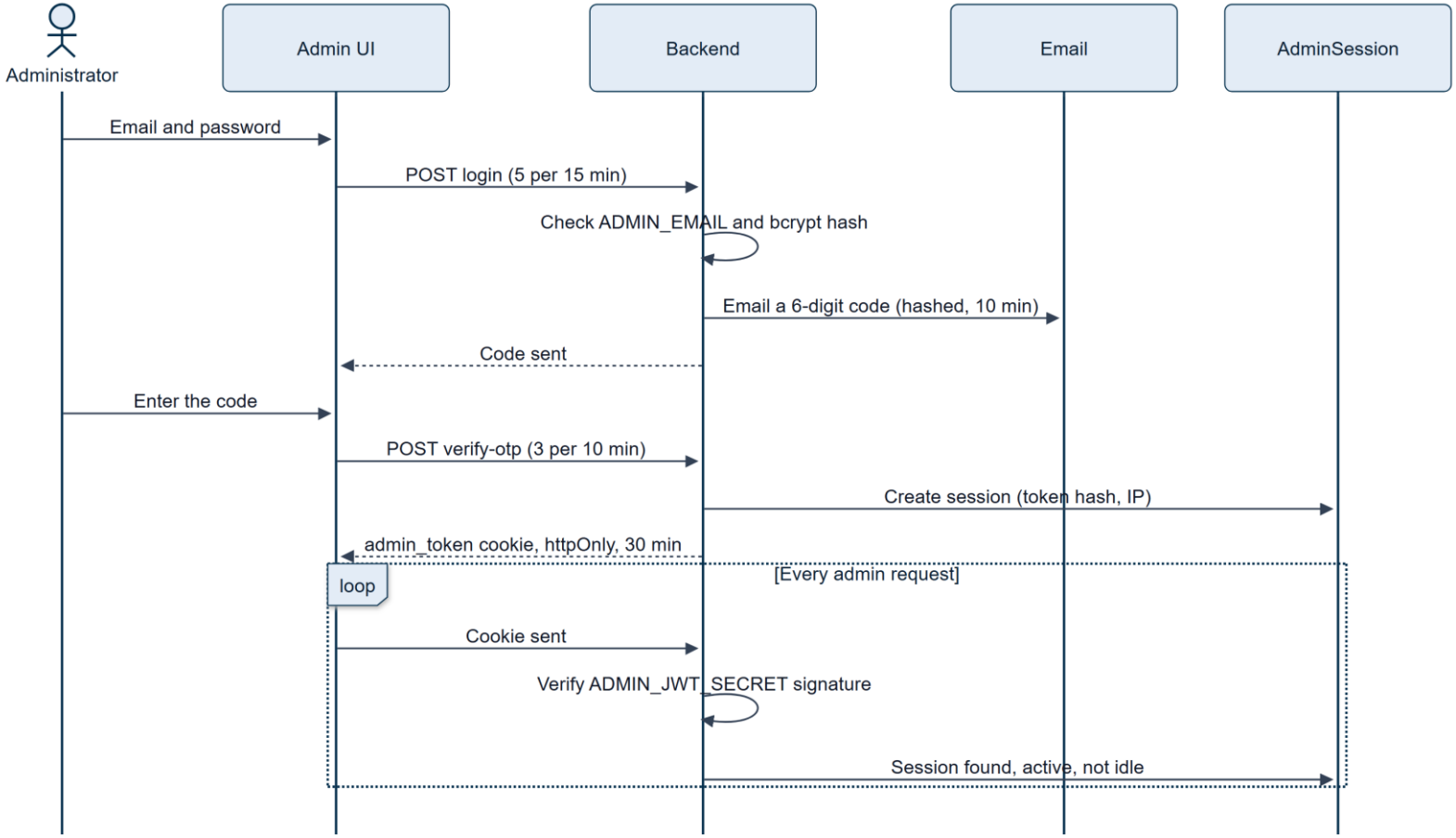


Figure 14.1 — Admin login: credentials, emailed one-time code, session cookie

The administrator's credentials are verified, a one-time code is emailed, and a successful verification creates a server-side session and sets an httpOnly cookie (Section 14.2).

14.2 Authentication and Sessions

Administrative credentials reside exclusively in server configuration (`ADMIN_EMAIL`, bcrypt-hashed `ADMIN_PASSWORD_HASH`), preventing database-injection privilege escalation.

Authentication Workflow.

1. **Credentials Validation:** Timing-safe evaluation returns generic error responses to prevent user enumeration.
2. **Emailed OTP Verification:** A single-use 6-digit OTP (SHA-256 hashed, 10-minute expiry, max 5 attempts) is dispatched to `ADMIN_EMAIL`. Route limiters enforce 5 login attempts / 15 mins and 3 verifications / 10 mins per IP.

Successful validation creates an `AdminSession` (storing SHA-256 session hash, IP, user-agent) with a 30-minute sliding inactivity timeout and sets the `admin_token` cookie. The cookie is marked `Secure` and `SameSite=None` across cross-domain HTTPS deployments. CSRF protection mandates custom request headers or allow-listed origins. An encrypted AES backup credential supports emergency maintenance (`POST /auth/superadmin/login`). The session carries the `super_admin` role, with permission fields structured to support future multi-admin RBAC (Chapter 21).

14.3 Modules

Table 14.1 — Admin panel modules and their capabilities

Module	Capabilities
1. Dashboard	Live platform overview refreshed every 30 seconds: users (total, freelancers, clients, active now, new today), jobs, contracts, open disputes, platform revenue, escrow balance, fees today, wallet totals, pending withdrawals, open reports, message count, recent admin activity, and a health card (database state, uptime, memory, Node version, environment)
2. Analytics	Range selector (7 days, 30 days, 90 days, 1 year); KPI cards with period-over-period change (gross volume, platform fees, accounts, job completion rate); charts of revenue by month, users by role, jobs by status, and top categories; a top-earners table with CSV export
3. Users	Search and filter; suspend and unsuspend with a reason; soft-delete; edit an account or a full profile; change role (never to `superadmin`); set an avatar; verify an email without a code; pass or reset a skill test; set the identity, phone, portfolio, and payment verification flags and the overall status; message the user as SkillKart Support; adjust the wallet; tabs for contracts, jobs, payments, wallet, reports, and activity
4. Jobs	Filter and search; edit any field; **close** (which rejects pending proposals), **reopen**, **soft-delete**. Gig orders appear here too, since they are jobs
5. Contracts	Inspect history, time logs, payments, and deliveries; edit terms; **pause**, **resume**, **cancel** (refunds funded escrow to the client), **force-complete** (pays outstanding escrow to the freelancer), **reassign** to another freelancer, add internal notes. Every change is written to the contract history with the administrator's email
6. Disputes	One queue of hourly and per-milestone disputes; full context (milestones, deliveries, time logs, the complete chat); six resolution outcomes (Table 12.5)
7. Payments	List and detail; **refund** and **release** a payment with a recorded reason (milestone payments are routed through the same approval function as a client's click; refunding a disputed milestone is refused)

Module	Capabilities
8. Wallets	List wallets with a balance; **adjust** a balance (signed amount up to ±1,000,000, with a reason, never below zero); **freeze** and **unfreeze** a wallet with a recorded reason (a frozen wallet cannot fund, lock, or withdraw money; Section 12.15)
9. Withdrawals	Queue of manual (`bank_transfer` and `paypal`) withdrawal requests; **complete** with a payout reference, or **reject** with a reason, which refunds the wallet automatically. Stripe withdrawals are automated and never need this queue
10. Transactions	A unified, date-ordered view of payments and withdrawals, with export
11. Chat	Search and read any conversation; reply as **SkillKart Support**; message any user; download a transcript
12. Reviews	Hide (with a reason), restore, or permanently delete a review; the reviewee's rating is recalculated each time
13. Notifications	**Broadcast** an announcement (title and message) to all users, all freelancers, or all clients, shown in each user's notification feed
14. Reports	Triage queue for user and content reports: assign to oneself, resolve with notes, or dismiss with a reason; categories and priorities as in Table 10.7
15. Monitoring	Three tabs: server errors (with a *resolved* toggle), API request logs, and email delivery logs
16. Audit Logs	Read-only, paginated record of administrative actions (Section 14.4)
17. Settings	View and edit platform settings grouped by category (Section 14.5)

Beyond the modules, a **Global Search** (Ctrl+K) in the header searches users, jobs, contracts, payments, withdrawals, reports, and reviews at once, and seven **detail screens** open from the lists (user, job, contract, dispute, payment, conversation, and report).

14.4 Accountability and the Audit Log

Sensitive administrative actions record append-only entries in `AdminAuditLog`, capturing administrator identity, action code (~45 distinct actions across 16 categories), resource ID, before/after values, IP, user agent, and severity (*info*, *warning*, *critical*). Direct financial and account state mutations (refunds, releases, wallet adjustments, dispute resolutions, contract terminations, suspensions) record at *critical* severity. Log persistence executes asynchronously to ensure platform operations are never blocked by logging failures.

14.5 Platform Settings

Platform parameters are stored as key–value documents (`SystemSettings`), seeded with defaults the first time an administrator signs in, and edited on the Settings page; changes to the maintenance toggle are audited at critical severity.

Table 14.2 — System settings

Key	Default	Status
`platform_fee_rate`	0.005 (0.5%)	**Live** — fee on every release; captured on each payment when funded
`withdrawal_min_amount`	Rs. 5	**Live** — minimum withdrawal
`escrow_release_days`	7	**Live** — milestone review window

Key	Default	Status
`maintenance_mode`	false	Stored and audited; reserved for a maintenance banner
`enable_ai_features`	true	Stored; reserved for a global AI switch
`enable_email_notifications`	true	Stored; reserved
`max_login_attempts`	5	Stored; the lockout currently uses a fixed rule (Table 12.1)
`otp_expiry_minutes`	10	Stored; the OTP expiry is set by an environment variable

A one-time correction in the seeding code brings legacy fee and withdrawal values in line with the current defaults for rows that no administrator has edited, so a deliberate administrator change is never silently reverted.

14.6 Administrative Surfaces Outside the Panel

Three legacy administrative interfaces predate the isolated panel and use marketplace RBAC:

- **Taxonomy Administration (`/admin/taxonomy`):** Allows `admin` and `superadmin` roles to curate the skill graph and review AI `TaxonomyProposal` suggestions.
- **Legacy User API (`/users`, `/admin`):** Legacy endpoints protected by higher-role rules; superseded by the admin panel module.
- **Content Management Backend (`/cms`):** 13 versioned CMS endpoints supporting draft scheduling and rollback, currently awaiting frontend integration.

14.7 Exports

Six lists — users, jobs, contracts, payments, withdrawals, and transactions — can be exported as **CSV**, **Excel** (formatted with a header row and filters), or **PDF** (landscape tables). Exports are produced on the server and named with the date; on the Android apps, files that the app generates itself (conversation transcripts, the top-earners CSV) are saved to the app's cache and handed to the system share sheet, because a browser-style download has nowhere to go inside a WebView (Section 15.5). Each export is recorded in the audit log.

15. Mobile Applications

15.1 Overview

Objective 9 of the project was to deliver native mobile access without maintaining a second codebase. SkillKart therefore ships **two Android applications built from the same React source** using Capacitor [15], a runtime that embeds a web application in a native Android shell and exposes native capabilities to it through plugins.

- **SkillKart** (`com.skillkart.app`) — the client and freelancer application. It contains the authentication and onboarding screens, the six public marketplace pages (browse projects, browse gigs, browse freelancers, and the job, gig, and freelancer detail pages), and both dashboards.
- **SkillKart Admin** (`com.skillkart.admin`) — a separate, standalone application that contains only the admin panel (Chapter 14), for administrators who need moderation access on the move without installing the marketplace application.

These are not a bare web page in a frame; each runs the web application inside the Android WebView component [40], with native capabilities added around it. Each integrates the platform: the camera, push notifications, the hardware back button, offline detection, the splash screen and status bar, an in-app system browser, and the native share sheet. The web build is unaffected by any of this: the same components run in all three targets, and a single check decides where behaviour differs.

15.2 One Codebase, Three Build Targets

A single environment value, `VITE_APP_TARGET`, selects the target at build time, and two helper functions derived from it — `isMobileApp()` and `isAdminApp()` — are the only places where the shared code branches between web and native.

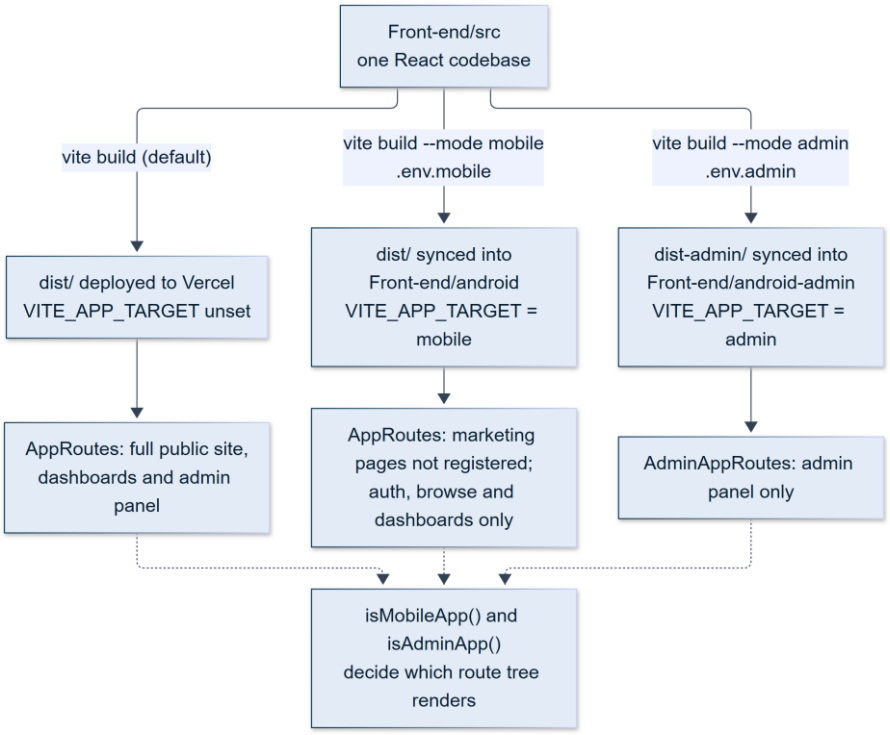


Figure 15.1 — One codebase, three build targets

The same Front-end/src is built three ways; isMobileApp() and isAdminApp() decide which route tree renders (Section 15.2).

Target	Build	Route tree
Web	`vite build` (default)	The full public site and both dashboards, plus the admin panel
Android app	`vite build --mode mobile`, then Capacitor sync	`AppRoutes`, with the public marketing, legal, AI-tool, skill, and category pages **not registered**
Admin Android app	`vite build --mode admin`, then Capacitor sync	`AdminAppRoutes`: only `/admin-panel/*`; every other path redirects to the admin login

In the Android build, the marketing and legal pages are neither registered nor loaded: they are `React.lazy` imports, so their code and assets are never fetched on a phone. The browse, detail, and authentication pages are kept, because a freelancer on a phone needs to find work and a client needs to find talent. The application's WebView has a fixed origin of `https://localhost`; this origin is included in the backend's CORS allow-list, and it is exempted from reCAPTCHA, because Google's reCAPTCHA v3 is unreliable inside an embedded WebView. The mobile build also omits the reCAPTCHA site key, so nothing even attempts to load.

15.3 Platform Integration

Table 15.1 — Capacitor plugins and their use

Plugin	Used for
`@capacitor/app`	The Android hardware back button (navigates back, or minimises the app on a root screen); receives the OAuth return deep link

Plugin	Used for
`@capacitor/browser`	Opens external links and the OAuth consent screen in a Chrome Custom Tab, with a way back into the app
`@capacitor/camera`	*Take Photo* in every image upload (Section 15.5)
`@capacitor/filesystem`, `@capacitor/share`	Saving and sharing files that the app generates (Section 15.5)
`@capacitor/network`	The offline screen: shown after three consecutive offline readings, with a Retry button
`@capacitor/push-notifications`	Receiving push notifications (Section 15.4)
`@capacitor/splash-screen`	A branded splash, hidden only once the sign-in state has been restored
`@capacitor/status-bar`	Brand-coloured status bar

Other behaviours specific to the native build:

- **External links never navigate the app away.** On a native WebView, `window.open` or `target="_blank"` replaces the application with no way back, so all external links go through a helper that opens the Chrome Custom Tab. This also covers server-generated downloads such as the admin exports.
- **Smooth rendering.** A stylesheet that applies only inside the native app disables `backdrop-filter` and hides scrollbars. These are the main sources of visual jank on Android WebViews and have no effect on the web build.
- **Navigation.** The root route resolves the signed-in user's role and sends them to the right dashboard instead of blindly redirecting to the login page (Section 20.6).
- **Reduced chrome.** The floating Buddy widget and the cookie banner are not shown; Buddy is reached through a tab in the dashboard sidebar.

15.4 Native-Specific Engineering

Social sign-in. Google does not allow OAuth sign-in inside an embedded WebView, so on Android the consent screen opens in the system browser. After the user approves, the provider redirects to a registered custom URL (`skillkart://auth-callback`), which Android hands back to the app; the app closes the browser tab, takes the token from the URL fragment, and continues through the same handler as the web.

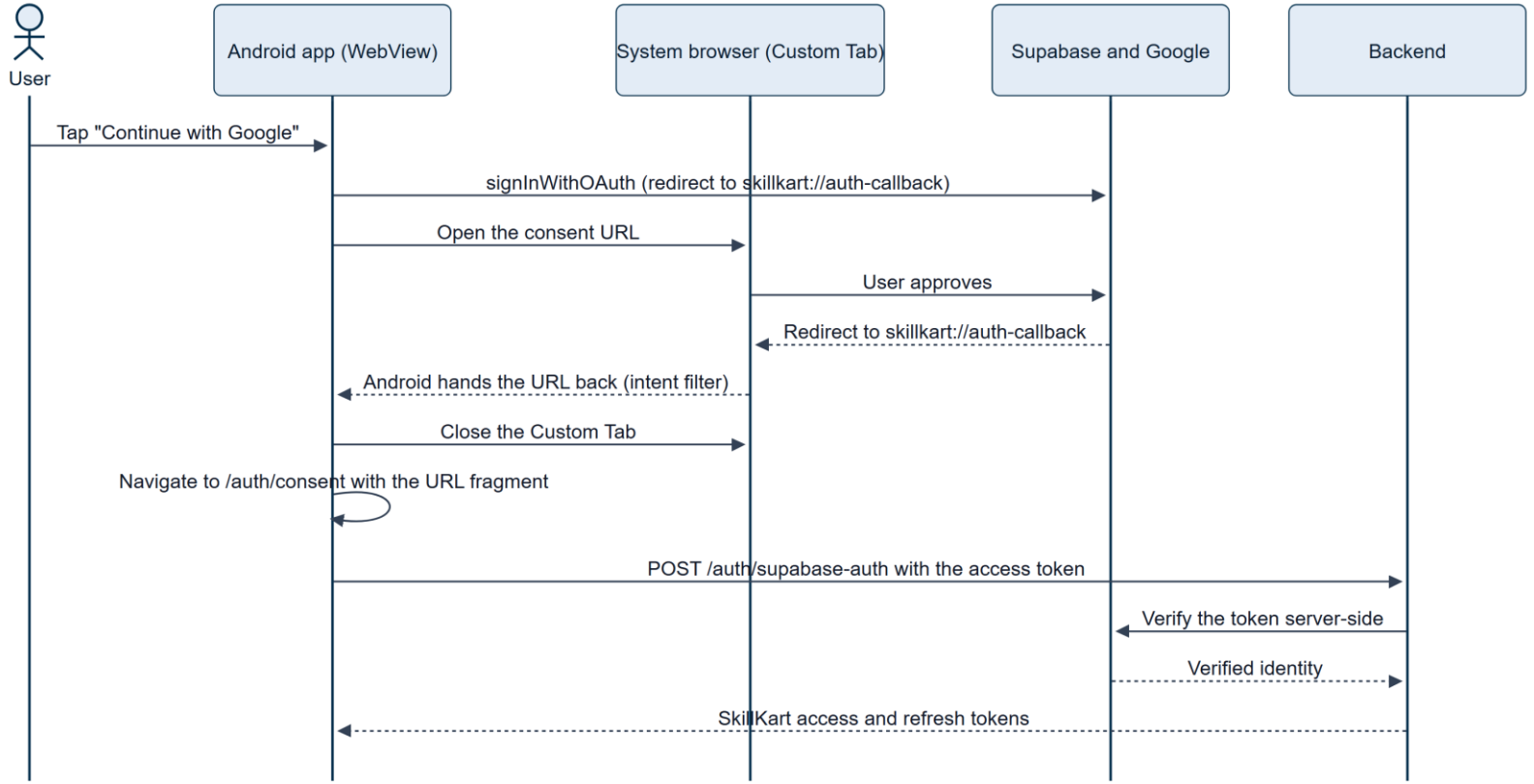


Figure 15.2 — Mobile social sign-in through the system browser and a deep link

Google blocks OAuth in embedded WebViews, so the consent screen opens in the system browser and the code returns through a registered URL scheme (Section 15.4).

Push notifications. Push delivery is hooked into the single `notify()` function (Section 12.19), so every event that creates a notification can also reach the phone. The server uses Firebase Cloud Messaging [17] through the Firebase Admin SDK; a device registers its token through an authenticated endpoint, tokens are stored on the user, and a token that FCM reports as no longer valid is pruned. Signing out sends the device token with the logout request, so the server unregisters that phone in the same authenticated call; signing out of every device, or changing the password, clears all of a user's tokens. Tapping a notification opens the screen named in the notification's `link`. Like every optional integration, push **fails open**: if the Firebase credential is absent, every call is a silent no-op, and on the client the registration is gated by a build flag because registering without a Firebase configuration file would crash the native app.

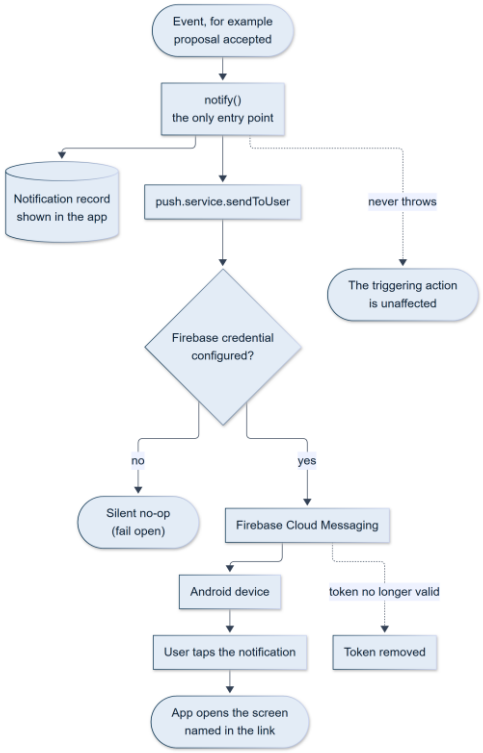


Figure 15.3 — Push notification path from an event to the phone

notify() stores the notification, then asks the push service to deliver it; failures never affect the triggering action (Section 15.4).

Skill-test proctoring in a WebView. The camera guard was rebuilt for Android after the first implementation, which used a face-detection library on the GPU, proved to silently detect nothing on a WebView. The current MediaPipe-based guard runs on the **CPU** inside the Android app, requests the front camera, and pauses its three-second countdown while the WebView's camera feed is frozen — measured on the emulator, the feed freezes for three to four seconds at a time. The detection model and its runtime are bundled into the application and served from the app's own origin, never from a CDN (Section 20.5).

15.5 Photos and Files

Photo upload. Every image upload uses one reusable component, `ImageSourcePicker`, which is a drop-in replacement for a hidden file input with the same `click()` interface. On the web, it opens the device's file picker directly. In the Android app it first shows a chooser, **Take Photo** or **Choose from**

Device; *Take Photo* uses the real `@capacitor/camera` plugin rather than the HTML `capture` attribute, because the latter relies on WebView file-chooser behaviour that varies too much across Android manufacturers to be dependable. Documents such as résumés keep a plain file input, because a camera cannot produce a PDF.

Downloads and sharing. A file that comes from a server URL (for example, an admin export) is opened through the system browser. A file that the app builds itself from data already on screen — a conversation transcript or a CSV — has only a `blob:` URL that exists in the WebView's memory and cannot be handed to a separate browser. A small helper, `downloadFile(blob, filename)`, therefore writes the file to the app's private cache (which needs no storage permission) and opens the native share sheet so that the user can save or send it. On the web it is an ordinary browser download.

15.6 Build and Distribution

The Android projects live in `Front-end/android` and `Front-end/android-admin`, with their own application identifiers and signing configuration. Both target Android API level 36 and support Android 7.0 (API 24) and later; both are at version 1.0. A build is produced by compiling the web bundle for the chosen mode, synchronising it into the Android project, and running Gradle (`cap:sync` and `cap:android` for the main application; `cap:admin` for the admin application). The Gradle wrapper is pinned to a version that fails with an unrelated-looking "unsupported class file major version" error if a newer Java runtime than expected is used, so the build is run with Java 21.

Signed APK files for both applications are kept in the repository's `Apk/` folder; the latest main-application build (10 October 2026, about 25 MB) includes the bundled face-detection model needed for proctoring.

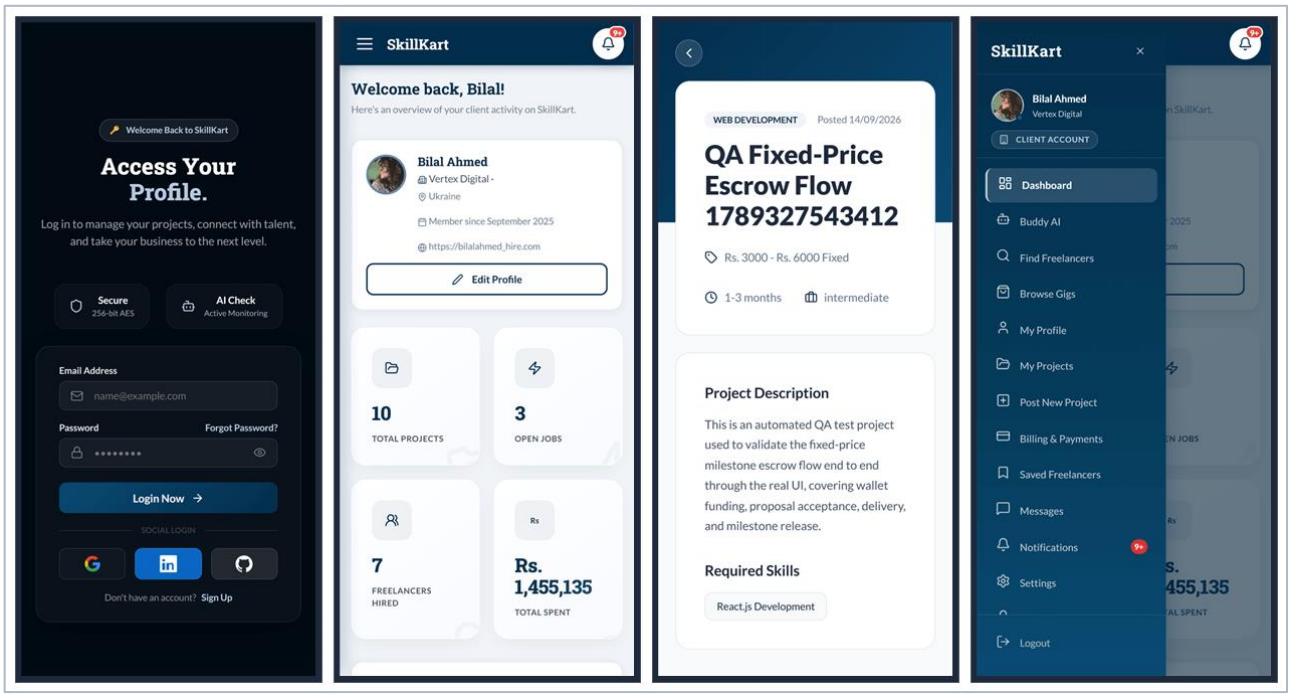


Figure 15.4 — Native Android application: sign-in, dashboard, job details and navigation drawer
The same React interface rendered inside the Capacitor WebView with native hardware integration (Section 15.1).

15.7 Verification and Limitations

The mobile behaviour was verified in two ways: by driving the build for the mobile target in a desktop browser at phone size with browser automation (which found, among others, the redirect loop described in Section 20.6), and by installing the debug build on an Android emulator, connecting to it through the Chrome DevTools protocol, and running the proctoring and navigation flows. Broader testing on a range of physical devices is a recommended next step.

Known limitations of the current mobile build are:

- **Admin push is not active.** The standalone admin application contains the push plugin and its Firebase configuration, but the push hook activates only for marketplace sign-ins, and the admin panel's isolated authentication never triggers it; administrators also receive no notifications to push. Wiring this up requires deciding which events matter to an administrator.
- **Both applications are at version code 1.** A store release would require versioning, a release signing certificate under the publisher's control, and a policy review.
- **iOS** is not built. The same Capacitor project could target it, but it needs a macOS build environment and was out of scope (Chapter 21).

16. Security

Security in SkillKart is layered across transport, authentication, authorisation, input handling, abuse prevention, data protection, financial integrity, and accountability. No single layer is relied on alone: each is designed so that the failure of the layer in front of it does not by itself expose money or data. The structure follows the categories of the OWASP Top Ten [10].

16.1 Authentication

Authentication is described in detail in Section 12.1; its security properties are summarised here.

- **Password hashing.** Passwords are hashed with bcrypt [24] at cost factor 12 in a Mongoose pre-save hook, so plaintext is never stored, and the password field is excluded from queries by default.
- **Password policy.** At least eight characters, with an upper-case letter, a lower-case letter, and a digit.
- **One-time passwords.** Six digits, ten-minute expiry, stored only as a SHA-256 hash, deleted automatically on expiry, five attempts, and a one-minute resend cooldown. Two-factor login is on by default.
- **Account lockout.** Five consecutive wrong passwords lock the account for two hours.
- **Tokens.** JSON Web Tokens [9] signed with separate secrets for access and refresh tokens. Refresh tokens **rotate** on every use and are tracked by identifier and family, so that replay of a rotated token revokes the whole family and increments the account's `tokenVersion`, which invalidates every token previously issued — the same mechanism that makes "sign out of all devices" and password changes effective immediately.
- **Per-request verification.** The authentication middleware reloads the user on every request and checks that the account is active, unlocked, and at the token's `tokenVersion`; the role claim inside the token is never trusted for authorisation, the database role is used. A blocked account becomes read-only and the interface shows a suspended notice.
- **Social login** is verified server-side with the identity provider, never by trusting a client-decoded token.

16.2 Authorisation (RBAC)

SkillKart defines six roles with a numeric level: `superadmin` (100), `admin` (90), `contenteditor` (50), `freelancer` and `client` (30), and `publicviewer` (10). Role middleware guards every protected route on the server (`requireRole`), and `ProtectedRoute` guards the frontend route trees, redirecting a user with the wrong role to an *unauthorised* page. Permission strings (`requirePermission`) are used on the CMS routes to separate authoring from publishing and deleting. A *higher-role* rule protects user management: an administrator can neither act on a user of equal or higher level nor assign such a role. Ownership is checked in the controllers, so that a client can read only their own contracts and a freelancer only their own proposals.

Marketplace roles and the admin panel are **strictly separated**. The panel uses its own secret, cookie, middleware, and session store (Section 14.1), so possession of a marketplace token of any role — including `superadmin` — never grants access to the panel.

16.3 Input Validation and Injection Protection

- **Schema validation.** Request bodies are validated with `express-validator`: eleven validator files with roughly ninety-five named chains for the marketplace API, and eighteen further schemas for the admin panel. Fields are length-limited, typed, and checked against allow-lists (for example, a job category must be one of the 68 defined categories); writable profile and job fields are whitelisted so that lifecycle and identity fields can never be set from a request body.
- **NoSQL injection.** `express-mongo-sanitize` removes query operators from request bodies, queries, and parameters. It is mounted *after* the body parsers, deliberately: the sanitiser only inspects keys that already exist, so mounting it earlier would sanitise nothing.
- **Cross-site scripting.** User-supplied rich text is stripped of every HTML tag and attribute before storage, and React escapes output by default.
- **Uploads.** Files are limited to 10 MB and to an allow-list of image, PDF, Word, and text types, checked first by extension and MIME type and then by their **magic bytes**, so that a renamed file cannot slip through; files are stored in cloud storage, never on the server's disk.
- **Request size.** JSON and form bodies are limited to 10 MB.

16.4 Transport and Headers

- **Helmet** [37] sets secure HTTP headers on the API, including a Content Security Policy that restricts scripts and the default source to the API's own origin.
- A **CORS allow-list**, built from configuration, restricts which origins may call the API; a request from an origin that is not on the list is refused with a 403.
- `trust proxy` is enabled, because the hosting platform terminates TLS in front of the application; without it, the application would believe every connection were insecure, would refuse to mark cookies `Secure`, and would see the proxy's address in place of the client's, which would undermine IP-based rate limiting.
- **Cookies.** The admin cookie is `httpOnly`, and `Secure` with `SameSite=None` over HTTPS (Section 20.8) [39]. Every state-changing admin request must also show that it came from the admin client — a custom header, an allow-listed `Origin`, or a Bearer token (Section 14.2) — and a request carrying a foreign `Origin` is already refused by the CORS layer.
- **Webhooks** from Stripe and Supabase are authenticated by verifying a signature over the raw request body; the Stripe handler refuses to process an event whose signature does not verify.
- **Compression** and cookie parsing support efficient transport.

16.5 Abuse Prevention

Table 16.1 — Rate limits

Limiter	Window / maximum (per IP)	Applies to
Global	15 minutes / 1,000 requests	All `/api/` routes
Authentication	15 minutes / 5 requests	Registration, login, password-reset request
One-time password	15 minutes / 20 requests	OTP verify and resend, password reset
Public tools	1 minute / 10 requests	The six public AI tools, contact form, newsletter
Admin login	15 minutes / 5 requests	Admin login
Admin code	10 minutes / 3 requests	Admin OTP verification
Admin API	1 minute / 100 requests	All other admin routes
Reports	1 hour / 10 requests (per signed-in user)	Filing a report
Mobile-app exemption	15 minutes / 12 requests	Requests that use the reCAPTCHA exemption for the mobile app (shared by registration, login, contact, and newsletter)

- **reCAPTCHA v3** [19] guards registration, login, the contact form, and the newsletter form. It is invisible and score-based: Google returns a score between 0 and 1 and the server rejects a request below a threshold (default 0.5). Each protected endpoint declares a distinct *action* name and the server checks that the submitted token was minted for that action; without this check, a token obtained from a public form could be replayed against the login endpoint.
- **Defence in depth.** reCAPTCHA sits *alongside* rate limiting, not in place of it: rate limiting bounds request volume, whereas reCAPTCHA scores legitimacy, and each covers the other's blind spot — a slow scripted attack and a fast human one.
- **Fail-open policy.** If reCAPTCHA is unconfigured or Google is unreachable, verification is skipped and the request proceeds. This is a deliberate availability-over-security trade-off at a single, bounded point: an outage at a third-party bot-detection service must never lock every user out of the platform's own login. The residual risk — loss of bot mitigation during such an outage — remains bounded by rate limiting.
- **The mobile-app exemption is capped.** The Android app's WebView cannot run reCAPTCHA reliably, so requests presenting its fixed origin are exempt from the score check. That origin is a header a script could also claim, so every request that uses the exemption counts against a much tighter per-IP budget (twelve per fifteen minutes) shared by the four protected endpoints: a real app user needs a handful, a bot needs thousands.
- **Contact-information masking** and a two-strike block discourage taking transactions off the platform (Section 12.18).

16.6 Data Protection

- Passwords and administrator credentials are stored only as bcrypt hashes; OTP codes and admin session tokens are stored only as SHA-256 hashes.

- The encrypted SuperAdmin credential file is protected with AES (through the `crypto-js` library) under a key from the environment, with owner-only file permissions (Section 14.2).
- On the client, the access token is held in memory and per-tab session storage; the refresh token is kept in local storage in obfuscated form. The obfuscation prevents casual reading, not determined extraction, and is documented as such; the real protection is that refresh tokens rotate and are revocable.
- Public API responses use explicit field whitelists, so that phone numbers, wallet balances, security settings, and payment identifiers never appear on a public profile.
- Request logs store method, path (without the query string), status, duration, user, and IP address — never request bodies, headers, or tokens — so there is nothing sensitive to redact.
- Secrets live only in environment files that are never committed; only `.env.example` files, containing no values, are tracked.

16.7 Financial Integrity

Because SkillKart moves money, several controls are specific to payments (Chapter 12).

- **Check first, write after.** Accepting a proposal validates the wallet before any record is written, and reverses its writes if funding then fails.
- **Atomic, conditional updates.** Every debit, credit, release, refund, and activation is a single conditional database update (for example, "debit only if the balance is at least the amount"), and money-bearing records are *claimed* before they are paid, so that two simultaneous requests cannot both succeed.
- **Reserve before paying out.** A Stripe withdrawal debits the wallet before calling Stripe and refunds the debit if Stripe fails, closing a race in which two simultaneous requests could both pass the balance check.
- **Idempotency.** Each Stripe session is credited at most once; gig activation, offer acceptance, and dispute resolution are single atomic claims, so a double click, a retried webhook, or a racing background job cannot apply an operation twice.
- **A price-staleness guard** refuses an acceptance if the price changed after the client viewed it.
- **Frozen wallets.** An administrator-frozen wallet cannot fund, lock, pay out, or withdraw; each debit repeats the check inside its atomic update (Section 12.15).
- **Signed simulated payments.** A simulated checkout-session identifier carries a nonce and an HMAC over its phase, job, amount, and user, so only an identifier the server issued is accepted (Section 12.15).
- **Start-up guards.** In a production environment the server refuses to start if `STRIPE_DUMMY` is set (a simulated payment would tell a client that money had moved when none had) or if `SKILL_TEST_EASY_MODE` is on (which would reveal test answers and defeat the verification system).

16.8 Accountability and Monitoring

Administrative accountability is provided by the audit log (Section 14.4). Operationally, every API request is recorded in `ApiLog`, every server error in `ErrorLog` (with a *resolved* flag), and every email attempt in `EmailLog`, all visible on the Monitoring page.

16.9 Resilience as a Security Property

Availability is part of security. The application is hardened against third-party failure: an AI outage returns a controlled 503; an unconfigured integration degrades quietly; and a process-level guard distinguishes **operational** errors — expected, recoverable failures, which it logs and survives — from genuinely unexpected rejections and exceptions, which it logs and then exits on so that the platform's supervisor can restart a clean process. The server also shuts down gracefully on `SIGTERM` and `SIGINT`, closing the HTTP server and the database connection, and forcing an exit after ten seconds if that stalls.

16.10 Limitations and Hardening Roadmap

A candid security review identifies several areas in which the current implementation is adequate for its purpose but would benefit from further hardening before a public launch:

- **Token lifetime.** The access token's default lifetime is seven days. A shorter lifetime (for example, fifteen minutes) would limit the value of a stolen token; rotation and revocation already make this change straightforward.
- **Random-number source.** One-time codes are generated with a general-purpose random-number generator; a cryptographically secure generator (`crypto.randomInt`) should be used.
- **Proctoring is client-side.** The camera and browser guards, and the per-question timers, run in the user's browser. The server records the violations a client reports but cannot independently enforce them, so a modified client could bypass them. Server-side enforcement of the time limits, and grading multiple-choice answers against the stored key, are listed in Chapter 21.
- **Administrator model.** The panel is single-operator, and its permission checks are placeholders until the reserved multi-administrator model is enabled.
- **Headers on the frontend host.** The Content Security Policy applies to the API host; the static frontend host sets no security headers of its own.
- **Log retention.** Request, error, email, and audit logs have no time-to-live, so they grow without bound.

These are hardening and policy gaps rather than defects in the money and contract state machines described in this report; they are listed so that the system is described as it is.

17. Workflow

This chapter follows the platform's principal workflows from start to finish, tying together the features described in Chapters 12 to 14.

17.1 Registration and Onboarding

A visitor chooses a role — client or freelancer — and registers with a name, email address, and password. The password is hashed, the account is created, and a six-digit code is emailed. After entering the code, the user is signed in and sent to the onboarding wizard for their role (eight steps for a freelancer, five for a client). A user who signs in with a social provider instead is created with a neutral role and picks Client or Freelancer on a short role screen before onboarding. When the wizard is submitted, the profile is complete and the user lands on their dashboard — or, for a freelancer, on a success screen that offers to take a skill test now or later.

17.2 Login and Session

On login, the credentials and the account state are checked. If two-factor login is enabled, a code is emailed and must be entered. Tokens are then issued, the user is sent to the dashboard for their role, and the session survives page reloads and new tabs through the rotating refresh token; it ends after 30 minutes of inactivity or when the user signs out.

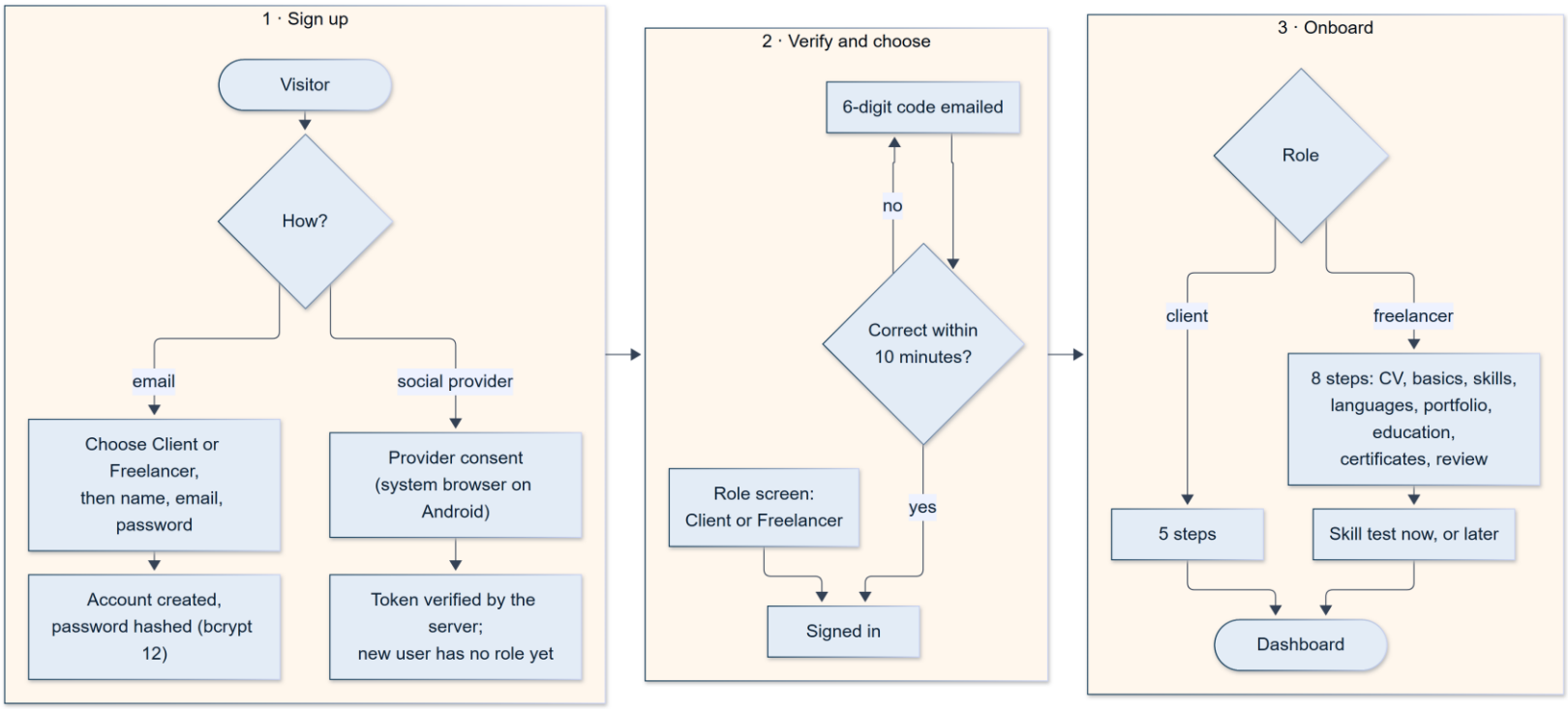


Figure 17.1 — Registration, onboarding and first sign-in

Role is chosen before the account exists for email sign-up; social sign-in users choose it afterwards (Sections 17.1 and 12.1).

17.3 Skill Verification

A freelancer opens the Skill Verification Center, chooses a skill and a level, and starts the test. After the camera check, the test runs in a locked full-screen view with a per-question timer. When it ends, the answers are graded by AI and each skill scoring at least 25% is verified at the chosen level for six months (Section 12.4).

17.4 Hiring on a Fixed-Price Job

1. The client **posts a job**, publishing it with any wallet balance.
2. A verified freelancer **applies** with a proposal, optionally drafted by AI; the apply gate checks eligibility first.
3. The client **accepts** the proposal, optionally splitting the price into milestones. The server checks that the wallet covers the first milestone, creates the contract and milestones, and locks the first milestone's amount. The freelancer is notified.
4. The freelancer **delivers** the first milestone, which starts the review window.
5. The client **approves** — or lets the window lapse — and the milestone is released: the fee is deducted and the payout is credited to the freelancer's wallet.
6. For each further milestone, the client **funds** it when ready, and steps 4 and 5 repeat.
7. When the last milestone is released, the contract and the job are **completed**, and each party can **review** the other.

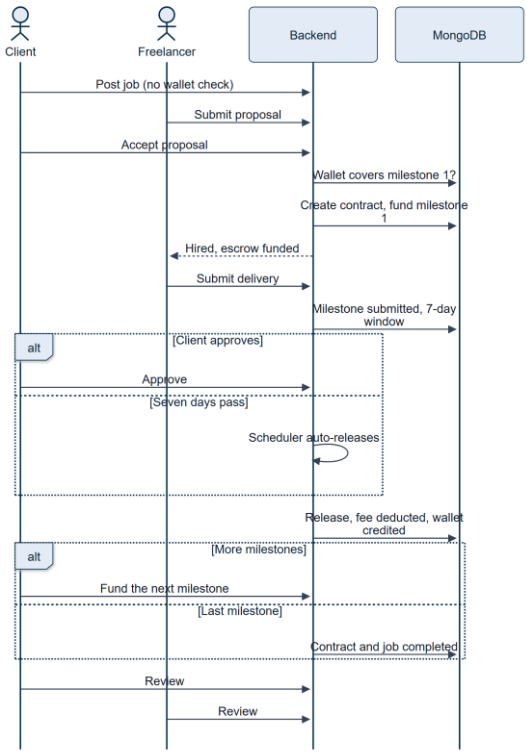


Figure 17.2 — End-to-end workflow of a fixed-price engagement

The funding check precedes every write; release is the same function whether triggered by a click or by the scheduler (Section 17.4).

17.5 Working on an Hourly Contract

When the client accepts an hourly proposal, a week's estimate is locked. The freelancer then starts a timer, works, and stops it; the time is billed against the lock at once, and the freelancer's wallet is credited, net of the fee. If the lock runs out, further logging pauses until the client tops up, at which point any pending time is billed in order. When either party asks to end the contract and the other agrees, the remaining time is settled and what is left in the lock returns to the client's wallet.

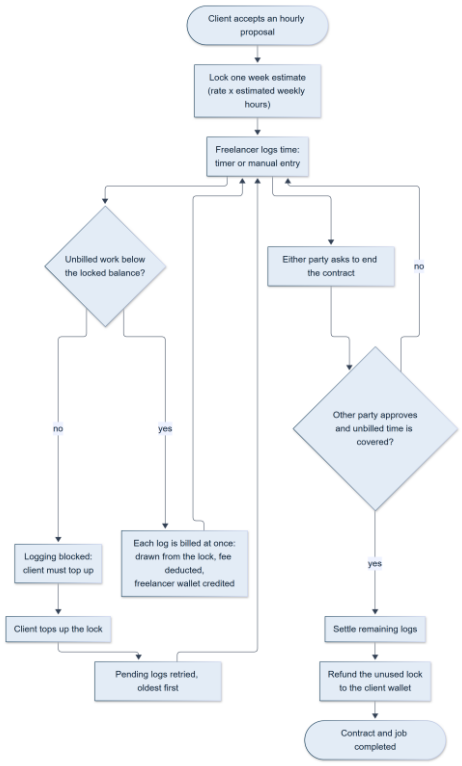


Figure 17.3 — Hourly engagement: lock, bill, top up, end

A lock is a hold, not a spend: whatever is unused returns to the client (Section 17.5).

17.6 Buying a Gig or Accepting a Custom Offer

A client opens a gig, picks a tier, and writes the requirements. The system creates an unpaid order, and the client pays a 50% deposit from the wallet or through Stripe. Only then does the freelancer see the order. After delivery, the client either requests a revision or pays the second half, which completes the order and credits the freelancer. A custom offer reaches the same state through a different door: the freelancer sends it in chat, the client accepts, and the same unpaid order is created.

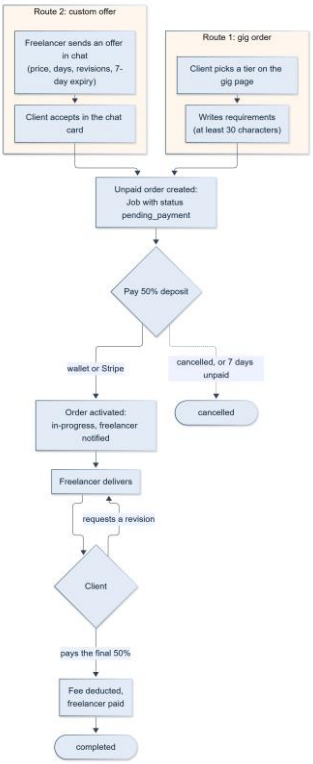


Figure 17.4 — Gig order and custom-offer workflow

A gig order and a custom offer converge on the same pending_payment order (Section 17.6).

17.7 Delivery, Revision, and Release

A delivery is a message with links. For a fixed-price milestone, the client can accept it, which releases the money, or request a revision with written feedback, which returns the milestone to the freelancer; this loop can repeat. If the client does nothing for the length of the review window, the scheduler releases the milestone automatically.

17.8 Disputes

If either party is dissatisfied, they raise a dispute with a reason. For a fixed-price contract the dispute attaches to the milestone under review; for an hourly contract, to the contract. The disputed money is frozen — it can no longer be auto-released — until an administrator reviews the evidence, including the full chat, and chooses to release, refund, split, cancel, or dismiss (Table 12.5). The decision is applied atomically, both parties are notified, and the action is audit-logged.

17.9 Withdrawing Earnings

A freelancer connects a Stripe account once, then requests a withdrawal of at least the minimum amount. The wallet is debited first; Stripe is then asked to pay out; if Stripe fails, the debit is refunded. Withdrawals the backend supports by bank transfer or PayPal are queued for an administrator, who completes or rejects them.

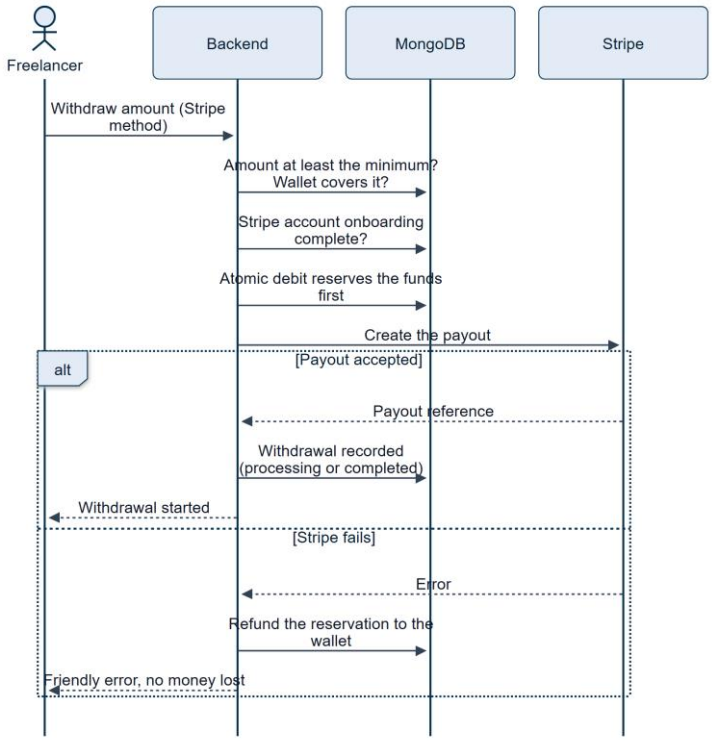


Figure 17.5 — Withdrawal by Stripe Connect payout

The wallet is debited before Stripe is called, which closes the double-payout race (Section 17.9).

17.10 Reviews

After completion, each party can leave a rating and a comment for the other. Reviews update the reviewee's average rating, appear on public profiles, and can be hidden by administrators for policy violations.

17.11 Administrative Moderation

An administrator signs in through the isolated two-step login, monitors the dashboard, and handles the queues: disputes, manual withdrawals, and reports. They may suspend a user, close a job, pause or cancel a contract, adjust a wallet, or reply to a user as SkillKart Support. Every consequential action is recorded in the audit log with the administrator's identity and the previous and new values.

17.12 Where the Money Goes: Worked Examples

The following examples use the default platform fee of 0.5%. All amounts are in rupees.

Table 17.1 — Worked examples of the three escrow models

Scenario	Steps	Result
Fixed price, Rs. 100,000 in two milestones (40,000 and 60,000)	At hire, 40,000 is debited from the client's wallet and locked. On approval, the fee is 40,000 × 0.5% = 200 and the freelancer receives 39,800. The client then funds the second milestone, 60,000; on approval the fee is 300 and the freelancer receives 59,700	Freelancer total 99,500; platform fee 500; the client was never asked to lock more than one milestone at a time
Hourly, Rs. 1,500 per hour, estimated 20 hours a week	At hire, 1,500 × 20 = 30,000 is locked. The freelancer logs 12 hours: 12 × 1,500 = 18,000 is drawn down, the fee is 90, and the freelancer receives 17,910. The contract then ends	12,000 remains in the lock and is refunded to the client's wallet
Gig order, Rs. 10,000	The client pays a deposit of 5,000. After delivery, the client pays the remaining 5,000. The fee is 10,000 × 0.5% = 50	The freelancer receives 9,950

18. Testing

18.1 Approach

Given the breadth of the system and the sensitivity of its financial and authorisation logic, testing concentrated on four properties: **functional correctness** of the contract and payment lifecycles, **data integrity** of the money flows, **resilience** against the failure of third-party services, and **usability** across roles and devices.

The project's verification was **manual and tool-assisted rather than automated**. The repository does not contain an automated unit or integration test suite: the backend's `npm test` script is a placeholder, and no test runner is configured in either application. Verification instead relied on four complementary practices, described below: structured manual testing by role, end-to-end checks driven through a real browser, scripted checks of individual backend functions against real data, and repeatable demo data. The absence of an automated regression suite is acknowledged as the most significant gap in the testing effort and is the first item in the future-work list (Chapter 21).

18.2 Manual and Exploratory Testing

Every major flow was exercised by hand under each user role: registration and onboarding, job posting, gigs and gig ordering, proposals, the contract lifecycle, milestone and hourly escrow, messaging, reviews, disputes, and every admin module. Both the happy path and the edge cases were covered, including role mismatches, invalid inputs, unauthorised access attempts, insufficient wallet balance at hire, concurrent actions on the same record, and interrupted flows such as a closed browser tab during payment. The backend was also exercised through its interactive Swagger documentation and its `/health` endpoint.

18.3 Real-Browser End-to-End Verification

The most productive technique was driving the running application through a real browser using Playwright [34], with real AI providers, a real backend, and throwaway test accounts that were removed afterwards. This was chosen deliberately, because a number of defects are invisible to code review and to unit-level checks: they appear only when a real user's clicks, timing, and browser state interact. Table 18.1 lists some of the defects this found.

Table 18.1 — Defects found through real-browser testing

Defect	How it was found	Resolution
A freshly opened second tab, or a browser restart, logged the user out everywhere	An end-to-end run opened successive fresh tabs; the bootstrap re-saved the *old* refresh token after the server had rotated it, so the next load replayed a rotated token and triggered the theft response	The client now persists the new token returned by the refresh call
The onboarding wizard's *Continue* button silently did nothing on a validation error	A click-through of onboarding with invalid input: the toast helper had no `warn` method, so the call threw	Method corrected; every call site of the same bug class was swept

Defect	How it was found	Resolution
The *Connect Stripe* button never sent its request	Clicking through the freelancer wallet: a toast helper was called as a function instead of through its methods, before the API call was made	Three call sites fixed; verified by reaching Stripe's hosted onboarding
Every gig order appeared "Delivered" immediately, and a final payment could be released with no delivery	A real gig-order flow: Mongoose saved an empty `delivery` object on every job, which is *truthy*	The field no longer defaults to an object, and every check tests `delivery.submittedAt`
An infinite redirect loop between `/` and `/login` in the mobile build	A phone-viewport browser run, with the browser's history calls instrumented to log the code that triggered each navigation	A role-aware root route replaces the unconditional redirect (Section 20.6)
Please fix the errors before saving on an unchanged client profile	The real validator chains were run against every client's would-be payload	Five stacked causes fixed (Section 20.9)
A camera stream that resolved after the user left the test left the camera on	Fault injection into the proctoring hook	A start-sequence guard discards and stops superseded streams (Section 20.5)

Where a flow depends on a camera, the tests used the browser's fake-device options with an image of a face, a blank track to simulate a covered camera, and an injected `NotFoundError` to simulate a device with no camera. For the native application, the debug build was installed on an Android emulator and driven through the Chrome DevTools protocol.

18.4 Scripted Backend Checks

For logic that is awkward to reach through the interface, small throwaway scripts called backend functions directly against real data and removed their effects afterwards: for example, a script that ran the actual validator chains against every seeded profile; a script that invoked each notification helper and confirmed the resulting records; and a script that sent a push message through the real Firebase service and confirmed the structured response. Tokens for API-level checks were minted directly from the authentication service, which avoids the login rate limit and the two-factor step. Before such a run, a snapshot of the affected collections was taken and restored afterwards, so that the shared development database stayed clean.

A later hardening pass applied the same approach with one important refinement: because the development and production configurations can name the same cloud database, every check was run against an **isolated, throw-away local MongoDB** that the test harness created and destroyed, never against the shared cluster. A single harness exercised the fixes in-process (about seventy-five assertions covering profile edits, one-time-code resend, the answer key, skill expiry, wallet freezing, signed payment sessions, the admin request guard, the reCAPTCHA exemption limit, the hosted-environment guards, push-token removal, and user reports), and a real-browser run (twenty-two assertions) drove the Report dialog, the corrected public pages, and the job page that no longer shows a client's email.

18.5 Test Data: the Demo Seeder

A **demo seeder** (`npm run seed:demo`) populates a realistic, interconnected marketplace across a simulated year of activity: about 200 users (100 freelancers and 100 clients), more than 300 jobs, more than 200 gigs, and over a thousand proposals, together with hundreds of contracts, payments, and reviews and thousands of messages and notifications. It builds every reference from a record that already exists, so the dataset is consistent by construction, and it writes data in the *current* model: real `Milestone` documents each linked to its own `Payment`, disputes on the milestone for fixed-price contracts and on the contract for hourly ones, and wallet-lock payments for hourly work. Two flags adjust a run: `--small` seeds a compact dataset for quick smoke tests, and `--keep` appends instead of clearing.

Table 18.2 — Fixed demo accounts

Account	Role	Notes
`test.freelancer0@skillkart.com`	Freelancer	Top-rated, about 41 completed projects, verified
`test.freelancer1@skillkart.com`	Freelancer	Mid-level
`test.client0@skillkart.com`	Client	SME
`test.client1@skillkart.com`	Client	Startup

These four accounts use the password `Pass123!@#`, all other seeded accounts use `Demo123!@#`, and two-factor login is switched off for all of them to make demonstrations quick. The seeder is a **destructive** tool: by default it drops twenty marketplace collections in whichever database `MONGODB_URI` names, so it first shows the target database and asks for its name to be typed back (`--yes` skips the prompt for a throwaway database), refuses a production environment unless `--allow-production` is given, and only runs when executed directly, so merely loading the file can no longer start a seed. It must still never be pointed at real data. Other scripts support operations: `setup:superadmin` creates the encrypted SuperAdmin credential, and two idempotent, dry-run-by-default migrations (`migrate:hourly-escrow` and `migrate:skill-verification`) bring existing records in line with later design changes. A skill-graph seed script loads the roughly 180-node taxonomy.

Keeping the seeder current is itself part of the work: when escrow moved from one payment per contract to one per milestone, the seeder needed a substantial rewrite, as otherwise the demonstration data would have drifted from the real schema (Section 20.12).

18.6 Resilience Testing

The AI subsystem was exercised by running the application with invalid or missing provider credentials, confirming that the API returned a controlled `503`, that the interface displayed a single notice while keeping its forms usable, and that the server stayed up. Email was exercised in sandbox mode and with the provider switch, payments in simulated mode, and optional integrations — push, reCAPTCHA, Supabase, Redis — in an unconfigured state to confirm that dependent features degrade rather than error. Authorisation was exercised through role-mismatch redirects and forbidden-action attempts.

18.7 Interface and Responsiveness Testing

The interface was verified on desktop and phone-sized viewports using browser automation to capture and inspect real rendered screens, and on the Android emulator. This practice surfaced and fixed a series of layout defects, among them a stepper that overflowed small screens, a notification bell that overlapped a header toggle, stacked small buttons, and dead padding at the top of pages, as well as a mobile viewport-height and authentication-layout issue and animated-background jank in the WebView. Because the technique is cheap, it was repeated throughout development rather than done once.

18.8 Performance Considerations

Listing endpoints use server-side pagination (default ten, maximum one hundred) and indexed queries; the production frontend build is code-split, with twenty-two pages loaded lazily; the sitemap is cached for an hour; and identical in-flight requests are de-duplicated on the client. Two known costs are accepted and documented: chat and project pages poll every few seconds, and two list calls on the dashboards return only the most recent ten records. No formal load testing was performed.

18.9 Test Gaps

For completeness, the following gaps are acknowledged: there is no automated regression suite; there was no formal load or penetration testing; and the Android applications were tested on an emulator and in a phone-sized browser, not across a range of physical devices. These are carried into Chapter 21.

19. Results

19.1 Achievement of Objectives

All project objectives (Chapter 5) were achieved and demonstrated on the running system, against realistic seeded data.

Table 19.1 — Achievement of the project objectives

#	Objective	Status	Evidence
1	Dual-sided marketplace: gigs, jobs and proposals, custom offers	✓ Achieved	Sections 12.7–12.10; gig orders, proposals, and chat offers all end in a tracked, paid engagement
2	Integrated AI	✓ Achieved	14 LLM-backed capabilities behind a six-provider router, plus a rules-based matchmaking engine (Chapter 13)
3	Secure escrow payments, wallets, withdrawals	✓ Achieved	Per-milestone and balance-lock escrow, Stripe top-ups and Connect payouts, configurable fee, auto-release (Sections 12.12–12.15)
4	Multi-factor verification	✓ Achieved	Email OTP, optional two-factor login, proctored three-level skill tests, skill-gated applying and publishing (Sections 12.1, 12.4, 12.5). An automated identity check was not built; an administrator can set an identity flag manually
5	Comprehensive administrative panel	✓ Achieved	17 modules with isolated authentication and an audit log (Chapter 14)
6	Strong security	✓ Achieved	Rotating tokens, validation, sanitisation, rate limits, bot detection, audit trail (Chapter 16); hardening roadmap in Section 16.10
7	Explicit contract lifecycle	✓ Achieved	State machines for jobs, proposals, contracts, milestones, payments, and withdrawals (Sections 10.5 and 12.11)
8	Resilience and graceful degradation	✓ Achieved	AI fallback and controlled 503, fail-open optional integrations, operational-error guard (Sections 13.4, 16.9)
9	Native mobile access	✓ Achieved	Two Capacitor-based Android applications with camera, push, offline, and back-button integration (Chapter 15)

19.2 System Statistics

Table 19.2 — System statistics

Metric	Value
Application source files	475 (197 backend, 202 frontend, 76 stylesheets)
Lines of code	117,939 (36,475 backend; 50,378 frontend JavaScript and JSX; 31,086 CSS)
Data models (collections)	40

Metric	Value
API route modules	26 (25 under `/api/v1/*` plus the isolated `/api/v1/admin-panel`)
API endpoints	317 under `/api/v1` (93 admin panel), plus 6 root and webhook routes
Controllers	35 (26 marketplace and 9 admin panel)
Service files	49 (26 core, 9 AI router, 14 email)
Middleware / validator / utility modules	10 / 11 / 15
Frontend pages / reusable components / API modules	79 / 57 / 24
Backend runtime dependencies	42 (and 5 development)
Frontend runtime dependencies	30 (and 12 development)
AI providers integrated	6 (Gemini, DeepSeek, Qwen, Groq, Claude, Ollama)
LLM-backed AI capabilities	14 (plus non-LLM matchmaking and on-device proctoring)
Admin panel modules	17 sidebar modules, 7 detail screens
Notification event types	25 (plus the administrator broadcast)
Scheduled jobs	5
User roles	6
Native Android applications	2 (client and freelancer; administrator)
Git commits	more than 300

19.3 Requirements Coverage

All 37 functional requirements in Tables 6.2 to 6.6 are implemented; the non-functional requirements in Table 6.7 are met by the mechanisms named there. Where an implementation is narrower than the requirement's wording suggests, the section cited in the requirement describes the exact behaviour.

19.4 Outcome

The result is a **complete, working, production-oriented** freelance marketplace rather than a prototype. It delivers the full engagement lifecycle — registration and verification, discovery, proposals and orders, escrow-secured work, delivery and review, disputes, and withdrawals — with AI assistance, layered verification, comprehensive administration, and native mobile access. It also demonstrates sound engineering practice: a layered architecture, explicit state machines, atomic money operations, resilient integration of unreliable third-party services, and defence in depth.

19.5 System Screenshots

The figures below show the running system captured on the seeded demonstration dataset described in Section 18.5. All names, companies, and amounts are fictitious. Further screen references and endpoint mappings are collected in Appendix D.

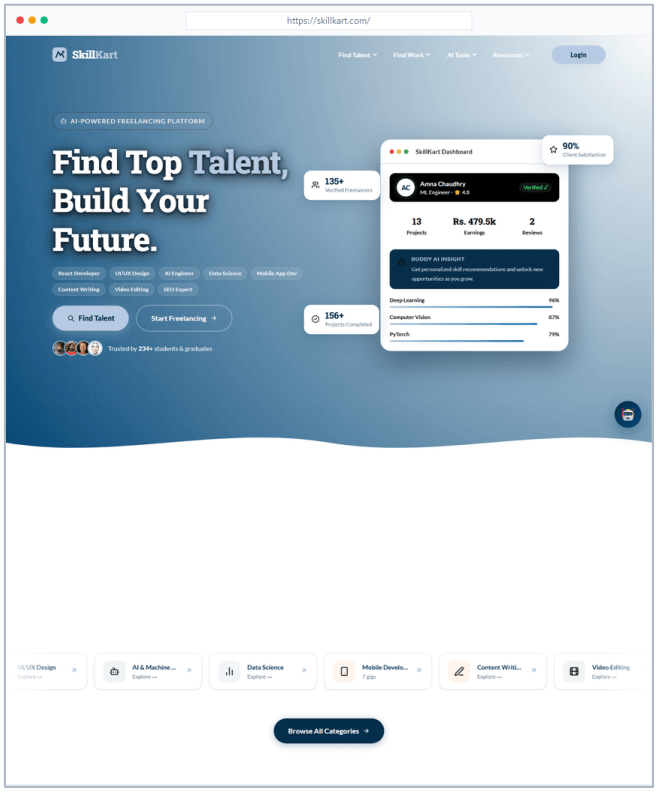


Figure 19.1 — Public home page

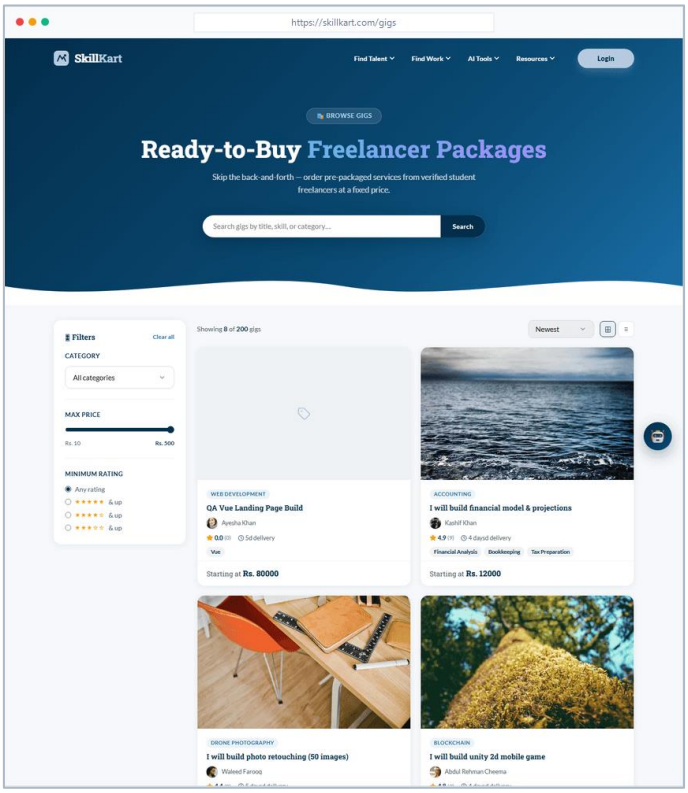


Figure 19.2 — Browse Gigs marketplace

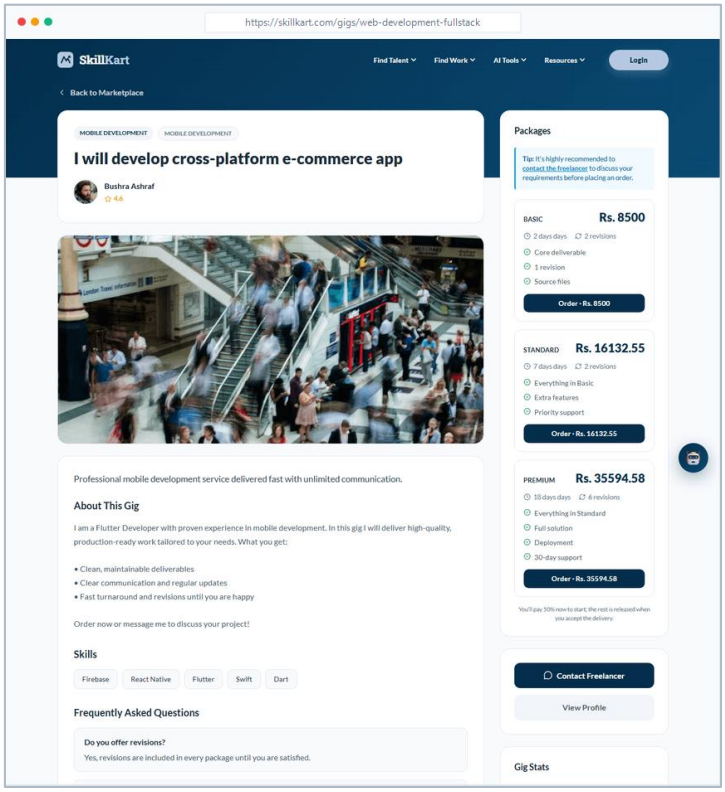


Figure 19.3 — Gig detail with tiered packages and requirements

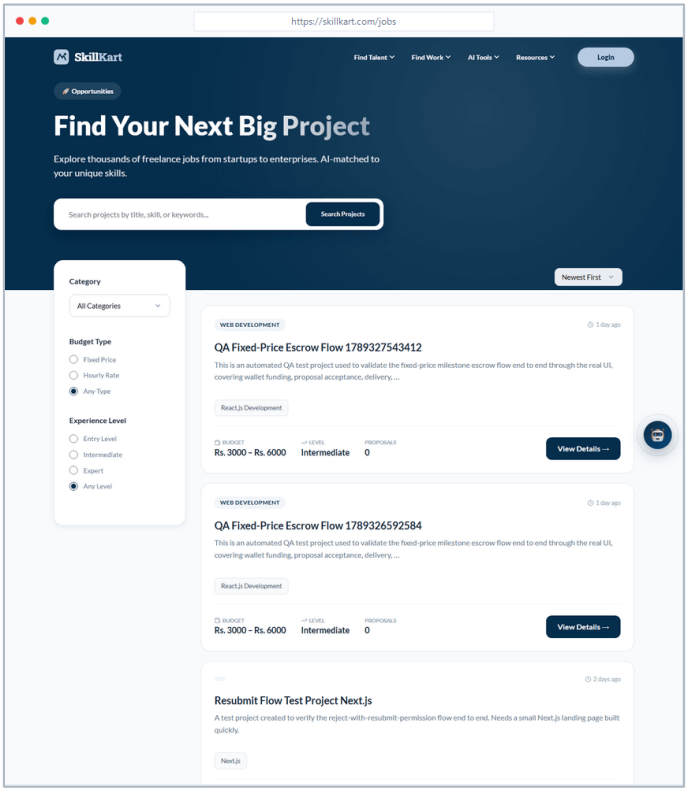


Figure 19.4 — Browse Projects and open client postings

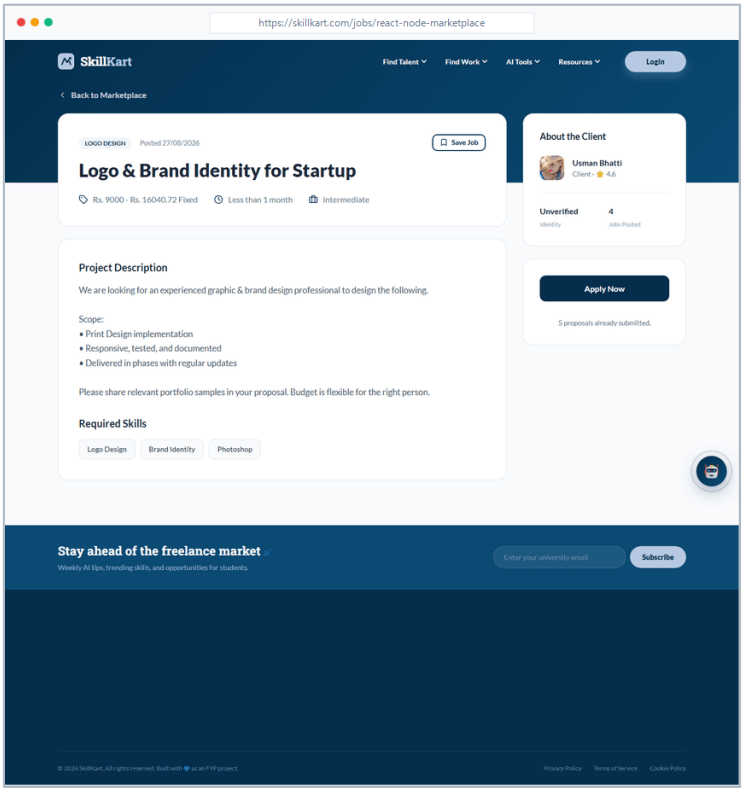


Figure 19.5 — Job detail and proposal submission

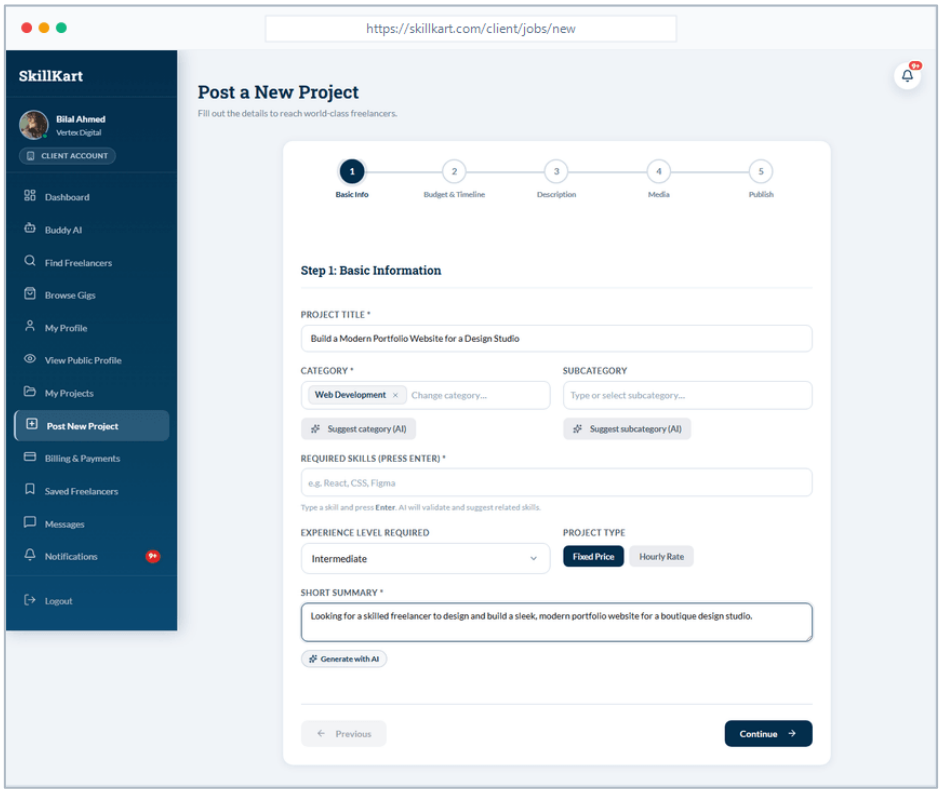


Figure 19.6 — Client post-a-job wizard

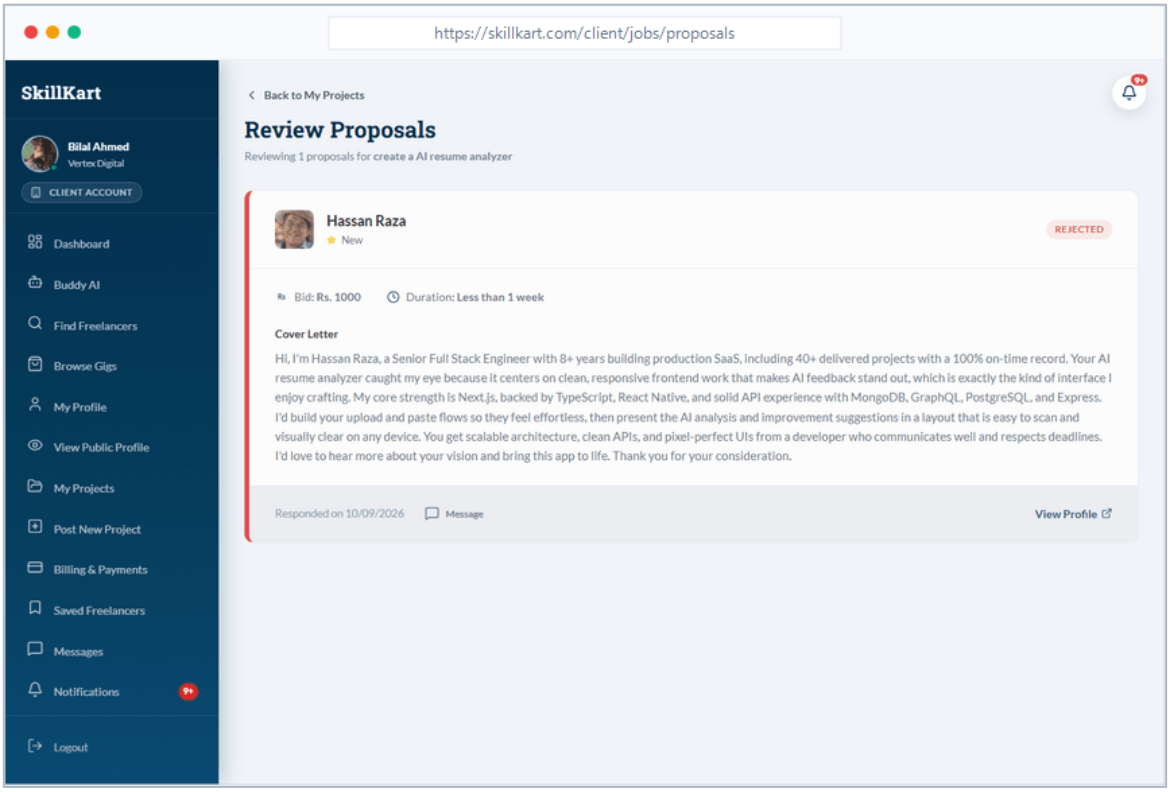


Figure 19.7 — Client reviewing received proposals

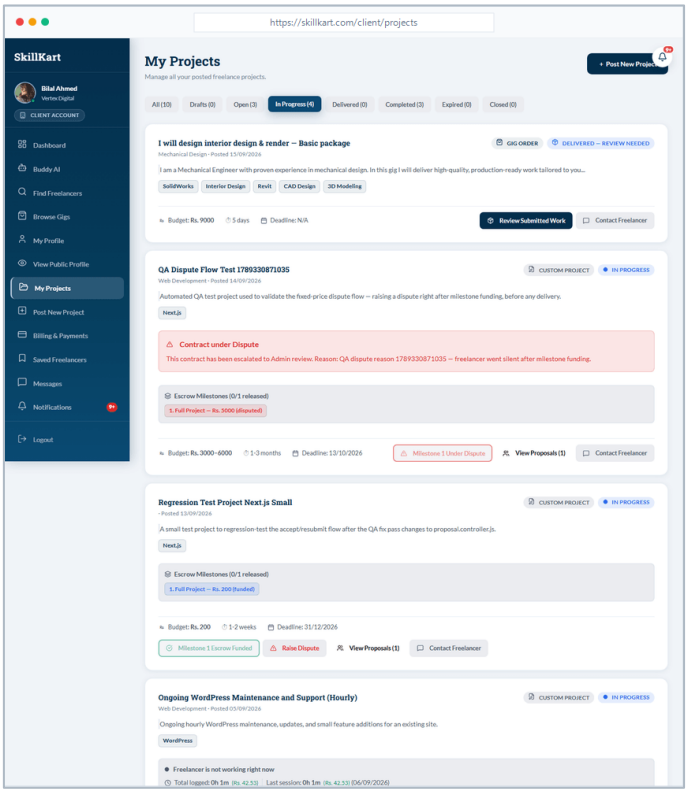


Figure 19.8 — Client My Projects with milestone escrow status

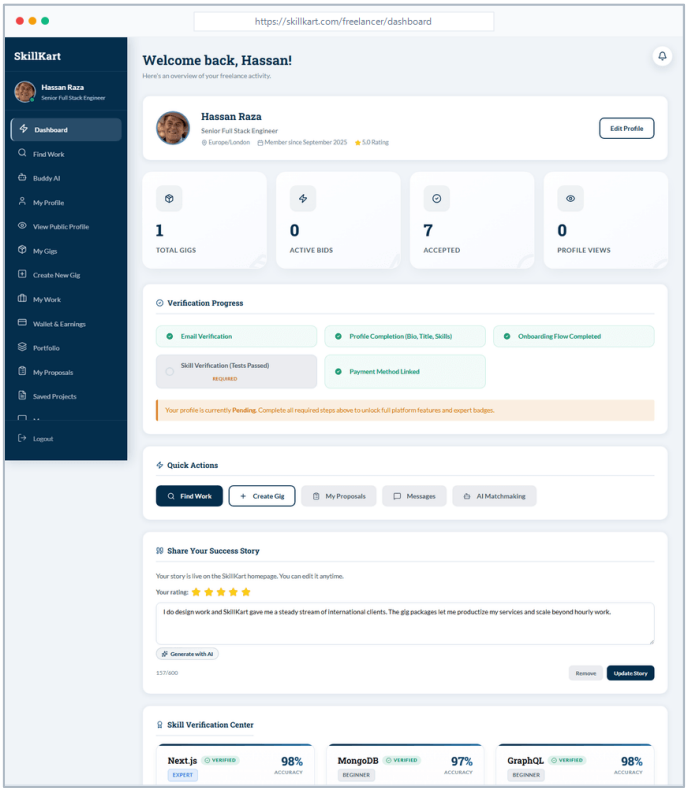


Figure 19.9 — Freelancer dashboard and verification center

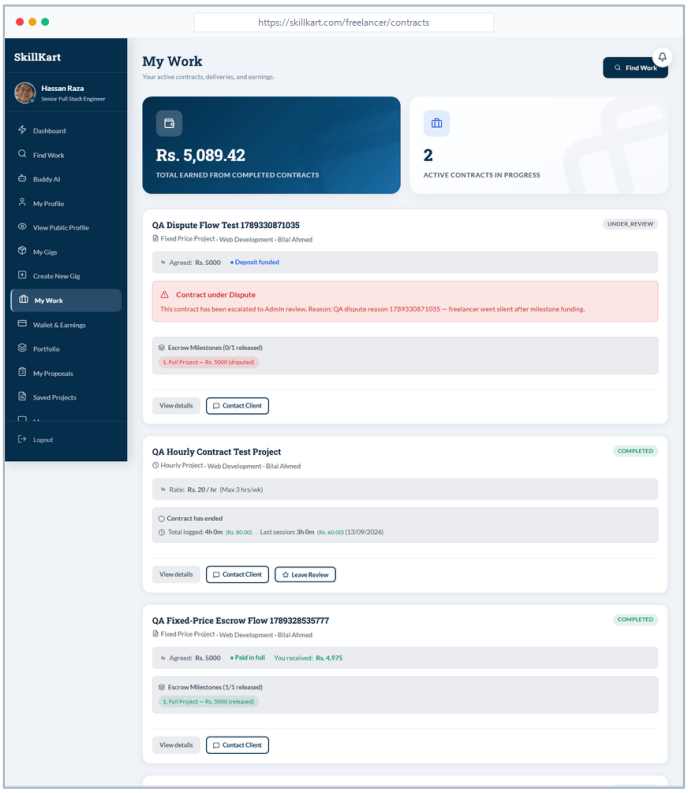


Figure 19.10 — Freelancer active contracts, timer, and work delivery

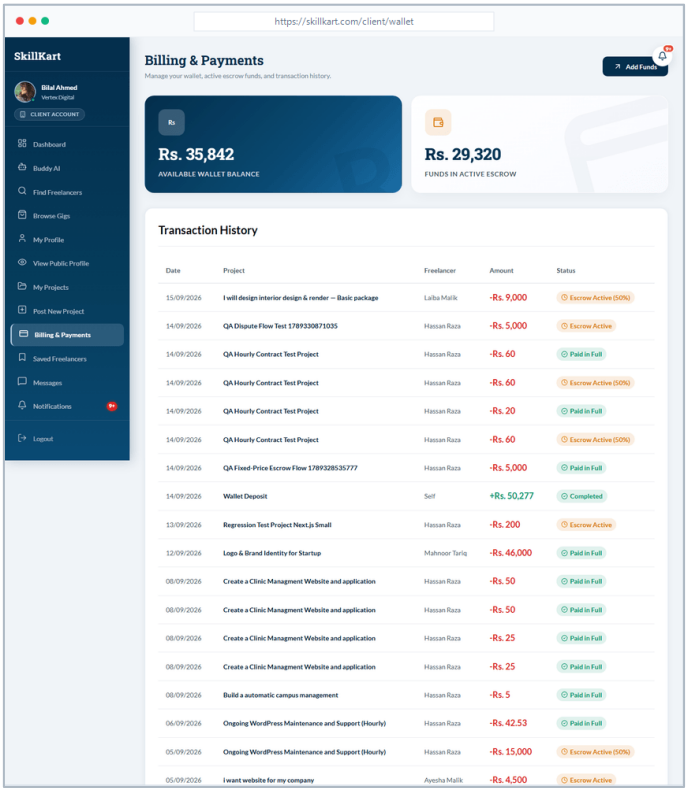


Figure 19.11 — Client wallet: balance, escrow, and transaction history

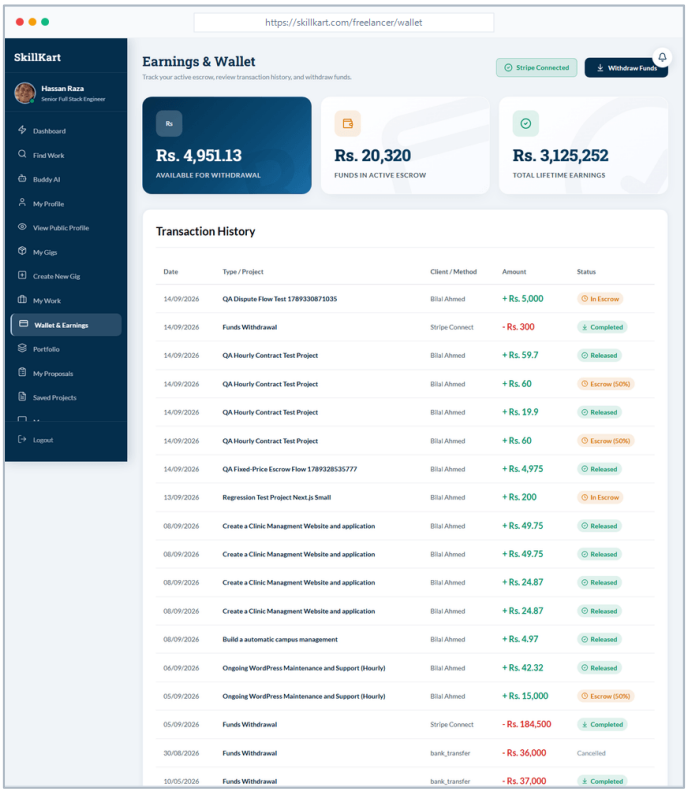


Figure 19.12 — Freelancer wallet and earnings overview

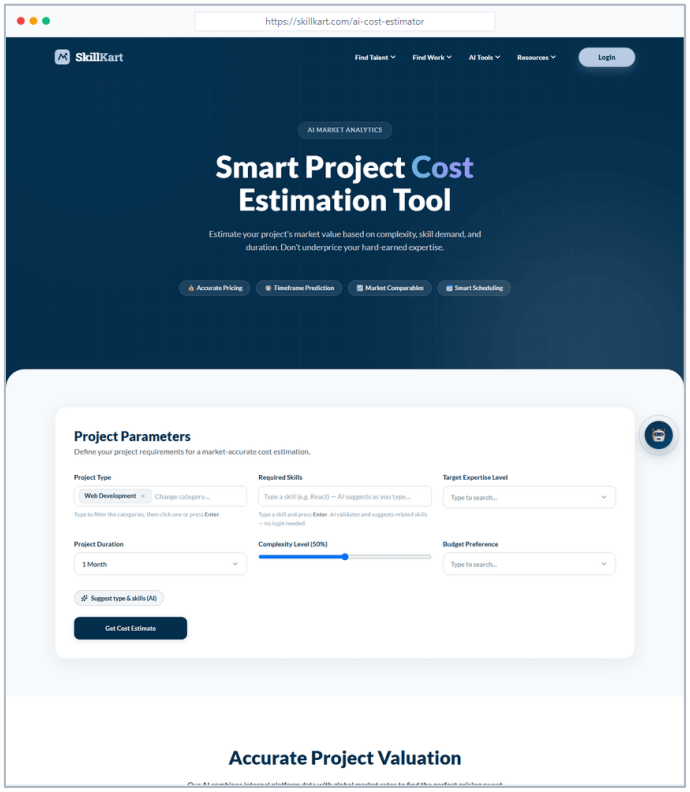


Figure 19.13 — AI-powered project cost estimator

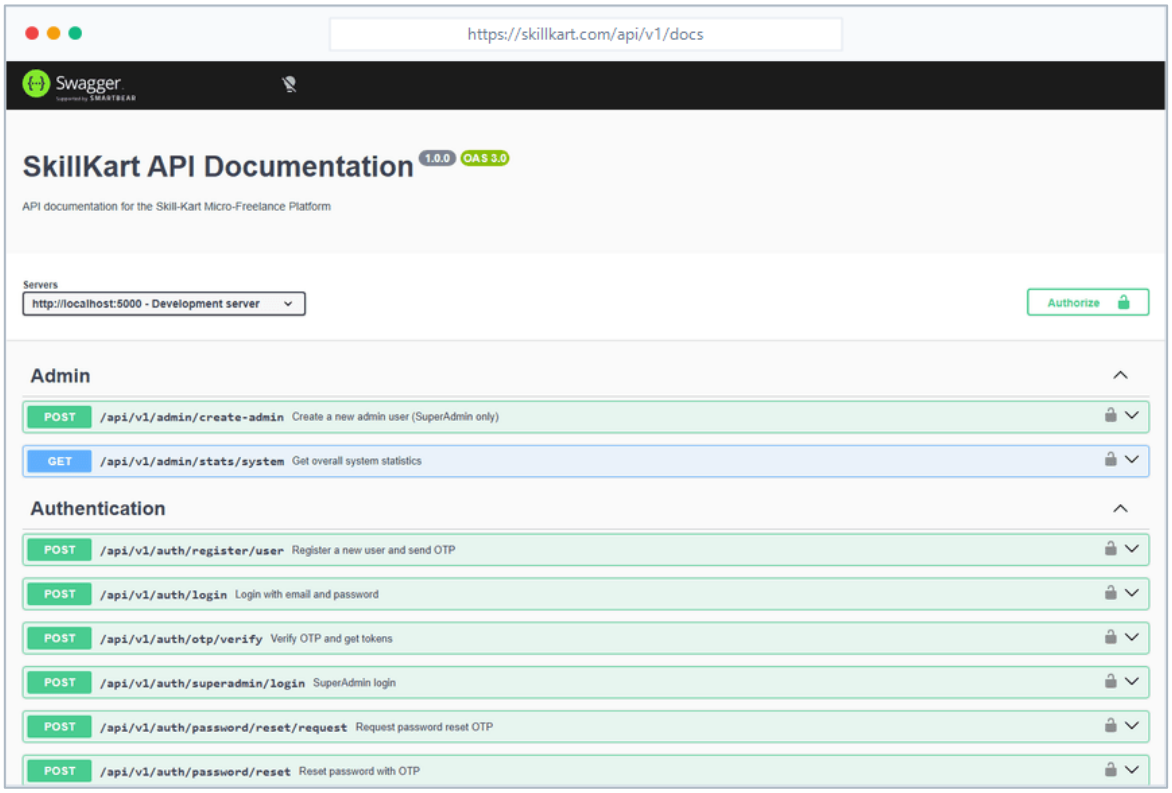


Figure 19.14 — Interactive API documentation (Swagger UI)

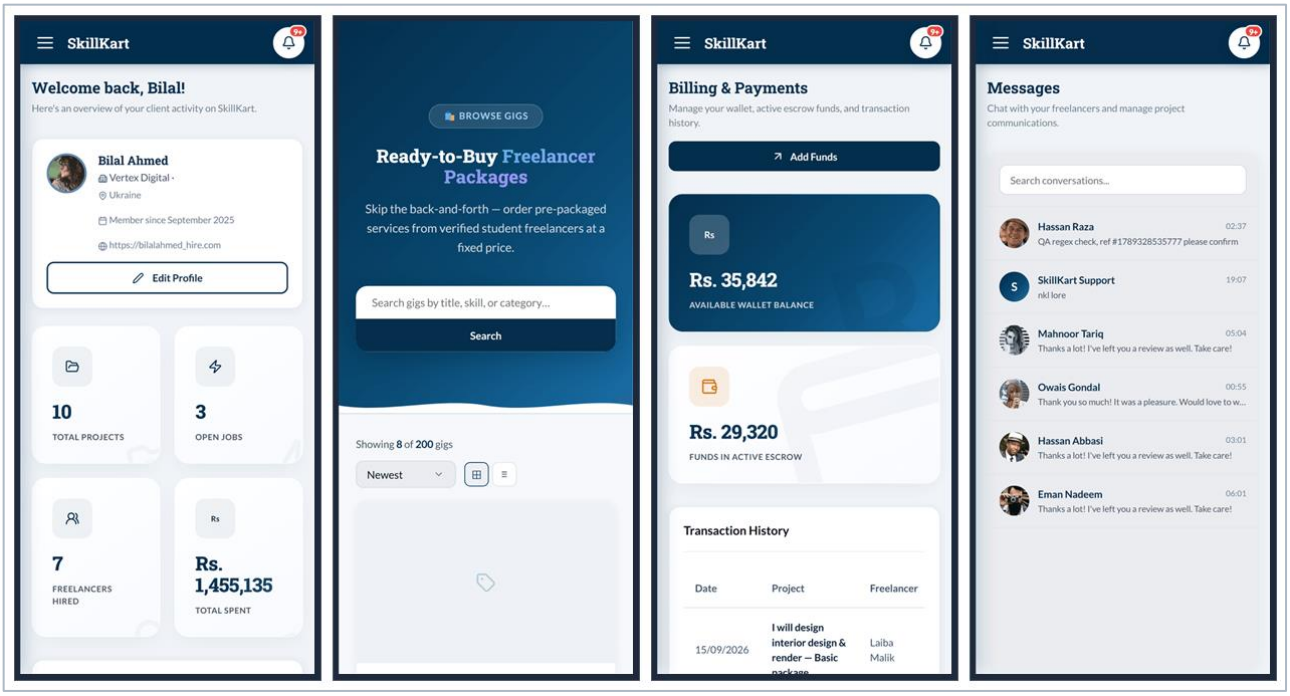


Figure 19.15 — Native Android application: client mobile workspace

Client mobile journey: dashboard overview, gig browsing, wallet management, and direct messaging.

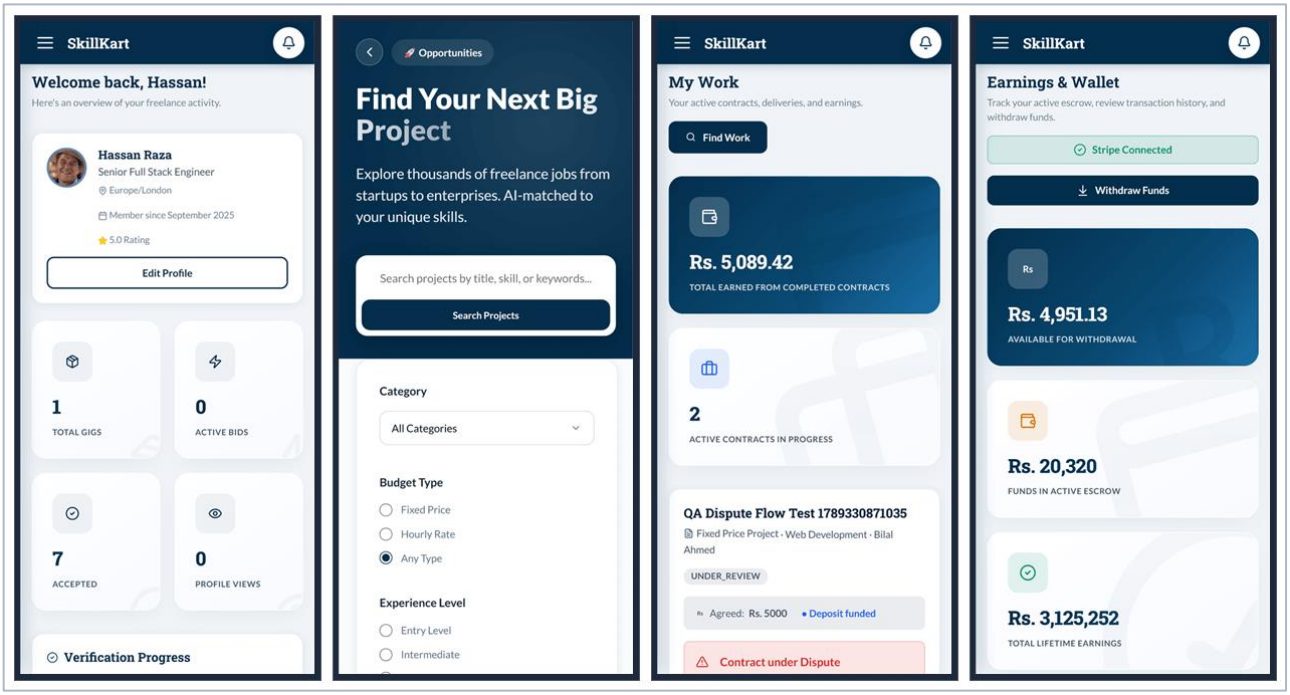


Figure 19.16 — Native Android application: freelancer mobile workspace

Freelancer mobile journey: dashboard overview, project search, active contract tracking, and wallet earnings.

19.6 Known Limitations

For completeness and honesty, the delivered system has the following known limitations. None changes the correctness of the money and contract flows.

- **No automated regression suite** (Section 18.9).
- **Polling, not push, for chat** — latency of a few seconds and extra requests (Section 12.18).

- **Gig orders have no contract**, so they have no milestone, automatic release, or dispute path (Section 12.9).
- **The CMS** is back-end only: pages can be authored and scheduled through the API, but there is no editor screen and no public page renders them.
- **Admin push notifications** and **automatic identity verification** were not built.
- **Several administrator settings** are stored but reserved (Table 14.2).
- **Redis** is connected at start-up but not yet used for caching.

20. Challenges Faced

Several substantive technical challenges were encountered and solved during the architectural design and implementation of SkillKart. The six principal challenges and their engineering solutions are detailed below.

20.1 Multi-Provider AI Routing and Resilient Graceful Degradation

Challenge. Free-tier and commercial LLM APIs exhibit unpredictable rate-limiting, variable latency, and intermittent downtime. Direct dependency on any single provider (such as Gemini or Claude) would cause critical user-facing flows (proposal drafting, skill evaluation, CV parsing) to crash during third-party outages.

Solution. A centralised AI provider router was engineered with automatic priority-based failover across six distinct backends: Gemini, DeepSeek, Qwen, Groq, Claude, and local Ollama instances. Each capability defines fallback priorities and strict execution timeouts. In the event of complete upstream failure, the router returns a controlled HTTP 503 response wrapped in an operational error handler, preventing application crashes and prompting an informative user notification while preserving form data.

Key Insight. External AI models must be treated as volatile non-blocking enhancements rather than hard runtime dependencies.

20.2 Concurrency and Financial Correctness Without Multi-Document Transactions

Challenge. Managing escrow locks, milestone fund releases, and wallet withdrawals requires strict atomicity and financial consistency. In a standalone MongoDB deployment without distributed replica-set transactions, concurrent client requests or overlapping webhooks could trigger double-credit conditions or negative balances.

Solution. The financial workflows were refactored into atomic single-document operations using guarded queries (`findOneAndUpdate` with pre-condition filters on current balance and state). For Stripe withdrawals, an optimistic reservation pattern was implemented: the wallet is debited immediately before dispatching the external API transfer, with automated compensatory refunds executed if the external gateway call fails. In-memory Mongoose documents are synchronised explicitly to prevent stale overwrites.

Key Insight. Financial integrity in distributed NoSQL environments is achieved through conditional single-document updates and reversible multi-phase state machines.

20.3 Session Integrity and Refresh-Token Family Invalidation

Challenge. Implementing rotating refresh tokens with strict reuse detection introduced race conditions in multi-tab browser sessions. When multiple tabs initiated simultaneous background token refresh requests, benign second tabs would present already-rotated tokens, mistakenly triggering the server's token-theft detection and logging out active users.

Solution. The backend rotation logic was augmented with a 10-second grace window during which an immediately superseded token remains acceptable for renewal while still invalidating any older historical tokens. Concurrently, frontend API interceptors were refactored to queue in-flight requests during active token rotation, ensuring that all concurrent queries await and reuse a single refreshed JWT credential.

Key Insight. Strict cryptographic rotation must accommodate browser-level network concurrency through managed grace periods and request queue synchronisation.

20.4 Edge AI Proctoring and Computer Vision on Android WebViews

Challenge. The skill-verification subsystem requires real-time proctoring to detect face presence without streaming private camera feeds to backend servers. Initial desktop implementations relying on WebGL GPU acceleration failed silently inside Android Capacitor WebViews, causing the proctoring engine to misreport camera availability and freeze frame analysis.

Solution. The proctoring pipeline was migrated to Google's MediaPipe Face Landmarker configured with an explicit CPU delegate on Android. A frame-analysis throttle running at 700 ms intervals was introduced, coupled with an active heartbeat monitor that pauses violation countdowns during background app switches or camera frame pauses. Strictness is dynamically governed via a centralised backend parameter.

Key Insight. Edge computer vision inside mobile WebViews must prioritise predictable CPU execution and stateful frame pacing over raw GPU pipelines.

20.5 Cross-Origin Cookie Security and Proxy Trust in Distributed Deployment

Challenge. In the production deployment topology, the React frontend (hosted on Vercel) and the Express API (hosted on Railway) operate across distinct origins. Initial deployments experienced authentication loops where administrative session cookies were dropped by modern browser cookie security policies.

Solution. The administrative authentication middleware was updated to issue cookies with `SameSite=None` and `Secure` attributes, paired with Express `trust proxy` configuration. Because reverse proxies terminate TLS at the platform perimeter, enabling proxy trust allows the backend to accurately identify secure HTTPS requests and preserve cookie delivery across cloud domains.

Key Insight. Cross-domain decoupled frontends require explicit reverse-proxy protocol recognition to satisfy modern third-party cookie constraints.

20.6 Transactional Email Reliability Under Cloud SMTP Egress Restrictions

Challenge. User registration, password recovery, and administrative 2FA rely strictly on one-time passcode verification. Standard SMTP connections on ports 465 and 587 functioned reliably in local development but were blocked by cloud platform egress firewalls to prevent spam abuse.

Solution. An extensible email provider factory was introduced, decoupling business logic from SMTP transports. For production environments, an HTTPS-based Gmail API transport was implemented, bypassing outbound port filtering entirely. The provider architecture allows instant environment-driven switching between direct API transports, sandbox sinks, and standard SMTP.

Key Insight. Production delivery of critical operational credentials requires HTTP/REST gateway transports resilient to cloud port-filtering policies.

21. Future Enhancements

SkillKart is feature-complete for its intended scope, and the limitations in Sections 16.10 and 19.6 define the natural next steps. They are grouped below by theme, roughly in order of recommended priority within each group.

21.1 Quality and Operations

- **An automated test suite.** Backend unit and integration tests for the escrow, contract, and authentication logic (using a test runner with HTTP-level checks), and a committed set of end-to-end browser tests derived from the verification scripts of Chapter 18, run in continuous integration. This is the single most valuable addition.
- **Observability.** Structured logging with a log library (one is already a declared dependency), log retention policies, and metrics and alerting for failures and latency.
- **Caching and scale.** Use the already-connected Redis for caching hot reads and for a shared rate-limit store, so that the application can run as several instances; move the administrator one-time-code store out of process memory for the same reason.
- **Load and security testing**, including an independent penetration test before a public launch.

21.2 Product Features

- **Real-time communication.** WebSocket-based chat and WebRTC video calls, replacing polling.
- **iOS.** The existing Capacitor project can be extended to iOS with comparatively little additional work, given a macOS build environment.
- **Disputes and contracts for gig orders**, giving the fixed-package route the same milestone, automatic-release, and dispute protections as the proposal-based route.
- **Hourly approvals.** An optional client approval step for logged time, for clients who want it.
- **Notification preferences** by event type, and push notifications for administrators.
- **A CMS editor** in the admin panel, so that the existing back end has a screen to author pages.
- **Multi-language and multi-currency support.**
- **Richer analytics**, including dispute analytics and configurable reports for administrators and users.

21.3 AI

- **A circuit breaker** for failing providers, so that a known-down provider is skipped for a cool-down instead of being retried on every request.
- **Embedding-based semantic matching** to complement the rules-based engine for deeper freelancer-to-job matching.

- **Grading against the stored answer key**, so that multiple-choice questions are scored deterministically rather than on the grader's own judgement, and server-side enforcement of test time limits.
- **AI-assisted dispute summaries**, automated profile-improvement coaching, and fraud detection.

21.4 Trust and Security

- **Automatic identity verification** through a verification provider, replacing the manual administrator flag.
- **A multi-administrator role model** — finance, support, moderator, and analytics roles — using the data model and route permissions already in place, with a persisted one-time-code store.
- **Shorter-lived access tokens**, a cryptographically secure code generator, and security headers on the frontend host (Section 16.10).

22. Conclusion

SkillKart set out to build more than a job board: a **secure, intelligent, and trustworthy** freelance marketplace that lowers the barrier for new freelancers while protecting everyone who takes part. The completed platform achieves this. It unifies the two dominant freelancing models — gigs and jobs with proposals — and adds a third route, custom offers in chat. It augments them with an integrated AI layer, a wallet-native escrow system that fits the way each kind of work is actually paid for, layered verification centred on a proctored skill test, a two-way review system, structured dispute resolution, and a comprehensive, isolated admin panel. It is delivered not only as a web application but as two native Android applications built from the same codebase.

Technically, the project demonstrates sound engineering across the stack: a decoupled client–server architecture; a strictly layered backend; explicit state machines for the contract, milestone, and payment lifecycles; money operations that are correct under concurrency without the help of database transactions; a resilient multi-provider AI subsystem that degrades gracefully instead of failing; and defence-in-depth security with rotating tokens, validation, rate limiting, bot detection, and an audit trail. Each of the nine objectives was achieved and demonstrated on a realistic, seeded dataset.

The project was also a substantial education in building a production-oriented application. The most instructive lessons were not the features themselves but the defects: a refresh-token bug that appeared only when real tabs were opened over real time; a camera guard that worked on a desktop and silently detected nothing on a phone; a redirect loop that only a live phone-sized run could expose; a push service that was perfectly configured and never worked; and a cookie that behaved correctly everywhere except across two domains. Each reinforced the same point — that a system must be exercised as its users will exercise it, in the environments in which they will use it — and each left a convention in the codebase that prevents its recurrence.

The report has also tried to be candid about what remains: the absence of an automated regression suite, the places where a designed control is not yet enforced, and the hardening that a public launch would warrant. These are recorded in Chapters 16, 19, and 21 not as failures but as an honest account of a system that is complete for its purpose and clear about its next steps. SkillKart stands as a complete, submission-ready Final Year Project that reflects both the breadth of its features and the depth of its engineering.

23. References

References are numbered in the order of the reference list and cited in the text in square brackets. Online documentation was consulted during development in 2026; each entry gives the canonical address of the resource.

[1] MongoDB, Inc., "MongoDB Manual," *MongoDB Documentation*. [Online]. Available: https://www.mongodb.com/docs/manual/

[2] OpenJS Foundation, "Node.js Documentation." [Online]. Available: https://nodejs.org/en/docs/

[3] OpenJS Foundation, "Express — Node.js web application framework." [Online]. Available: https://expressjs.com/

[4] Meta Open Source, "React Documentation." [Online]. Available: https://react.dev/

[5] Vite contributors, "Vite — Next Generation Frontend Tooling." [Online]. Available: https://vite.dev/

[6] Automattic, Inc., "Mongoose ODM Documentation." [Online]. Available: https://mongoosejs.com/docs/

[7] Stripe, Inc., "Stripe API Reference" and "Stripe Connect Documentation." [Online]. Available: https://docs.stripe.com/api and https://docs.stripe.com/connect

[8] Google, "Gemini API Documentation." [Online]. Available: https://ai.google.dev/gemini-api/docs

[9] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[10] OWASP Foundation, "OWASP Top Ten Web Application Security Risks." [Online]. Available: https://owasp.org/www-project-top-ten/

[11] Fiverr International Ltd., "Fiverr." [Online]. Available: https://www.fiverr.com/

[12] Upwork Global Inc., "Upwork." [Online]. Available: https://www.upwork.com/

[13] Freelancer Technology Pty Ltd., "Freelancer.com." [Online]. Available: https://www.freelancer.com/

[14] DeepSeek, "DeepSeek API Documentation." [Online]. Available: https://api-docs.deepseek.com/

[15] Ionic, "Capacitor Documentation." [Online]. Available: https://capacitorjs.com/docs

[16] Google AI Edge, "MediaPipe Face Landmarker guide." [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker

[17] Google, "Firebase Cloud Messaging." [Online]. Available: https://firebase.google.com/docs/cloud-messaging

[18] Cloudinary Ltd., "Cloudinary Documentation." [Online]. Available: https://cloudinary.com/documentation

[19] Google, "reCAPTCHA v3." [Online]. Available: https://developers.google.com/recaptcha/docs/v3

[20] Supabase, Inc., "Supabase Auth Documentation." [Online]. Available: https://supabase.com/docs/guides/auth

[21] J. Kässi and V. Lehdonvirta, "Online Labour Index: Measuring the online gig economy for policy and research," *Technological Forecasting and Social Change*, vol. 137, pp. 241–248, Dec. 2018.

[22] P. A. Pavlou and D. Gefen, "Building effective online marketplaces with institution-based trust," *Information Systems Research*, vol. 15, no. 1, pp. 37–59, Mar. 2004.

[23] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. of Information and Computer Science, Univ. of California, Irvine, CA, USA, 2000.

[24] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conf. (FREENIX Track)*, Monterey, CA, USA, Jun. 1999, pp. 81–91.

[25] Google, "Gmail API." [Online]. Available: https://developers.google.com/gmail/api

[26] OpenAPI Initiative, "OpenAPI Specification." [Online]. Available: https://spec.openapis.org/oas/latest.html

[27] Anthropic, "Claude API Documentation." [Online]. Available: https://docs.anthropic.com/

[28] Groq, Inc., "GroqCloud Documentation." [Online]. Available: https://console.groq.com/docs

[29] Alibaba Cloud, "Qwen — Model Studio Documentation." [Online]. Available: https://www.alibabacloud.com/help/en/model-studio/

[30] Ollama, "Ollama — run large language models locally." [Online]. Available: https://ollama.com/

[31] Railway Corp., "Railway Documentation." [Online]. Available: https://docs.railway.com/

[32] Vercel, Inc., "Vercel Documentation." [Online]. Available: https://vercel.com/docs

[33] MongoDB, Inc., "MongoDB Atlas Documentation." [Online]. Available: https://www.mongodb.com/docs/atlas/

[34] Microsoft, "Playwright Documentation." [Online]. Available: https://playwright.dev/docs/intro

[35] Tailwind Labs, "Tailwind CSS Documentation." [Online]. Available: https://tailwindcss.com/docs

[36] P. Resnick and R. Zeckhauser, "Trust among strangers in Internet transactions: Empirical analysis of eBay's reputation system," in *The Economics of the Internet and E-Commerce*, ser. Advances in Applied Microeconomics, vol. 11, M. R. Baye, Ed. Amsterdam, The Netherlands: Elsevier Science, 2002, pp. 127–157.

[37] Helmet.js contributors, "Helmet — help secure Express apps with various HTTP headers." [Online]. Available: https://helmetjs.github.io/

[38] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," Internet Engineering Task Force, RFC 6749, Oct. 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749

[39] Mozilla, "Set-Cookie: SameSite attribute," *MDN Web Docs*. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie

[40] Google, "WebView — Android Developers." [Online]. Available: https://developer.android.com/reference/android/webkit/WebView

24. Appendix

Appendix A — API Summary

The backend exposes a versioned REST API under `/api/v1`. Interactive documentation for the annotated routes is served by Swagger at `/api/v1/docs`. Table A.1 lists the route groups with their endpoint counts; the counts add up to 317.

Table A.1 — API route groups

Route group	Endpoints	Purpose
`/auth`	28	Registration, login, OTP, token refresh, social sign-in, role change, password and email change, account deactivation and deletion, push-token registration
`/users`	10	Administrator-only user management (list, search, create, update, block, role)
`/freelancers`	16	Freelancer profile, onboarding submission, CV parsing, portfolio, saved jobs, public search
`/clients`	17	Client profile, onboarding draft and submission, uploads, saved freelancers, AI industry recommendation
`/jobs`	15	Job posting and management, search, AI category suggestions, gig-order delivery and revision, job review
`/gigs`	10	Gig creation, management, banner upload, ordering, cancelling an unpaid order
`/proposals`	10	Proposal submission, eligibility pre-check, AI draft, edit, withdraw, accept, reject
`/offers`	4	Custom offers sent in chat: create, accept, decline, withdraw
`/contracts`	7	Contract lifecycle: list, detail, dispute, end request and response, history
`/deliveries`	3	Delivery submission and review
`/time-tracking`	4	Timer start and stop, manual logs, log list for hourly contracts
`/payments`	13	Milestone funding, hourly escrow top-up, gig-order deposit and final payment, wallet checkout and withdrawal, Stripe Connect
`/messages`	4	Conversations, messages, attachment upload
`/notifications`	6	List, unread count, mark read, delete
`/reviews`	4	Create, read, update, delete reviews
`/testimonials`	3	A freelancer's success story
`/reports`	1	File a report about a profile, job, gig, or review
`/skills`	13	Skill taxonomy, graph, resolution, expansion, validation, match score
`/skill-tests`	4	Proctoring configuration, test generation, submission, history
`/matchmaking`	2	Rules-based freelancer matching
`/ai`	6	Public AI tools: Buddy, cost estimate, brief suggestion, skill validation and expansion, text generation
`/public`	18	Public search, profiles, statistics, categories, testimonials, contact, newsletter
`/cms`	13	Content management (back-end only)

Route group	Endpoints	Purpose
`/admin`	11	Legacy administrator routes, including taxonomy administration
`/superadmin`	2	SuperAdmin dashboard and health
`/admin-panel`	93	The isolated admin panel (Chapter 14)

Six further routes exist outside the `/api/v1` prefix: `GET /health`, `GET /`, `GET /sitemap.xml`, `GET /robots.txt`, and the signature-verified webhooks `POST /webhooks/stripe` and `POST /webhooks/supabase`.

Appendix B — Folder Structure

```
Skill-Kart----Micro-Freelance-Platform/
├── Apk/                      Signed release APKs (main app builds and admin app)
├── Backend/
│   ├── scripts/              Demo seeder, SuperAdmin setup, one-off migrations
│   └── src/
│       ├── config/           Environment, database, email, Redis, Supabase, Swagger
│       ├── controllers/      HTTP request handlers (35, including 9 in adminPanel/)
│       ├── middleware/       Authentication, RBAC, validation, rate limits, logging (10)
│       ├── models/           Mongoose schemas (40)
│       ├── routes/           API route definitions (26)
│       ├── services/         Business logic: escrow, AI, matching, email, push, ... (49 files)
│       │   ├── ai/           Provider router and the six providers
│       │   └── email/        Email provider registry, factory, providers, templates
│       ├── utils/            Shared helpers (JWT, errors, constants, gating, sanitising, ...)
│       ├── validators/       express-validator schemas (11)
│       ├── webhooks/         Stripe and Supabase webhooks
│       ├── scripts/          seedSkillGraph.js (taxonomy graph seed)
│       ├── app.js            Express application assembly
│       └── server.js         HTTP bootstrap and start-up checks
├── Front-end/
│   ├── android/              Native Android project (Capacitor): client and freelancer app
│   ├── android-admin/        Native Android project (Capacitor): administrator app
│   ├── assets/, assets-admin/   Source icon and splash images for each native build
│   ├── public/               Static assets: icons, manifest, robots.txt, Open Graph image
│   ├── scripts/              cap-admin.cjs — drives the admin app's Capacitor CLI workaround
│   └── src/
│       ├── pages/            public, client, freelancer, adminPanel, common, admin
│       ├── Components/       Reusable UI components
│       ├── context/          Auth, AdminAuth, Client, Freelancer, Notification providers
│       ├── api/              Axios resource modules (one per resource)
│       ├── layouts/          Route layouts and shells
│       ├── routes/           AppRoutes.jsx, AdminAppRoutes.jsx
│       ├── hooks/            Route guards, back button, push, deep link, polling, ...
│       ├── utils/            Token manager, toast, logger, platform detection, downloads, ...
│       ├── constants/        Shared constants (job categories)
│       └── data/             Static reference data
└── docs/                     Report, presentation, guides, and the thesis build tooling
```

Appendix C — Environment Variables (without secrets)

The backend is configured entirely through environment variables. Secret **values** are never committed; only `.env.example` files are tracked. The minimum configuration is a database URI and two JWT secrets for the backend, and the API address for the frontend.

Table C.1 — Environment variable groups

Group	Variables
Core	`NODE_ENV`, `PORT`, `API_VERSION`
Database	`MONGODB_URI`
Redis (optional)	`REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`, `REDIS_DB`

Group	Variables
Authentication	`JWT_SECRET`, `JWT_REFRESH_SECRET`, `JWT_EXPIRE` (default 7d), `JWT_REFRESH_EXPIRE` (default 30d), `ENCRYPTION_KEY`, `SUPERADMIN_CREDENTIALS_PATH`
URLs and CORS	`CORS_ORIGIN`, `FRONTEND_URL`, `CLIENT_URL`, `PUBLIC_SITE_URL`
Email and OTP	`EMAIL_PROVIDER` (`smtp` or `gmail`), `EMAIL_ENABLED`, `EMAIL_SANDBOX`, `EMAIL_SANDBOX_RECIPIENT`, `MARKETING_EMAIL`; SMTP: `SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM_EMAIL`, `SMTP_FROM_NAME`; Gmail API: `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_REFRESH_TOKEN`, `GOOGLE_REDIRECT_URI`, `GOOGLE_EMAIL`; `OTP_EXPIRY_MINUTES`, `OTP_LENGTH`
Payments	`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_DUMMY`
AI providers	`GEMINI_API_KEY`, `GEMINI_MODEL_LITE`, `GEMINI_MODEL_HEAVY`, `DEEPSEEK_API_KEY`, `DEEPSEEK_BASE_URL`, `DEEPSEEK_MODEL_LITE`, `DEEPSEEK_MODEL_HEAVY`, `DEEPSEEK_TIMEOUT_MS`, `QWEN_API_KEY`, `QWEN_BASE_URL`, `QWEN_MODEL`, `QWEN_TIMEOUT_MS`, `GROQ_API_KEY`, `ANTHROPIC_API_KEY`, `OLLAMA_BASE_URL`, `OLLAMA_MODEL`, `OLLAMA_TIMEOUT_MS`, `OLLAMA_DISABLED`, and `AI_PRIORITY_<PROVIDER>` for each of the six providers
Skill tests	`SKILL_TEST_PROCTOR_STRICTNESS` (0–1, default 0.5), `SKILL_TEST_EASY_MODE` (testing only; refused in production)
Admin panel	`ADMIN_JWT_SECRET`, `ADMIN_EMAIL`, `ADMIN_PASSWORD_HASH`, `ADMIN_SESSION_TIMEOUT`, `ADMIN_OTP_EXPIRY`, `ADMIN_MAX_OTP_ATTEMPTS`
Push notifications	`FIREBASE_SERVICE_ACCOUNT_PATH` or `FIREBASE_SERVICE_ACCOUNT_JSON`
reCAPTCHA	`RECAPTCHA_SECRET_KEY`, `RECAPTCHA_MIN_SCORE`, `RECAPTCHA_TIMEOUT_MS`
Rate limiting and uploads	`RATE_LIMIT_WINDOW_MS`, `RATE_LIMIT_MAX_REQUESTS`, `WEBHOOK_SECRET`
Optional services	`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`
Frontend (public, `VITE_` prefix)	`VITE_API_URL`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_RECAPTCHA_SITE_KEY`, `VITE_ENVIRONMENT`; build-time: `VITE_APP_TARGET`, `VITE_PUSH_NOTIFICATIONS_ENABLED`

A few variables that appear in the example file — for instance `OTP_MAX_ATTEMPTS`, `MAX_LOGIN_ATTEMPTS`, `BCRYPT_ROUNDS`, and `LOG_LEVEL` — are placeholders that the current code does not read; the corresponding values are fixed in code (Table 12.1).

Appendix D — Major Screen & Interface Verification Matrix

The SkillKart web platform and native mobile applications encompass 32 primary web views and 25 mobile screens across four user roles (Visitor, Client, Freelancer, and Administrator). Table D.1 documents the complete interface catalog, outlining route paths, security controls, and primary user interactions, with visual implementations presented in Chapter 19 (Figures 19.1–19.16) and Chapter 15 (Figure 15.4).

Table D.1 — Interface and screen verification matrix

Screen / View	Role	Route / Path	Access Control	Primary Capabilities	Reference
Public Home	Visitor	`/`	Public	Marketplace hero, category directory, featured talent, search entry	Fig. 19.1
Browse Gigs	Visitor / All	`/browse-gigs`	Public	Filterable catalog of pre-packaged freelancer services, price sorting	Fig. 19.2
Gig Detail	Visitor / All	`/gigs/:id`	Public	Tiered package selection (Basic/Standard/Premium), freelancer bio	Fig. 19.3
Browse Projects	Visitor / All	`/browse-projects`	Public	Custom job opportunities, taxonomy-expanded skill filters, budgets	Fig. 19.4
Job Detail	All	`/jobs/:id`	Public / Apply Gate	Detailed brief, client history, proposal submission entry	Fig. 19.5
Client Post Job	Client	`/client/jobs/new`	Client Role	Five-step job wizard, budget allocation, AI description suggestions	Fig. 19.6
Review Proposals	Client	`/client/jobs/:id/proposals`	Client Role	Comparative proposal evaluation, chat initiation, milestone negotiation	Fig. 19.7
My Projects	Client	`/client/projects`	Client Role	Contract state tracking, milestone funding status, delivery review	Fig. 19.8
Freelancer Dashboard	Freelancer	`/freelancer/dashboard`	Freelancer Role	Active contracts, earnings tally, Skill Verification Center access	Fig. 19.9
Active Contracts	Freelancer	`/freelancer/contracts`	Freelancer Role	Milestone deliverable submissions, time tracker logs, revision history	Fig. 19.10
Client Wallet	Client	`/client/wallet`	Client Role	Stripe deposit checkout, escrow ledger, locked fund balances	Fig. 19.11
Freelancer Wallet	Freelancer	`/freelancer/wallet`	Freelancer Role	Net earnings, available balances, Stripe Connect withdrawal requests	Fig. 19.12
AI Cost Estimator	Visitor / All	`/ai-cost-estimator`	Rate Limited	Interactive AI budget estimation based on project scope and stack	Fig. 19.13
Swagger API Docs	Developer / Public	`/api/v1/docs`	Public	Interactive OpenAPI documentation for all 317 endpoints	Fig. 19.14
Client Mobile App	Client	Native Android	Client Role	Project oversight, gig ordering, mobile wallet top-up, push notifications	Fig. 19.15
Freelancer Mobile App	Freelancer	Native Android	Freelancer Role	Job browsing, proposal dispatch, timer logs, mobile wallet withdrawals	Fig. 19.16
Native Auth & Drawer	Mobile (All)	Native Android	Public / Role	Native login, registration, biometric drawer navigation	Fig. 15.4

Appendix E — Database Collections

The 40 collections are, by domain:

- **Identity and access:** `User`, `RefreshToken`, `OTP`, `AdminUser`, `AdminSession`, `ClientOnboardingDraft`, `OnboardingProgress`
- **Profiles and files:** `FreelancerProfile`, `ClientProfile`, `FileUpload`
- **Marketplace:** `Job`, `Gig`, `Proposal`, `CustomOffer`
- **Engagement and payments:** `Contract`, `Milestone`, `ContractHistory`, `Delivery`, `TimeLog`, `Payment`, `Withdrawal`
- **Communication and feedback:** `Conversation`, `Message`, `Notification`, `Review`, `Testimonial`
- **Skills and verification:** `SkillNode`, `SkillRelationship`, `TaxonomyProposal`, `SkillTestAttempt`
- **Administration and operations:** `AdminAuditLog`, `Report`, `PlatformSettings`, `SystemSettings`, `ApiLog`, `ErrorLog`, `EmailLog`
- **Content and public forms:** `CMSContent`, `NewsletterSubscriber`, `ContactMessage`

Of these, `AdminUser` and `OnboardingProgress` are defined but not currently used by any code path (Sections 14.2 and 12.2).

Appendix F — Running the Project

The project runs locally with the commands below, from the repository root unless noted.

```
npm run install:all                          install root, Backend and Front-end dependencies
npm start                                    run Backend and Front-end together
npm --prefix Backend run dev                 Backend only (nodemon, port 5000)
npm --prefix Front-end run dev               Front-end only (Vite, port 3001)
npm --prefix Backend run setup:superadmin  create or rotate the encrypted SuperAdmin credential
npm --prefix Backend run seed:demo           load the full demo dataset (DESTRUCTIVE: asks you to
type the database name first)
npm --prefix Backend run seed:demo:small   load a small dataset for quick checks
npm --prefix Front-end run build             production web build
npm --prefix Front-end run cap:android     build, sync and open the Android project
```

The backend's health is at `/health` and its interactive documentation at `/api/v1/docs`. The demo marketplace accounts are listed in Table 18.2; the admin panel is a separate login at `/admin-panel/login`, using the credentials configured in the backend's environment (`ADMIN_EMAIL` and `ADMIN_PASSWORD_HASH`) and an emailed code. With no Stripe key, the payment flow runs in simulated mode.

End of Report

SkillKart — An AI-Powered Freelance Marketplace Platform